



# AX SDK 使用说明

文档版本：V2.1

发布日期：2024/07/09

# 目 录

|                        |    |
|------------------------|----|
| 前 言 .....              | 11 |
| 修订历史 .....             | 12 |
| 1 准备工作 .....           | 13 |
| 1.1 编译工具链 .....        | 13 |
| 1.1.1 配置安装环境（可选） ..... | 13 |
| 1.1.2 安装工具链 .....      | 15 |
| 1.2 Linux 内核代码 .....   | 17 |
| 1.2.1 方法一 .....        | 17 |
| 1.2.2 方法二 .....        | 17 |
| 1.3 U-boot 源码 .....    | 18 |
| 1.3.1 方法一 .....        | 18 |
| 1.3.2 方法二 .....        | 18 |
| 1.4 Atf 源码 .....       | 18 |
| 1.4.1 方法一 .....        | 18 |
| 1.4.2 方法二 .....        | 19 |
| 1.5 Optee 源码 .....     | 19 |
| 1.5.1 方法一 .....        | 19 |
| 1.5.2 方法二 .....        | 19 |
| 2 安装 SDK .....         | 21 |
| 2.1 解压缩 SDK 包 .....    | 21 |
| 2.2 展开 SDK 包内容 .....   | 21 |
| 2.3 介绍 SDK 包内容 .....   | 21 |

|                                |           |
|--------------------------------|-----------|
| <b>3 编译版本 .....</b>            | <b>23</b> |
| 3.1 编译步骤.....                  | 23        |
| 3.2 工程支持.....                  | 25        |
| 3.3 atf 编译.....                | 25        |
| 3.4 Uboot 编译 .....             | 26        |
| 3.5 Kernel 编译 .....            | 26        |
| 3.6 rootfs 编译 .....            | 27        |
| 3.6.1 AX620E 文件系统编译 .....      | 28        |
| 3.7 Optee 编译.....              | 28        |
| <b>4 分配 Memory 和存储空间 .....</b> | <b>29</b> |
| 4.1 存储空间概况.....                | 29        |
| 4.2 内存空间分配的调整 .....            | 29        |
| 4.2.1 系统的内存大小调整.....           | 30        |
| 4.2.2 修改 CMM 的地址和大小.....       | 30        |
| 4.3 FLASH 存储空间的分配 .....        | 31        |
| 4.4 调整分区大小.....                | 33        |
| 4.5 新建或删除分区 .....              | 34        |
| <b>5 根文件系统 .....</b>           | <b>37</b> |
| 5.1 根文件系统简介 .....              | 37        |
| 5.2 利用 BusyBox 制作根文件系统.....    | 38        |
| 5.2.1 获取 BusyBox 源码.....       | 38        |
| 5.2.2 配置 BusyBox .....         | 38        |
| 5.2.3 编译和安装 BusyBox .....      | 39        |
| 5.2.4 制作根文件系统.....             | 40        |
| 5.3 使用 SDK 中的 BusyBox .....    | 41        |

|                                    |           |
|------------------------------------|-----------|
| 5.3.1 BusyBox 路径 .....             | 41        |
| 5.3.2 BusyBox 的编译 .....            | 41        |
| 5.4 文件系统简介 .....                   | 42        |
| 5.4.1 JFFS2 .....                  | 42        |
| 5.4.2 SquashFS .....               | 42        |
| 5.4.3 EXT4 .....                   | 43        |
| 5.4.4 UBIFS .....                  | 43        |
| <b>6 日志系统 .....</b>                | <b>45</b> |
| 6.1 日志系统简介 .....                   | 45        |
| 6.2 syslogd 和 klogd .....          | 45        |
| 6.3 axsyslogd 和 axklogd .....      | 47        |
| 6.3.1 axsyslogd .....              | 47        |
| 6.3.2 axklogd .....                | 48        |
| 6.4 日志系统的部署 .....                  | 48        |
| 6.4.1 系统原生日志的部署 .....              | 49        |
| 6.4.2 SDK 自研日志系统的部署 .....          | 51        |
| 6.4.3 只读文件系统的日志部署 .....            | 56        |
| 6.5 集成自定义模块 Log 到日志系统 .....        | 59        |
| 6.5.1 集成应用程序 Log .....             | 59        |
| 6.5.2 集成驱动程序 Log .....             | 68        |
| 6.6 动态修改 Log 等级 .....              | 74        |
| 6.6.1 动态修改 Linux 内核原生 Log 等级 ..... | 74        |
| 6.6.2 SDK 自研日志系统 log 等级控制流程 .....  | 74        |
| 6.6.3 动态修改 user log 等级 .....       | 75        |
| 6.6.4 动态修改 kernel log 等级 .....     | 78        |

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>7 SPI NAND NOR Flash 工程指南 .....</b>             | <b>80</b> |
| 7.1 SPI NOR Flash 工程构建 .....                       | 80        |
| 7.1.1 修改 uboot 的分区信息 .....                         | 80        |
| 7.1.2 配置 kernel 以支持 squashfs 文件系统 .....            | 81        |
| 7.1.3 生成 rootfs 镜像 .....                           | 81        |
| 7.1.4 生成 spl 镜像 .....                              | 82        |
| 7.1.5 双 spi-nor flash 支持 .....                     | 84        |
| 7.1.6 NFS 挂载 .....                                 | 85        |
| 7.1.7 SPI NOR Flash SecondSource 物料支持 .....        | 86        |
| 7.2 SPI NAND Flash 工程创建 .....                      | 87        |
| 7.2.1 检查 AX620Q_nand_arm32_k419_defconfig 文件 ..... | 87        |
| 7.2.2 修改 uboot 的分区信息 .....                         | 87        |
| 7.2.3 生成 ubi rootfs .....                          | 89        |
| 7.2.4 制作 UBIFS .....                               | 90        |
| 7.2.5 SPI NAND Flash SecondSource 物料支持 .....       | 91        |
| <b>8 U-Boot 移植应用开发指南 .....</b>                     | <b>97</b> |
| 8.1 概述 .....                                       | 97        |
| 8.1.1 AX620E 启动流程 .....                            | 97        |
| 8.1.2 U-Boot 目录结构 .....                            | 98        |
| 8.2 移植 U-Boot .....                                | 101       |
| 8.2.1 基于 AX630C_emmc_arm64_k419 新建 board .....     | 101       |
| 8.2.2 基于 AX620E 参考 board 直接修改 .....                | 110       |
| 8.3 烧写 U-Boot .....                                | 111       |
| 8.3.1 AXDL 烧写 u-boot.bin .....                     | 111       |
| 8.3.2 AXDL repartiton .....                        | 114       |

|                                       |            |
|---------------------------------------|------------|
| 8.4 U-Boot Debug 方法 .....             | 116        |
| 8.4.1 TRACE32 设备工具简介 .....            | 116        |
| 8.4.2 使用 TRACE32 调试 U-Boot 程序 .....   | 117        |
| 8.5 附录 .....                          | 122        |
| 8.5.1 网络支持 .....                      | 122        |
| 8.5.2 SD 卡升级 .....                    | 125        |
| 8.5.3 SD 卡启动 .....                    | 126        |
| 8.5.4 SD 卡裸卡升级 .....                  | 127        |
| 8.5.5 U 盘升级 .....                     | 127        |
| 8.5.6 支持修改 xml 修改 partition 的大小 ..... | 128        |
| 8.5.7 文件系统 SPARSE 压缩格式镜像 .....        | 130        |
| 8.5.8 U-boot 加密功能 .....               | 131        |
| 8.5.9 回读分区数据 .....                    | 132        |
| <b>9 网络安全注意事项 .....</b>               | <b>136</b> |
| 9.1 Uboot 使用注意事项 .....                | 136        |
| 9.1.1 串口 .....                        | 136        |
| 9.1.2 命令行 .....                       | 137        |
| 9.2 Linux 及 Busybox 使用安全 .....        | 138        |
| 9.2.1 Root 账户 .....                   | 138        |
| 9.2.2 运行权限 .....                      | 140        |
| 9.2.3 SSH 服务 .....                    | 141        |
| 9.2.4 SCP 服务 .....                    | 141        |
| 9.2.5 SFTP 服务 .....                   | 142        |
| 9.2.6 Telnet 服务 .....                 | 142        |
| 9.2.7 文件权限 .....                      | 142        |

|                                          |            |
|------------------------------------------|------------|
| 9.3 SDK 驱动使用网络安全注意事项 .....               | 143        |
| 9.3.1 串口 .....                           | 143        |
| 9.3.2 USB.....                           | 143        |
| 9.3.3 Linux 使用安全注意事项.....                | 143        |
| 9.4 其他注意事项.....                          | 143        |
| 9.4.1 裸片烧写 .....                         | 143        |
| 9.4.2 SD 卡 mount 权限.....                 | 144        |
| 9.4.3 JTAG.....                          | 144        |
| <b>10 Security 开发指南 .....</b>            | <b>145</b> |
| 10.1 Security 方案内容 .....                 | 145        |
| 10.2 Secure Boot .....                   | 145        |
| 10.2.1 Secure boot 概述 .....              | 145        |
| 10.2.2 bootrom 加载校验 spl 镜像.....          | 146        |
| 10.2.3 spl 加载校验 uboot/atf/optee 镜像 ..... | 153        |
| 10.2.4 uboot 加载校验 kernel/dtb 镜像 .....    | 155        |
| 10.3 Optee 使用 .....                      | 157        |
| 10.3.1 Optee 编译.....                     | 157        |
| 10.3.2 Optee 地址相关配置 .....                | 158        |
| 10.3.3 CA/TA 编译 .....                    | 158        |
| 10.3.4 Optee debug .....                 | 158        |
| 10.3.5 Optee 相关路径说明 .....                | 159        |
| 10.3.6 HUK 配置 .....                      | 159        |
| 10.3.7 DIE ID .....                      | 159        |
| 10.4 平台安全配置 .....                        | 160        |
| 10.4.1 Firewall 简介 .....                 | 160        |

|                                 |            |
|---------------------------------|------------|
| 10.4.2 Firewall 配置 .....        | 160        |
| 10.5 Security 涉及的 Efuse 说明..... | 160        |
| 10.5.1 PUB_KEY_HASH .....       | 160        |
| 10.5.2 SEC_BOOT .....           | 161        |
| 10.5.3 SEC_SYS.....             | 161        |
| 10.5.4 EFUSE_SPACE .....        | 161        |
| 10.5.5 JTAG_DISABLE.....        | 161        |
| <b>11 应用与内核 Debug 使用说明.....</b> | <b>162</b> |
| 11.1 软硬件配置要求.....               | 162        |
| 11.1.1 软件要求.....                | 162        |
| 11.1.2 硬件要求.....                | 162        |
| 11.2 Linux 内核 ramoops.....      | 163        |
| 11.2.1 简介 .....                 | 163        |
| 11.2.2 使用方法.....                | 164        |
| 11.3 Linux memory 转储.....       | 164        |
| 11.4 gdb 调试应用程序 coredump .....  | 169        |
| 11.5 strace 调试应用程序.....         | 171        |
| <b>12 Clock 使用说明.....</b>       | <b>173</b> |
| 12.1 CPU .....                  | 173        |
| 12.2 DDR.....                   | 173        |
| 12.3 EMMC.....                  | 174        |
| 12.4 FLASH.....                 | 174        |
| 12.5 AUDIO .....                | 175        |
| 12.6 EMAC.....                  | 176        |
| 12.7 MCLK.....                  | 177        |

|                                  |            |
|----------------------------------|------------|
| <b>13 SDK OS Memory 统计 .....</b> | <b>183</b> |
| 13.1 概要说明 .....                  | 183        |
| 13.1.1 需求背景 .....                | 183        |
| 13.1.2 支持的功能 .....               | 183        |
| 13.1.3 不支持的功能 .....              | 184        |
| 13.1.4 统计策略 .....                | 184        |
| 13.1.5 PROC 文件节点的说明 .....        | 185        |
| 13.2 使用说明 .....                  | 185        |
| 13.2.1 内存次数统计使用 .....            | 185        |
| 13.2.2 统计每个模块申请的内存大小和次数 .....    | 186        |
| 13.2.3 模块的内核内存详细信息查看 .....       | 187        |
| 13.2.4 设置模块内核分配的次数显示限制 .....     | 188        |
| 13.3 注意事项 .....                  | 188        |
| 13.3.1 单模块模式和 ALL 模式切换 .....     | 188        |
| 13.3.2 Policy 设置时机 .....         | 188        |
| 13.3.3 Policy 选择 .....           | 189        |
| <b>14 Recovery 升级使用说明 .....</b>  | <b>190</b> |
| 14.1 Recovery 编译 .....           | 190        |
| 14.2 Recovery 脚本说明 .....         | 191        |
| 14.3 Recovery 升级方法说明 .....       | 191        |
| <b>15 OTA 升级使用说明 .....</b>       | <b>193</b> |
| 15.1 OTA 编译 .....                | 193        |
| 15.1.1 依赖文件: .....               | 193        |
| 15.1.2 打包 img: .....             | 194        |
| 15.1.3 ota 升级: .....             | 194        |

|                              |            |
|------------------------------|------------|
| <b>16 BOOTROM 相关介绍 .....</b> | <b>196</b> |
| 16.1 chip mode 介绍.....       | 196        |
| 16.2 BOOTROM 涉及的字符含义介绍.....  | 196        |

AEXRA CONFIDENTIAL FOR SIPEED

## 权利声明

爱芯元智半导体股份有限公司或其许可人保留一切权利。

非经权利人书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

## 注意

您购买的产品、服务或特性等应受商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非商业合同另有约定，本公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

# 前言

本文档旨在指导您安装 AX620E 的 SDK 以及编译工具链。

## 适用产品

AX620E 系列产品（AX630C、AX620Q）

## 适读人群

- 软件开发工程师
- 技术支持工程师

## 符号与格式定义

| 符号/格式                                                                                      | 说明                                                                                 |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>xxx</code>                                                                           | 表示您可以执行的命令行。                                                                       |
| 斜体                                                                                         | 表示变量。如，“ <i>安装目录</i> /AX620E_SDK_Vx.x.x/build 目录”中的“ <i>安装目录</i> ”是一个变量，由您的实际环境决定。 |
|  说明/备注： | 表示您在使用产品的过程中，我们向您说明的事项。                                                            |
| <b>注意：</b>                                                                                 | 表示您在使用产品的过程中，我们需要您注意的事项。                                                           |

## 修订历史

| 文档版本 | 发布时间       | 修订说明                                    |
|------|------------|-----------------------------------------|
| V1.0 | 2023/08/30 | 文档初版                                    |
| V1.1 | 2023/09/13 | 修改 xml, imgsign 路径                      |
| V1.2 | 2023/09/20 | 增加章节 7.1.4                              |
| V1.3 | 2023/11/3  | 增加章节 7.2                                |
| V1.4 | 2023/11/15 | 增加章节 14 章 feature 配置说明                  |
| V1.5 | 2024/01/23 | 修改 sdk、board 相关，删除章节 14 章 feature 配置说明  |
| V1.6 | 2024/01/27 | 添加 MCLK 寄存器配置描述                         |
| V1.7 | 2024/02/23 | 优化分区增删，同步调整 recovery 章节分区相关内容，增加 ota 章节 |
| V1.8 | 2024/03/06 | 增加章节 7.1.7 和 7.2.5                      |
| V1.9 | 2024/03/06 | 修改章节 10.2.2 中公钥生成命令                     |
| V2.0 | 2024/04/07 | 增加 16 章节，BOOTROM 相关介绍                   |
| V2.1 | 2024/07/09 | 修改 7.1.1 章节关于 NorFlash 分区调整说明           |

# 1 准备工作

## 1.1 编译工具链

由于本 SDK 同时支持 arm 和 arm64 两种架构，所以我们使用的工具链版本存在两个，一个是支持 arm 架构的 gcc version 7.5.0（Linaro GCC 7.5-2019.12）；另一个是支持 arm64 架构的 gcc version 9.2.1 20191025 (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10))，请您自行至 ARM 官方网站下载并安装该版本工具链。

官方网站的地址：

<https://developer.arm.com/downloads/-/gnu-a>

[https://armbian.astra.in.ua/\\_toolchain/](https://armbian.astra.in.ua/_toolchain/)

### 1.1.1 配置安装环境（可选）

对于一台新的 Linux 服务器或笔记本电脑，在安装工具链之前，需要您配置工具链的安装环境。

#### 操作步骤

现以一台新 ubuntu 笔记本电脑为例说明环境配置过程：

测试笔记本电脑 Ubuntu 的版本号是：ubuntu-22.04.1

**步骤1** 请您以 root 用户登录需要配置安装环境的主机。

**步骤2** 在任意目录下依次进行如下操作安装依赖包：

**步骤3** 执行 `sudo apt-get install make` 命令；

**步骤4** 执行 `sudo apt-get install libc6:i386` 命令后输入 y；

**步骤5** 执行 `sudo apt-get install lib32stdc++6` 命令后输入 y；

**步骤6** 执行 `sudo apt-get install zlib1g-dev` 命令后输入 y；

**步骤7** 执行 `sudo apt-get install libncurses5-dev` 命令后输入 `y`;

**步骤8** 执行 `sudo apt-get install ncurses-term` 命令;

**步骤9** 执行 `sudo apt-get install libncursesw5-dev` 命令;

**步骤10** 执行 `sudo apt-get install g++` 命令后输入 `y`;

**步骤11** 执行 `sudo apt-get install u-boot-tools` 命令后输入 `y`;

**步骤12** 执行 `sudo apt-get install texinfo` 命令后输入 `y`;

**步骤13** 执行 `sudo apt-get install texlive` 命令后输入 `y`;

**步骤14** 执行 `sudo apt-get install gawk` 命令后输入 `y`;

**步骤15** 执行 `sudo apt-get install libssl-dev` 命令后输入 `y`;

**步骤16** 执行 `sudo apt-get install openssl` 命令后输入 `y`;

**步骤17** 执行 `sudo apt-get install bc` 命令后输入 `y`;

**步骤18** 执行 `sudo apt-get install bison` 命令后输入 `y`;

**步骤19** 执行 `sudo apt-get install flex` 命令后输入 `y`;

**步骤20** 执行 `sudo dpkg-reconfigure dash` 命令后选择 “No” ;

**步骤21** 执行 `sudo apt-get install gcc libgcc1 gdb` 命令;

**步骤22** 执行 `sudo apt-get install build-essential` 命令;

**步骤23** 执行 `sudo apt-get install lib32z1` 命令;

**步骤24** 执行 `sudo apt-get install u-boot-tools` 命令;

**注意:**

为了确保低版本的 Ubuntu 系统可以正常安装所有依赖包，您需要执行上述最后一条命令，否则编译时会遇到找不到共享库 `libz.so.1` 的错误。为确保 uBoot 编译能够通过，请您务必执行上述最后一条命令。

**步骤25** 执行 `sudo apt install device-tree-compiler` 命令安装 `dtc`;

**步骤26** 执行 `sudo python3 -m pip install --upgrade pip` 命令；

**步骤27** 执行 `sudo pip3 install lxml` 命令。

## 1.1.2 安装工具链

### 操作步骤

**步骤1** 请您以 root 用户登录需要安装工具链的环境。

**步骤2** 在任意目录下依次执行如下命令安装工具链：

**步骤3** arm 架构工具链

#### a. glibc 版本工具链

```
sudo mkdir /usr/local/ARM-toolchain
cd /usr/local/ARM-toolchain
sudo wget http://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
tar -vxf gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
rm gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
vi /etc/profile
export PATH="/usr/local/ARM-toolchain/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin/:$PATH"
source /etc/profile
```

#### b. uclibc 版本工具链

uclibc 版本的 arm 架构工具链为 arm-AX620E-linux-uclibcgnueabihf，请通过项目途经获取该工具链。

**步骤4** arm64 架构工具链

```
sudo mkdir /usr/local/ARM-toolchain
```

```
cd /usr/local/ARM-toolchain
tar -vxf gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz
rm gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz
vi /etc/profile
export PATH="/usr/local/ARM-toolchain/gcc-arm-9.2-2019.12-x86_64-aarch64-
none-linux-gnu/bin:$PATH"
source /etc/profile
```

**步骤5** 工具链安装完成后，执行 `arm-linux-gnueabi-hf-gcc -v` 命令，查看 arm 架构工具链是否按照成功，如果安装成功，系统将显示如下工具链版本号：

```
~$ arm-linux-gnueabi-hf-gcc -v
Using built-in specs.
COLLECT_GCC=arm-linux-gnueabi-hf-gcc
COLLECT_LTO_WRAPPER=/usr/local/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabi-hf/bin/..../libexec/gcc/arm-linux-gnueabi-hf/7.5.0/lto-
wrapper
Target: arm-linux-gnueabi-hf
Configured with: '/home/tcwg-buildslave/workspace/tcwg-make-release_1/snapshots/gcc.git~linaro-7.5-2019.12/configure' SHELL=/bin/ba
sh --with-mpc=/home/tcwg-buildslave/workspace/tcwg-make-release_1/build/builds/destdir/x86_64-unknown-linux-gnu --with-mpfr=/home/
tcwg-buildslave/workspace/tcwg-make-release_1/build/builds/destdir/x86_64-unknown-linux-gnu --with-gmp=/home/tcwg-buildslave/works
pace/tcwg-make-release_1/build/builds/destdir/x86_64-unknown-linux-gnu --with-gnu-as --with-gnu-ld --disable-libmudflap --enable-l
to --enable-shared --without-included-gettext --enable-nls --with-system-zlib --disable-sjlj-exceptions --enable-gnu-unique-object
--enable-linker-build-id --disable-libstdc++-pch --enable-c99 --enable-clocale=gnu --enable-libstdc++-debug --enable-long-long --wi
th-cloog=no --with-ppl=no --with-isl=no --disable-multilib --with-float=hard --with-fpu=vfpv3-d16 --with-mode=thumb --with-tune=cor
tex-a9 --with-arch=armv7-a --enable-threads=posix --enable-multiarch --enable-libstdc++-time=yes --enable-gnu-indirect-function --w
ith-build-sysroot=/home/tcwg-buildslave/workspace/tcwg-make-release_1/build/sysroots/arm-linux-gnueabi-hf --with-sysroot=/home/tcwg
-buildslave/workspace/tcwg-make-release_1/build/builds/destdir/x86_64-unknown-linux-gnu/arm-linux-gnueabi-hf/libc --enable-checking
=release --disable-bootstrap --enable-languages=c,c++,fortran,lto --build=x86_64-unknown-linux-gnu --host=x86_64-unknown-linux-gnu
--target=arm-linux-gnueabi-hf --prefix=/home/tcwg-buildslave/workspace/tcwg-make-release_1/build/builds/destdir/x86_64-unknown-linu
x-gnu
Thread model: posix
gcc version 7.5.0 (Linaro GCC 7.5-2019.12)
```

图 1-1 ARM32 工具链版本号

**步骤6** 在任意目录下执行 `aarch64-none-linux-gnu-gcc -v` 命令，查看 arm64 架构工具链是否按照成功，如果工具链安装成功，系统将显示如下工具链的版本号：

```
$ aarch64-none-linux-gnu-gcc -v
Using built-in specs.
COLLECT_GCC=aarch64-none-linux-gnu-gcc
COLLECT_LTO_WRAPPER=/usr/local/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu/bin/..../libexec/gcc/aarch64-none-linux-
gnu/9.2.1/lto-wrapper
Target: aarch64-none-linux-gnu
Configured with: /tmp/dgbotter/bbs/rhev-vm2--rhe6x86_64/buildbot/rhe6x86_64--aarch64-none-linux-gnu/build/src/gcc/confi
gure --target=aarch64-none-linux-gnu --prefix= --with-sysroot=/aarch64-none-linux-gnu/libc --with-build-sysroot=/tmp/d
gbotter/bbs/rhev-vm2--rhe6x86_64/buildbot/rhe6x86_64--aarch64-none-linux-gnu/build/build-aarch64-none-linux-gnu/install
//aarch64-none-linux-gnu/libc --with-bugurl=https://bugs.linaro.org/ --enable-gnu-indirect-function --enable-shared --
disable-libssp --disable-libmudflap --enable-checking=release --enable-languages=c,c++,fortran --with-gmp=/tmp/dgbotter
/bbs/rhev-vm2--rhe6x86_64/buildbot/rhe6x86_64--aarch64-none-linux-gnu/build/build-aarch64-none-linux-gnu/host-tools --
with-mpfr=/tmp/dgbotter/bbs/rhev-vm2--rhe6x86_64/buildbot/rhe6x86_64--aarch64-none-linux-gnu/build/build-aarch64-none-l
inux-gnu/host-tools --with-mpc=/tmp/dgbotter/bbs/rhev-vm2--rhe6x86_64/buildbot/rhe6x86_64--aarch64-none-linux-gnu/build
/build-aarch64-none-linux-gnu/host-tools --with-isl=/tmp/dgbotter/bbs/rhev-vm2--rhe6x86_64/buildbot/rhe6x86_64--aarch64
-none-linux-gnu/build/build-aarch64-none-linux-gnu/host-tools --enable-fix-cortex-a53-843419 --with-pkgversion='GNU To
olchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)'
Thread model: posix
gcc version 9.2.1 20191025 (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10))
```

图 1-2 ARM64 工具链版本号

## 1.2 Linux 内核代码

我们将为您提供以下两种方法用于准备 Linux 内核代码。

### 1.2.1 方法一

在 AX620E\_SDK\_Vx.x.x 目录下（AX620E\_SDK\_Vx.x.x：先参考 2.1 解压缩 SDK），执行 `./sdk_unpack.sh` 命令。此时，程序将自动从源拉取 linux 内核源码并自动打 AX 的 kernel patch。拉取的源将在 `sdk_unpack.sh` 脚本中进行配置。

#### 举例说明

linux-4.19.125 内核，可通过指令：

```
wget -t 5 https://mirror.tuna.tsinghua.edu.cn/kernel/v4.x/linux-4.19.125.tar.gz
```

来获取，而 linux-5.15.73 可通过执行：

```
wget -t 5 https://mirror.tuna.tsinghua.edu.cn/kernel/v5.x/linux-5.15.73.tar.gz
```

命令来获取。如果以上源已过期，需要您自行寻找并下载合适的源。

您也可以通过 1.2.2 方法二来获取 Linux 代码。

### 1.2.2 方法二

通过 `sdk_unpack.sh` 脚本自动下载，在解压 SDK 包后，执行 `./sdk_unpack.sh`，该脚本会自动下载 linux4.19.125 和 linux-5.15.73 两个内核，前提是该 PC 能够正常访问互联网。

## 1.3 U-boot 源码

### 1.3.1 方法一

在 AX620E\_SDK\_Vx.x.x 目录下（AX620E\_SDK\_Vx.x.x：先参考 [2.1 解压缩 SDK](#)），执行 `./sdk_unpack.sh` 命令。此时，程序将自动从源拉取 `u-boot-2020.04.tar.bz2` 的源码并自动打 AX 的 `u-boot-2020.04.patch`。拉取的源将在 `sdk_unpack.sh` 脚本中进行配置。

#### 举例说明

如执行 `wget -t 5 https://ftp.denx.de/pub/u-boot/u-boot-2020.04.tar.bz2` 命令。如果该源已过期，需要您自行寻找并下载合适的源。

您也可以通过 1.3.2 方法二方法来获取 u-boot 代码。

### 1.3.2 方法二

需要您自行下载 `u-boot-2020.04` 代码包，并将该代码包的地址路径作为 `sdk_unpack.sh` 脚本的输入参数。

如执行 `./sdk_unpack.sh -u ~/boot/uboot/u-boot-2020.04.tar.bz2` 命令。

## 1.4 Atf 源码

### 1.4.1 方法一

在 AX620E\_SDK\_Vx.x.x 目录下（AX620E\_SDK\_Vx.x.x：先参考 [2.1 解压缩 SDK](#)），执行 `./sdk_unpack.sh` 命令。此时，程序将自动从源拉取 `v2.7.0.zip` 的源码并自动打 AX 的 `arm-trusted-firmware.patch`。拉取的源将在 `sdk_unpack.sh` 脚本中进行配置。

#### 举例说明

如，执行 `wget -t 5 https://github.com/ARM-software/arm-trusted-firmware/archive/refs/tags/v2.7.0.zip` 命令。如果该源已过期，需要您自行寻找并下载合适的源。

您也可以通过 1.4.2 方法二来获取 Atf 代码。

## 1.4.2 方法二

需要您自行下载 `v2.7.0.zip` 代码包，并将该代码包的地址路径作为 `sdk_unpack.sh` 脚本的输入参数。

如，执行 `./sdk_unpack.sh -a ~/boot/atf/v2.7.0.zip` 命令。

## 1.5 Optee 源码

### 1.5.1 方法一

在 `AX620E_SDK_Vx.x.x` 目录下（`AX620E_SDK_Vx.x.x`：先参考 2.1 解压缩 SDK），执行 `./sdk_unpack.sh` 命令。此时，程序将自动从源拉取 `3.21.0.zip` 的源码并自动打 AX 的 `optee_os-3.21.0.patch`。拉取的源将在 `sdk_unpack.sh` 脚本中进行配置。

### 举例说明

如，执行 `wget -t 5 https://github.com/OP-TEE/optee_os/archive/refs/tags/3.21.0.zip` 命令。如果该源已过期，需要您自行寻找并下载合适的源。

您也可以通过 1.5.2 方法二来获取 Optee 代码。

### 1.5.2 方法二

需要您自行下载 `3.21.0.zip` 代码包，可到此路径下载：[https://github.com/OP-TEE/optee\\_os/releases/tag/3.21.0](https://github.com/OP-TEE/optee_os/releases/tag/3.21.0)，并将该代码包的地址路径作为 `sdk_unpack.sh` 脚本的输入参数。

如，执行 `./sdk_unpack.sh -o ~/boot/optee/3.21.0.zip` 命令。

AEXRA CONFIDENTIAL FOR SIPEED

## 2 安装 SDK

### 2.1 解压缩 SDK 包

#### 操作步骤

- 步骤1** 请您以 root 用户登录需要安装 SDK 的环境。
- 步骤2** 进入 SDK 包所在目录，执行 `tar -xvf AX620E_SDK_Vx.x.x.tgz` 命令解压该压缩包。
- 步骤3** 解压后，您将得到 `AX620E_SDK_Vx.x.x` 目录。

### 2.2 展开 SDK 包内容

#### 操作步骤

- 步骤1** 请您进入解压后的 `AX620E_SDK_Vx.x.x` 目录。
- 步骤2** 执行 `./sdk_unpack.sh` 命令展开 SDK 包中的内容。

#### 说明：

- 执行 `./sdk_unpack.sh` 命令时，您可以带一个路径参数，用于指定 linux 内核源码、uboot、atf 和 optee 等 tar 包的位置。您也可以不指定该路径，解压脚本后将自动拉取 Kernel 代码。
- `sdk_clean.sh` 脚本用于清理展开后的 SDK 包中的内容。`sdk_clean.sh` 脚本是 `sdk_unpack.sh` 脚本的逆向操作，执行该脚本后所有通过 `sdk_unpack.sh` 脚本生成的文件及目录均会被清除，如果有私有数据请提前备份，谨慎使用！

### 2.3 介绍 SDK 包内容

展开 SDK 包后的目录如下图所示：

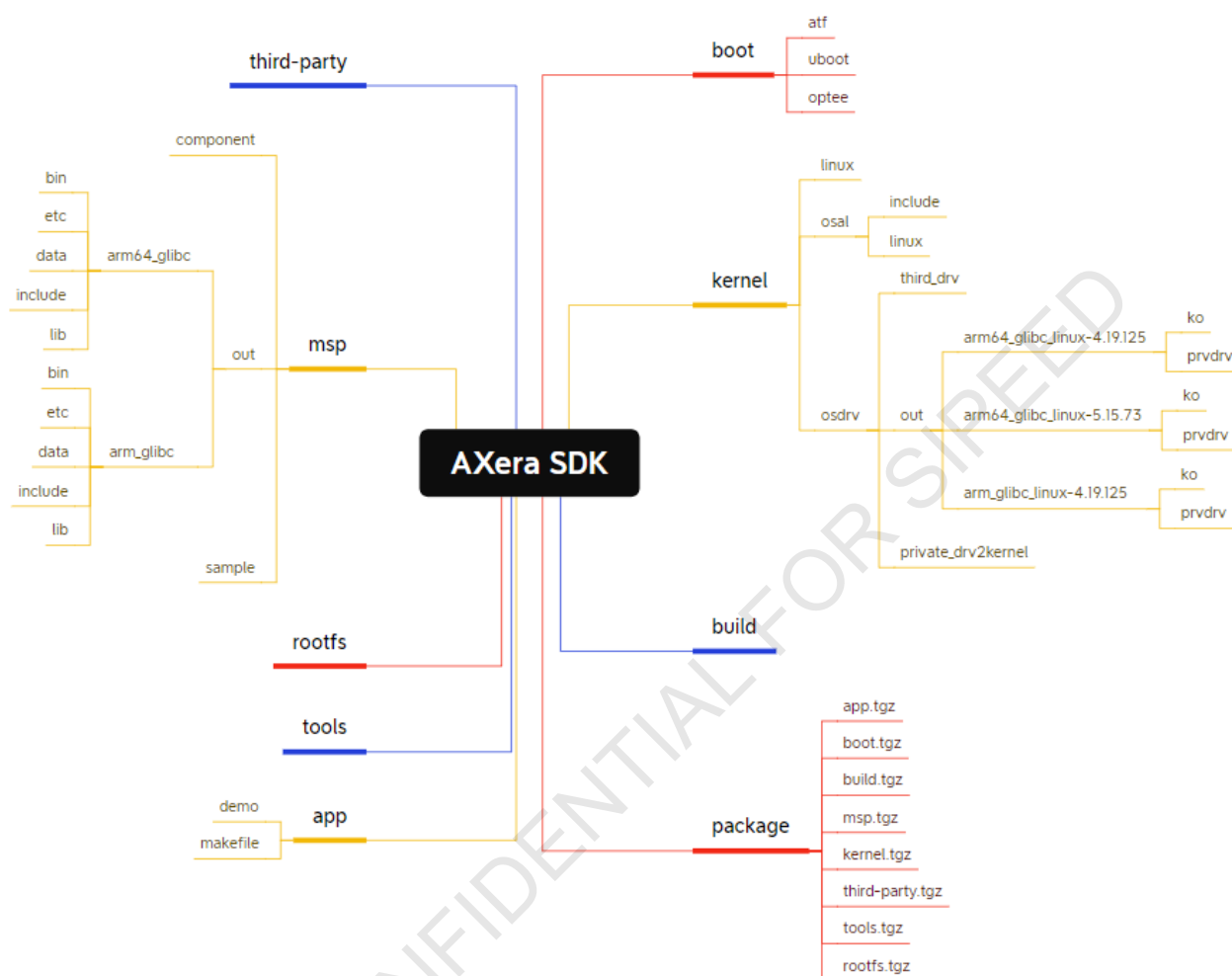


图 2-1 AX620E\_SDK\_Vx.x.x 目录结构

## 【说明】

- msp/out 目录下包含了 arm64\_glibc 和 arm\_glibc 两个目录，它们分别对应 arm 64 位系统基于 glibc 库和 arm 32 位系统架构基于 glibc 库编译生成的 SDK 资源文件；
- kernel/osdrv/out 目录下包含了 arm64\_glibc\_linux-4.19.125、arm64\_glibc\_linux-5.15.73 和 arm\_glibc\_linux-4.19.125 这 3 个目录，arm64\_glibc\_linux-4.19.125 表示该文件夹下的二进制文件是基于 arm 64 位系统+glibc+linux-4.19.125 编译生成的，另外两个目录的命名规则与此一致。

## 3 编译版本

### 3.1 编译步骤

- 请确保您已经成功编译工具链。更多内容，请参考[编译工具链](#)。
- 编译前，请确认您需要使用的工程。

#### 操作步骤

**步骤1** 请您以 root 用户登录需要编译版本的环境。

**步骤2** 进入 `安装目录/AX620E_SDK_Vx.x.x/build` 目录。

**步骤3** 执行 `make plist`，查看当前支持的所有工程列表，例如：

```
-----project list-----  
  
p=AX620Q_fastnor_arm32_k419  
  
p=AX630C_nand_arm64_k419  
  
p=AX630C_emmc_arm32_k419  
  
p=AX630C_fastemmc_arm32_k419  
  
p=AX630C_emmc_arm64_k419  
  
p=AX620Q_nand_arm32_k419  
  
p=AX630C_emmc_arm64_k515  
  
p=AX630C_fastemmc_arm64_k419  
  
p=AX620Q_nor_arm64_k515  
  
p=AX620Q_nor_arm32_k419
```

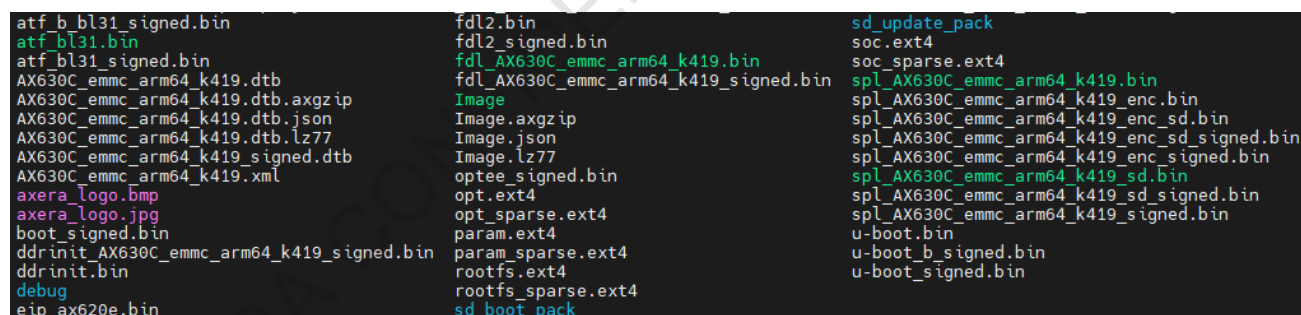
工程的命名方式为：CHIPNAME\_FLASHTYPE\_ARCH\_KERNEL。

其中 CHIPNAME 包含：AX620Q 和 AX630C；FLASHTYPE 包含 emmc、nand、nor、fastnor 以及 fastemmc，其中 emmc、nand 和 nor 表示 flash 类型，而包含 fast 的表示该工程为快速启动工程；而 ARCH 包含 arm 和 arm64，分别代表 arm 32 位系统和 arm 64 位系统；KERNEL 可能的值为 k419 和 k515，它们分别对应 linux-4.19.125 和 linux-5.15.73。

**步骤4** 执行 `make p=AX630C_emmc_arm64_k419 clean all install axp` 命令。该命令用于实现全版本的编译并实现 axp 打包。另外，SDK 支持并行编译，可通过 -j 参数执行并行编译，如：`make p=AX630C_emmc_arm64_k419 clean all install axp -j8`。

**步骤5** 编译成功后，将在 build 目录下生成 out 目录。

**步骤6** 进入 out 目录后，您将看到 AX630C\_emmc\_arm64\_k419 目录和 AX630C\_emmc\_arm64\_k419\_VX.X.X\_XXXXXXXXXX.axp 包。在 build/out/AX630C\_emmc\_arm64\_k419/images 目录中，您将看到如下图所示的编译生成的各个文件：



```

atf_b_bl31_signed.bin      fdl2.bin                  sd_update_pack
atf_bl31.bin              fdl2_signed.bin          soc.ext4
atf_bl31_signed.bin       fdl_AX630C_emmc_arm64_k419.bin  soc_sparse.ext4
AX630C_emmc_arm64_k419.dtb  fdl_AX630C_emmc_arm64_k419_signed.bin  spl_AX630C_emmc_arm64_k419.bin
AX630C_emmc_arm64_k419.dtb.axgzip  Image                     spl_AX630C_emmc_arm64_k419_enc.bin
AX630C_emmc_arm64_k419.dtb.json     Image.axgzip              spl_AX630C_emmc_arm64_k419_enc_sd.bin
AX630C_emmc_arm64_k419.dtb.lz77      Image.json                 spl_AX630C_emmc_arm64_k419_enc_signed.bin
AX630C_emmc_arm64_k419_signed.dtb     Image.lz77                 spl_AX630C_emmc_arm64_k419_sd_signed.bin
AX630C_emmc_arm64_k419.xml           optee_signed.bin          spl_AX630C_emmc_arm64_k419_sd.bin
axera_logo.bmp               opt.ext4                   spl_AX630C_emmc_arm64_k419_sd_signed.bin
axera_logo.jpg               opt_sparse.ext4            spl_AX630C_emmc_arm64_k419_signed.bin
boot_signed.bin              param.ext4                  u-boot.bin
ddrinit_AX630C_emmc_arm64_k419_signed.bin  param_sparse.ext4         u-boot_b_signed.bin
ddrinit.bin                  rootfs.ext4                 u-boot_signed.bin
debug                        rootfs_sparse.ext4
eip_ax620e.bin               sd_boot_pack
  
```

图 3-1 编译生成的文件列表

AX630C\_emmc\_arm64\_k419\_VX.X.X\_XXXXXXXXXX.axp 包则是将上图中的 fdl2、u-boot、image 等文件统一打包，方便您使用 AX\_DL 下载工具下载。

**步骤7** 如果是 nor 工程，生成的 axp 包则会有所不同，这是因为 nor 工程受 flash 大小限制，无法将所有的 sensor 都放到 axp 包中去，只能放下一颗 sensor 的配置，故 AX620E nor 工程的 axp 包名称中是会包含支持的 sensor 名的，比如：AX620Q\_nor\_arm32\_k419\_os04a10\_VX.X.X\_XXXXXXXXXX.axp，其中

AX620Q\_nor\_arm32\_k419 为工程名，而 os04a10 为该 axp 支持的 sensor 类型名称。

#### 说明：

关于如何使用 AX\_DL 工具下载 Demo 板，请您参考《AXDL 工具使用指南》文档获取相关内容。

## 3.2 工程支持

SDK 支持的工程，可以在解压并展开 SDK 后，进入工程根目录下的 build 目录，执行 make plist 命令来查看。如：

```
-----project list-----  
p=AX620Q_fastnor_arm32_k419  
p=AX630C_nand_arm64_k419  
p=AX630C_emmc_arm32_k419  
p=AX630C_fastemmc_arm32_k419  
p=AX630C_emmc_arm64_k419  
p=AX620Q_nand_arm32_k419  
p=AX630C_emmc_arm64_k515  
p=AX630C_fastemmc_arm64_k419  
p=AX620Q_nor_arm64_k515  
p=AX620Q_nor_arm32_k419
```

图 3-2 工程列表

列出的即为该 SDK 所支持的工程，工程名命名规则为：

CHIPNAME\_[fast]FLASHTYPE\_ARCH\_KERNEL

其中 CHIPNAME 为芯片类型，包含 AX620Q 和 AX630C；FLASHTYPE 为闪存类型，包含 emmc、nand 和 nor，其前面的 fast 表示支持快速启动；ARCH 为芯片架构，包含 arm 和 arm64，分别为 32 位和 64 位系统；KERNEL 支持两个版本 k419 和 k515，它们分别代表 linux-4.19.125 和 linux-5.15.73 两个内核版本。请根据实际情况选择合适的工程进行编译，可生成该工程对应的 axp 包，如：make p=AX630C\_emmc\_arm64\_k419 all install axp，会在 build/out 目录下生成对应的 axp 包。

## 3.3 atf 编译

atf 编译后生成可用于下载的文件为 atf\_bl31\_signed.bin，可以单独编译，例如编译

AX630C\_emmc\_arm64\_k419 工程，步骤如下：

- 步骤1** 进入安装目录/AX620E\_SDK\_Vx.x.x/boot/atf 目录。
- 步骤2** 执行 `make p=AX630C_emmc_arm64_k419 clean all install` 命令。
- 步骤3** 编译成功后，将在 build 目录下生成 out/AX630C\_emmc\_arm64\_k419 目录。
- 步骤4** 进入 build/out/AX630C\_emmc\_arm64\_k419 目录后，您将看到 `atf_bl31_signed.bin`

## 3.4 Uboot 编译

uboot 编译后生成可用于下载的文件为 `u-boot_signed.bin`，`config` 文件位于 `boot/uboot/u-boot-2020.04/configs` 目录，如 `AX630C_emmc_arm64_k419_defconfig` 是 AX630C\_emmc\_arm64\_k419 工程的配置文件。

uboot 可以单独进行编译，例如编译 AX630C\_emmc\_arm64\_k419 工程，步骤如下：

- 步骤1** 进入安装目录/AX620E\_SDK\_Vx.x.x/boot/uboot 目录。
- 步骤2** 执行 `make p=AX630C_emmc_arm64_k419 clean all install` 命令。
- 步骤3** 编译成功后，将在 build 目录下生成 out/AX630C\_emmc\_arm64\_k419 目录。
- 步骤4** 进入 build/out/AX630C\_emmc\_arm64\_k419 目录后，您将看到 `u-boot_signed.bin`

## 3.5 Kernel 编译

我们的当前支持两个版本的 Linux kernel，它们分别是 `linux-5.15.73` 和 `linux-4.19.125`。对于 `linux-5.15.73`，目前只支持 arm 64 位系统，而 `linux-4.19.125` 则同时支持 arm 32 位系统和 arm 64 位系统。kernel 的编译可进入 SDK 的 `kernel/linux` 目录，执行 `make p=XXX clean all insall` 指令，编译完成后生成可用于下载的文件为 `boot_signed.bin`，该文件位于 `build/out/XXX/images` 目录，其中 XXX 为工程名。Kernel config 文件位于 `kernel/linux/$LINUX_DIR/arch/$ARCH/configs` 目录，其中如 `$LINUX_DIR` 表示 Linux 目录，它可能是 `linux-5.15.73` 和 `linux-4.19.125`，而 `$ARCH` 表示芯片架构，它可能是 `arm` 和 `arm64`。以 AX630C\_emmc\_arm64\_k419 工程为例，它对应的 `defconfig` 文件为 `axera_AX630C_emmc_arm64_k419_defconfig`，该文件位于 `kernel/linux/linux-4.19.125/arch/arm64/configs` 目录。`dtb` 文件与 `defconfig` 文件所在的位置类似。

kernel 可以单独编译，例如编译 AX630C\_emmc\_arm64\_k419 工程，步骤如下：

**步骤1** 进入安装目录/AX620E\_SDK\_Vx.x.x/kernel/linux 目录。

**步骤2** 执行 `make p=AX630C_emmc_arm64_k419 clean all install` 命令。

**步骤3** 编译成功后，将在 build 目录下生成 out/AX630C\_emmc\_arm64\_k419 目录。

**步骤4** 进入 build/out/AX630C\_emmc\_arm64\_k419 目录后，您将看到 images 和 objs

## 3.6 rootfs 编译

rootfs 文件位于 SDK 中 rootfs 目录，该目录下包含的文件如下：

arm arm64 busybox Makefile

其中 arm 文件夹下存放的是 arm 架构的 rootfs 资源文件；arm64 下存放的是 arm64 架构的 rootfs 文件。arm 和 arm64 的目录结构如下：

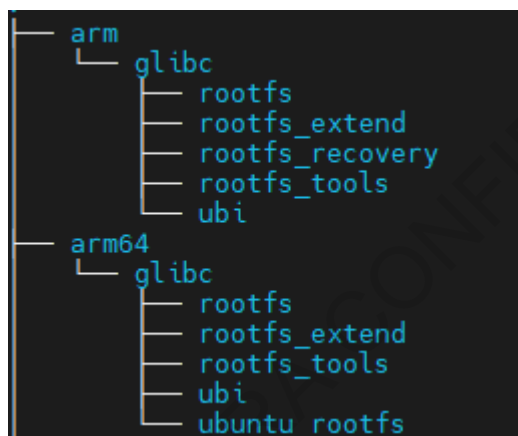


图 3-3 rootfs 目录结构

glibc 表示该 rootfs 是基于 glibc 编译生成，glibc 目录下包含：rootfs、rootfs\_extend、rootfs\_recovery、rootfs\_tools 和 ubi 等目录，其中 rootfs 目录包含了系统正常开机所需的最小 rootfs 资源，比如 AX620Q\_fastnor\_arm32\_k419 工程使用的 rootfs 资源，就只包含 rootfs 文件夹下的内容，这样既可保证系统正常工作，又使得系统体积最小，可达到加速启动的目的。而 rootfs\_extend 则是一些扩展资源，如 ssh、ftp、lrz、libcrypto 等一些扩展使用的工具、库等。rootfs\_recovery 是系统进入 recovery 模式下的文件系统，recovery 模式是一种系统升级模式。rootfs\_tools 则是存放一些调试工具如 gdb、gdbserver 和 strace 等。ubi 目录下则存放的是 nand 文件系统相关的一些

配置资源。对于 arm64，还存在一个 ubuntu\_rootfs，这里存放的是 Ubuntu 文件系统。

### 3.6.1 AX620E 文件系统编译

如需编译 AX620E 的文件系统，进入 rootfs 目录

例如生成 AX630C\_emmc\_arm64\_k419 工程的文件系统，可执行：

```
make p=AX630C_emmc_arm64_k419 all install image
```

共生成四个文件 rootfs\_sparse.ext4, soc\_sparse.ext4, opt\_sparse.ext4, param\_sparse.ext4 位于 build/out/AX630C\_emmc\_arm64\_k419/images 目录。

## 3.7 Optee 编译

Optee 是针对有 trustzone 需求使用，是可选择的，暂时只支持编译 AX630C\_emmc\_arm64\_k419 和 AX630C\_emmc\_arm64\_k515 工程，编译后生成可用于下载的文件为 optee\_signed.bin，可以单独编译，步骤如下：

**步骤1** 进入安装目录/AX620E\_SDK\_Vx.x.x/boot/optee 目录。

**步骤2** 执行 make p=AX630C\_emmc\_arm64\_k419 clean all install 命令。

**步骤3** 编译成功后，将在 build 目录下生成 out/AX630C\_emmc\_arm64\_k419 目录。

**步骤4** 进入 build/out/AX630C\_emmc\_arm64\_k419 目录后，您将看到 optee\_signed.bin

**注意：**

如果不需要 optee，可以在 build/projects/AX630C\_emmc\_arm64\_k419/project.mak 中配置 SUPPORT\_OPTEE := false，在 build 目录编译 axp 时所有 optee 相关文件都不会被编译。

## 4 分配 Memory 和存储空间

### 4.1 存储空间概况

在 SDK 中，默认 DDR 的空间范围为 2G Byte，其地址分配如下图所示：

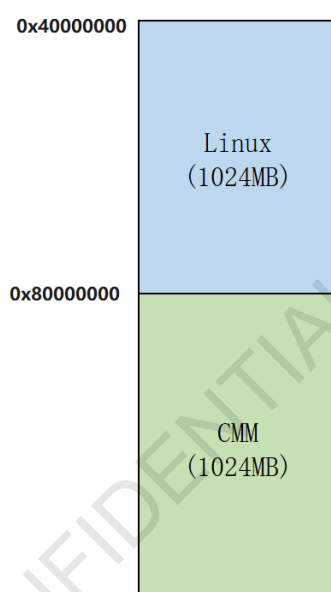


图 4-1 内存分布示意图

根据上图所示，SDK 的内存整体可分为以下两段空间：

- linux 所包含的 1024M Byte，整个 linux 将占据此段空间。
- 作为大段物理连续内存管理的 CMM（Contiguous Memory Model）的空间，共 1024M。

说明：

这里仅以 2GB DDR 空间分布供参考，实际情况可能存在差异，请以实际情况为准。

### 4.2 内存空间分配的调整

如果您想调整上述空间的大小，请参考下面的详细介绍。

## 4.2.1 系统的内存大小调整

方法一

修改 build/project/{PROJECT}/project.mk(partition.mk) 配置 (PROJECT 为工程名)，以 AX620Q\_nor\_arm32\_k419 工程为例，修改 build/project/AX620Q\_nor\_arm32\_k419/project.mk 文件的 OS\_MEM\_SIZE 配置：

```
build > projects > AX620Q_nor_arm32_k419 > M project.mk

# linux OS memory config
OS_MEM_SIZE      := 107 #MB
OS_MEM           := mem=$(strip $(OS_MEM_SIZE))M
```

图 4-2 内存调整

默认情况下，系统内存是 256M，修改时只需将 107M 修改为您期望的值。

方法二

在 uboot 启动阶段，按任意键停止启动，通过设置环境变量的方式调整系统内存，然后重启。如下所示：

```
Environment size: 496/131068 bytes
AXERA-UBOOT=>setenv bootargs mem=107M console=ttyS0,115200n8 earlyprintk=dw_uart,board_id=0,boot_reason=0x0,0x04992000 initcall_debug=0 loglevel=8 ax_boot_delay=10 vmalloc=768M root=/dev/
mmcblk0p9 rootfstype=ext4 rw rootwait blkdevparts=mmcblk0:1536K(uboot),1536K(uboot_bk),1024K(env),5120K(param),1024K(logo),1024K(dtb),65536K(kernel),1024K(atf),131072K(rootfs),1048576K(soc
),6377472K(opt)
AXERA-UBOOT=>saveenv
Saving Environment to MMC... Writing to MMC(0)... OK
```

图 4-3 设置环境变量

## 4.2.2 修改 CMM 的地址和大小

CMM 的地址和大小的配置位于 build/project/{PROJECT}/project.mk(partition.mk) 文件 (PROJECT 为工程名)，以 AX620Q\_nor\_arm32\_k419 工程为例，对应的文件为 build/project/AX620Q\_nor\_arm32\_k419/project.mk。CMM\_START\_ADDR 和 CMM\_SIZE 会自动根据 SYS\_DRAM\_SIZE(DDR 大小)和 OS\_MEM\_SIZE 自动适配，无需手动修改。

```
> build > projects > AX620Q_nor_arm32_k419 > M project.mak
# cmm memory config
CMM_START_ADDR      := $(call AddAddressMB, $(SYS_DRAM_BASE), $(OS_MEM_SIZE))
CMM_SIZE             := $(shell echo $$((($SYS_DRAM_SIZE) - $(OS_MEM_SIZE))))
CMM_POOL_PARAM       := anonymous,0,$(strip $(CMM_START_ADDR)),$(CMM_SIZE)M
```

图 4-4 CMM 配置

修改配置后，可在板端查看/soc/scripts/ auto\_load\_all\_drv.sh 文件：

```
insmod /soc/ko/ax_cmm.ko cmm pool=anonymous,0,0x46B00000,149M
```

板端查看 CMM 配置

如果需要在板端直接修改 CMM 配置，也可以通过 vi 来修改该文件来实现（只读文件系统除外）。

## 4.3 FLASH 存储空间的分配

AX620E eMMC 板的存储设备是 EMMC，以 AX630C\_emmc\_arm64\_k419 board 中非 AB 分区的 EMMC 空间为例，分配如下图所示：

|                  |
|------------------|
| SPL(768K)        |
| DDRINIT(512K)    |
| ATF(256K)        |
| ATF_BAK(256K)    |
| UBOOT(1536K)     |
| UBOOT_BAK(1536K) |
| ENV(1M)          |

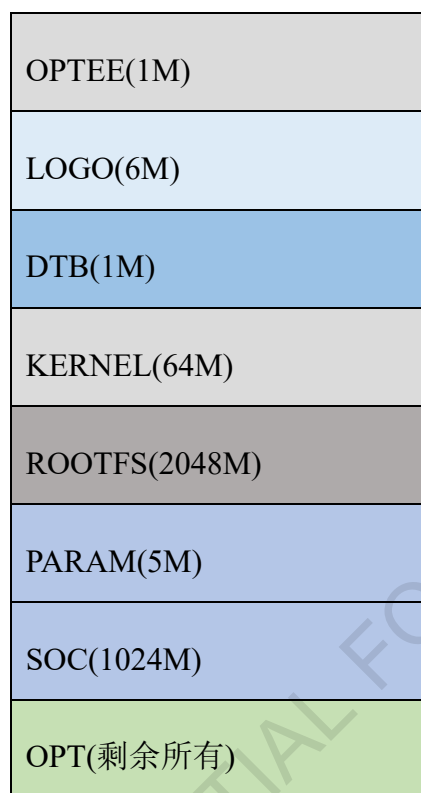


图 4-5 EMMC 空间分布示意图

EMMC 空间分配：

uboot 和备份 uboot 分配 1.5 MB；EVN 环境变量分配 1 MB；

param 和 logo 暂时未使用；

dtb 分配 1 MB 空间，不同板子使用不同的配置；

Kernel 分配 64 MB 空间；

atf: arm trust firmware bl31

optee: 当不需要 optee 时可不编译 optee 以及占用 emmc 分区空间

其次：rootfs 文件系统和 kernel 各 2048 MB，64 MB

Soc 分区用来存放 ko 和 lib 库

Opt 分区用来存放应用程序，sensor, ipcdemo，剩余 emmc 空间。

🔑 说明：

- ATF、Optee、Uboot、DTB、Kernel、Rootfs、Param、SOC、OPT 属于 User Data Area。
- ATF、Uboot、env、DTB、Kernel、Rootfs、SOC 是必须要有，其中 ATF、Uboot、env、DTB、Kernel、Rootfs 是系统启动所必须，SOC 中存放了原厂提供的驱动和共享库。

OPT 分区存储了 sample 程序等，用户可根据需要增删。

## 4.4 调整分区大小

若只需要修改分区的大小只需要修改下图 4-6 文件中，对应的 size 即可，无需额外修改文件，其余脚本会自动适配。如下：

进入 build/projects/{board\_name}/project.mak(partition.mak)配置文件，调整配置文件中的 kernel、param、rootfs 和 opt 对应的 Size 即可。注意 eMMC 工程是在 partion.mak 或者 partion\_ab.mak 中配置的，project.mak 中可以看到引用。

我们不建议您调整 opt 以前的分区（如，kernel、rootfs 和 soc 分区），您可以根据实际需要自行调整 opt 分区。

```
#partition size config
SPL_PARTITION_SIZE      := 768K
DDRINIT_PARTITION_SIZE  := 512K
ATF_PARTITION_SIZE      := 256K
ATF_B_PARTITION_SIZE    := 256K
UBOOT_PARTITION_SIZE    := 1536K
UBOOT_B_PARTITION_SIZE  := 1536K
ENV_PARTITION_SIZE      := 1M
ifeq ($(strip $(SUPPORT_OPTEE)),TRUE)
OPTEE_PARTITION_SIZE    := 1M
endif
LOGO_PARTITION_SIZE     := 6M
DTB_PARTITION_SIZE      := 1M
KERNEL_PARTITION_SIZE   := 64M
ROOTFS_PARTITION_SIZE   := 2048M
PARAM_PARTITION_SIZE    := 5M
SOC_PARTITION_SIZE      := 1024M
OPT_PARTITION_SIZE      := 4096M
AUTO_FIT_PARTITION      := OPT
```

图 4-6 build/projects/AX630C\_emmc\_arm64\_k419/partion.mak 配置文件内容

该文件中的 bootargs 配置经过脚本解析，最终被使用的位置在 uboot/u-boot-2020.04/include/configs/ax620e\_common.h 和 kernel 的 dtb 文件中。

```
#define NOR_MTDPARTS "mtdparts=spi4.0:" FLASH_PARTITIONS
#define NAND_MTDPARTS "mtdparts=spi4.0:" FLASH_PARTITIONS

/* bootargs for eMMC */
#define BOOTARGS_EMMC DEBUG_BOOT_ARGS KERNEL_BOOTARGS
```

图 4-7 uboot/u-boot-2020.04/include/configs/ax620e\_common.h

在 build 目录下执行 `make p=AX630C_emmc_arm64_k419 axp` 命令重新生成 axp 包，用 AX\_DL 工具下载，就可以调整分区大小。

**注意：**

如果在您调整 kernel 分区大小后系统出现“cannot malloc buf for image kernel”报错，您可以适当增大 `boot\uboot\uboot-2020.04\include\configs\ax620e_common.h` 配置文件中的 `#define CONFIG_SYS_MALLOC_LEN` 参数（`32 << 20`）。

## 4.5 新建或删除分区

如果需要增加或删除分区，请您参考如下操作步骤：

- 步骤1**     `rootfs/Makefile`    #创建根目录下对应的目录并制作分区镜像
- 步骤2**     `rootfs/rootfs/etc/fstab`    #配置新增分区的文件系统信息
- 步骤3**     `build/projects/partition_tbl.mak`    #新增分区时需要增加相应的分区信息，分区命名格式要相同 `xxx_PART`，`xxx_PARTITION_SIZE`，若新增分区已存在则不用重新增加
- 步骤4**     `build/projects/{board_name}/project.mak(partition.mak)`    #在 `#partition size config` 下面增加或删除相应的分区信息及修改分区信息，如图 4-8，建议放在 `OPT` 分区前面，并和 `OPT` 分区相邻，`FLASH_PARTITIONS` 变量中增加相应的分区信息，如图 4-9。

```
#partition size config
SPL_PARTITION_SIZE := 768K
DDRINIT_PARTITION_SIZE := 512K
ATF_PARTITION_SIZE := 256K
ATF_B_PARTITION_SIZE := 256K
UBOOT_PARTITION_SIZE := 1536K
UBOOT_B_PARTITION_SIZE := 1536K
ENV_PARTITION_SIZE := 1M
ifeq ($(strip $(SUPPORT_OPTEE)),TRUE)
OPTEE_PARTITION_SIZE := 1M
endif
LOGO_PARTITION_SIZE := 6M
DTB_PARTITION_SIZE := 1M
KERNEL_PARTITION_SIZE := 64M
ROOTFS_PARTITION_SIZE := 2048M
PARAM_PARTITION_SIZE := 5M
SOC_PARTITION_SIZE := 1024M
OPT_PARTITION_SIZE := 4096M
AUTO_FIT_PARTITION := OPT
```

图 4-8 project.mak(partition.mak) 配置文件内容

```
FLASH_PARTITIONS := $(SPL_PART),
ifeq ($(strip $(SUPPORT_DDRINIT_PART)),TRUE)
FLASH_PARTITIONS += $(DDRINIT_PART),
endif
FLASH_PARTITIONS += $(ATF_PART),$(ATF_B_PART),$(UBOOT_PART),$(UBOOT_B_PART),
FLASH_PARTITIONS += $(ENV_PART),$(LOGO_PART),
ifeq ($(strip $(SUPPORT_OPTEE)),TRUE)
FLASH_PARTITIONS += $(OPTEE_PART),
endif
FLASH_PARTITIONS += $(DTB_PART),$(KERNEL_PART),
FLASH_PARTITIONS += $(ROOTFS_PART),$(PARAM_PART),$(SOC_PART),$(OPT_PART)
```

图 4-9 project.mak(partition.mak) 配置文件内容

**步骤5** 新增分区，build/axp\_make.sh 中按如下步骤修改

**步骤6** 增加相应分区文件路径，如图 4-10，路径名需要和 xxx\_PART，xxx\_PARTITION\_SIZE 对应，即增加 xxx\_PATH=xxxxxxx 即可。（axp\_make.sh 文件中 get\_axp\_parm 函数会自动解析 FLASH\_PARTITIONS 中的信息，匹配路径，自动整合打包命令。）

```
DTB_A_PATH=$IMG_PATH/${PROJECT}_signed.dtb
DTB_B_PATH=$IMG_PATH/${PROJECT}_signed.dtb
EIP_PATH=$IMG_PATH/eip_ax620e.bin
FDL1_PATH=$IMG_PATH/fdl_${PROJECT}_signed.bin
FDL2_PATH=$IMG_PATH/fdl2_signed.bin
```

**步骤7** 4-10 分区文件路径

**步骤8** 删除分区，在 4-9 中删除 FLASH\_PARTITIONS 中删除对应的分区即可

**步骤9** 在 build 目录下执行 `make p=AX630C_emmc_arm64_k419 axp` 重新生成 axp 包，用 AX\_DL 工具下载，就可以实现分区的增加或删除。

**！注意：**

- bootargs 默认保存到 env 分区。
- 如果 env 分区地址变化，设备原 env 分区存储的内容无法恢复。

## 5 根文件系统

### 5.1 根文件系统简介

根文件系统（root Filesystem）是内核启动时挂载（mount）的第一个文件系统，如果系统不能从指定设备上挂载根文件系统，会导致系统启动出错。系统启动成功之后可以自动或手动挂载其他文件系统。很明显，一个系统中是可以同时存在不同的文件系统的。

根文件系统的“根”说明该文件系统是加载其他文件系统的“根”，否则其他的文件系统无法进行挂载。根文件系统被挂载在根目录“/”之下，在根目录下就存在该文件系统的各个目录和文件，比如/bin、/sbin、/mnt 等，然后将其他文件系统挂载在/mnt 目录下。

根文件系统包含系统启动时所必须的目录和关键性文件，以及挂载其他文件系统所必须的文件。如下图所示：

```
4096 Nov  4 11:52 .
4096 Nov 10 15:42 ..
4096 Nov  4 11:52 bin
4096 Nov  4 11:52 dev
4096 Nov 10 15:42 etc
4096 Nov  4 11:52 lib
   3 Nov  4 11:52 lib64 -> lib
  11 Nov  4 11:52 linuxrc -> bin/busybox
4096 Nov  4 11:52 media
4096 Nov  4 11:52 mnt
4096 Nov  4 11:52 opt
4096 Nov  4 11:52 param
4096 Nov  4 11:52 proc
4096 Nov  4 11:52 root
4096 Nov  4 11:52 run
4096 Nov 10 15:42 sbin
4096 Nov  4 11:52 soc
4096 Nov  4 11:52 sys
4096 Nov  4 11:52 tmp
4096 Nov  4 11:52 usr
4096 Nov  4 11:52 var
```

图 5-1 根文件系统顶层目录结构图

因此，内核必须要和根文件系统相互配合才能工作，只有内核本身是无法工作的。

## 5.2 利用 BusyBox 制作根文件系统

BusyBox 是一个遵循 GPL 协议、以自由软件形式发行的应用程序。BusyBox 在单一的可执行文件中提供了精简的 Unix 工具集，可运行于多款 POSIX 环境的操作系统，如 Linux、FreeBSD 等。由于 BusyBox 可执行文件的文件比较小，使得它非常适合使用于嵌入式系统。

BusyBox 和 uboot/kernel 的操作类似，都需要先配置、编译和安装，操作成功之后才能开始制作根文件系统。

### 5.2.1 获取 BusyBox 源码

获取 BusyBox 的源码的方式又两种：通过官方网站获取，或者直接利用 AX SDK 中发布的 Busybox。

#### 通过官方网站获取 BusyBox 源码

您可以登录 BusyBox 官网（<https://www.busybox.net/>）获取 BusyBox 的源代码。源代码如，BusyBox-1.35.0.tar.gz2。

#### 通过 SDK 获取 BusyBox 源码

在 AX 发布的 SDK 包中，已经包含了一份 BusyBox 源码。该源码的基于官方 busybox-1.35.0 版本的基础上，增加了 AX 自定义内容，关于自定义的内容说明，请参考 5.3 使用 SDK 中的 Busybox 中的说明。

### 5.2.2 配置 BusyBox

#### 准备工作

- 请您确保已经成功获取 BusyBox 源码。更多内容，请参考 5.2.1 获取 BusyBox 源码。

## 操作步骤

**步骤1** 请您以 root 用户登录需要配置 BusyBox 的环境。

**步骤2** 执行 vim Makefile 命令配置交叉编译环境，参照如下内容修改 Makefile 配置文件：

```
CROSS_COMPILE ?= aarch64-none-linux-gnu-  
ARCH ?= arm64
```

**步骤3** 执行 make menuconfig 命令配置 menuconfig 并选择静态编译：

```
BusyBox Settings --->  
[*] Build BusyBox as a static binary (no shared libs)
```

### 5.2.3 编译和安装 BusyBox

#### 准备工作

➤ 请您确保已经成功配置 BusyBox。更多内容，请参考 5.2.2 配置 BusyBox。

#### 操作步骤

**步骤1** 请您以 root 用户登录需要编译和安装 BusyBox 的环境。

**步骤2** 依次执行如下命令编译和安装 BusyBox：

```
make  
make install
```

**步骤3** 编译和安装成功后，系统将在 BusyBox 目录下的 \_install 目录生成以下目录和文件：

```
drwxrwxr-x 5 xxx xxx 4096 12 月 30 19:58 ./  
drwxr-xr-x 37 xxx xxx 4096 12 月 30 19:58 ../  
drwxrwxr-x 2 xxx xxx 4096 12 月 30 19:58 bin/
```

```
lrwxrwxrwx 1 xxx xxx 11 12月 30 19:58 linuxrc -> bin/BusyBox*  
drwxrwxr-x 2 xxx xxx 4096 12月 30 19:58 sbin/  
drwxrwxr-x 4 xxx xxx 4096 12月 30 19:58 usr/
```

🔑 说明：

上述目录和文件后方的 xxx 和 xxx 分别表示用户和组。

## 5.2.4 制作根文件系统

### 准备工作

➤ 请您确保已经成功编译和安装 BusyBox。更多内容，请参考 5.2.3 编译和安装 BusyBox。

### 操作步骤

**步骤1** 请您以 root 用户登录需要制作根文件系统的环境。

**步骤2** 依次执行如下命令生成基本的目录：

```
mkdir rootfs  
cd rootfs  
cp -r BusyBox-1.35.0/_install/*.  
mkdir etc dev lib tmp var mnt home proc
```

**步骤3** 配置 etc、lib 和 dev 目录。

1. etc 目录可参考系统/etc 目录下的文件。其中，最主要的文件包括 inittab、fstab、init.d/rcS 等文件。您可以从 BusyBox 的 examples 目录下拷贝这些文件至当前目录，并根据您的实际需要自行修改。
2. dev 目录下的设备文件，可以从系统中拷贝或执行 mknod 命令生成需要的设备文件。
3. lib 目录是存放应用程序所需要的库文件。请您根据应用程序需要拷贝相应的库文件至当前目录。

上述步骤完成后，一个完整的根目录系统就生成了。

### 5.3 使用 SDK 中的 BusyBox

使用 SDK 中的 BusyBox，可免去制作根文件系统中的繁杂步骤，直接通过 Makefile 可直接生成根文件系统。

#### 5.3.1 BusyBox 路径

SDK 中的 BusyBox 位于 SDK 包的 packages 下的 rootfs.tgz 中：

|                                                                                     |                    |                 |             |            |
|-------------------------------------------------------------------------------------|--------------------|-----------------|-------------|------------|
|    | app.tar.gz         | 2023/8/30 22:02 | WinRAR 压缩文件 | 6,760 KB   |
|    | boot.tar.gz        | 2023/8/30 22:02 | WinRAR 压缩文件 | 1,371 KB   |
|    | build.tar.gz       | 2023/8/30 22:02 | WinRAR 压缩文件 | 24 KB      |
|   | kernel.tar.gz      | 2023/8/30 22:02 | WinRAR 压缩文件 | 3,036 KB   |
|  | msp.tar.gz         | 2023/8/30 22:03 | WinRAR 压缩文件 | 105,534 KB |
|  | rootfs.tar.gz      | 2023/8/30 22:02 | WinRAR 压缩文件 | 15,911 KB  |
|  | third-party.tar.gz | 2023/8/30 22:02 | WinRAR 压缩文件 | 14,950 KB  |
|  | tools.tar.gz       | 2023/8/30 22:02 | WinRAR 压缩文件 | 228,355 KB |

图 5-2 Busybox 位于 rootfs.tar.gz 中

通过 sdk\_unpack.sh 脚本解压 SDK 后，BusyBox 的路径位于 rootfs 目录：

```
~/projects/SDK/AX620E_SDK_V1.7.0_P1_20240125215321_N011/rootfs$ ll
total 24
drwxrwxr-x  5          4096 Jan 25 22:31 ./
drwxrwxr-x 12          4096 Jan 26 17:55 ../
drwxrwxr-x  3          4096 Jan 25 22:31 arm/
drwxrwxr-x  3          4096 Jan 25 22:31 arm64/
drwxrwxr-x  3          4096 Jan 25 22:31 busybox/
-rw-rw-r--  1          1016 Jan 25 22:31 Makefile
```

图 5-3 解压后 busybox 所在路径

#### 5.3.2 BusyBox 的编译

默认情况下 BusysBox 是不参与编译的，如果您需编译 BusyBox，有两种方式可以实现。

方式一：进入到 `rootfs/busybox` 目录下，执行 `make p=XXX busybox`。这种方式的缺点是每次需要单独执行，如果您希望每次编译整个工程时自动编译 `BusyBox`，建议您使用方式二。

方式二：在工程 `build/projects` 目录下对应每个 `project` 的 `.mak` 文件，如 `AX630C_emmc_arm64_k419` 工程对应的 `.mak` 文件为 `build/projects/AX630C_emmc_arm64_k419/project.mak`。修改该文件的配置项，将：

```
BUILD_BUSYBOX := FALSE
```

修改为：

```
BUILD_BUSYBOX := TRUE
```

这样在编译整个工程时，每次就会自动编译 `Busybox` 了。如果 `.mak` 文件中不包含 `BUILD_BUSYBOX` 配置，直接添加上即可。

`Busybox` 在编译时，会使用 `busybox-1.35.0/configs/AX620E_defconfig` 作为默认的 `config` 文件，如果您需要修改 `config` 配置，可在 `rootfs` 目录下执行 `make p=xxx bbxconfig`，执行完成后会在 `busybox-1.35.0` 目录下生成 `.config` 文件，请在完成 `config` 配置后将 `busybox-1.35.0/.config` 拷贝到 `busybox-1.35.0/configs/AX620E_defconfig` 文件，否则配置信息将丢失。

## 5.4 文件系统简介

### 5.4.1 JFFS2

JFFS2 全称是 `Journalling Flash File System Version 2`，是 `Red Hat` 公司开发的闪存文件系统，其前身是 `JFFS`，最早只支持 `NOR Flash`。自 2.6 版以后开始支持 `NAND Flash`，适合嵌入式系统使用。

生成 JFFS2 文件系统镜像的命令是：

```
mkfs.jffs2 -r rootfs -o rootfs.jffs2 --pad=0xa00000 -n -l
```

### 5.4.2 SquashFS

`SquashFS` 是专门为一般的只读文件系统的使用而设计，它可应用于数据备份，或是系统资源

紧张的电脑上使用。最初版本的 SquashFS 采用 gzip 的数据压缩。版本 2.6.34 之后的 Linux 内核增加了对 LZMA 和 LZO 压缩算法的支持，版本 2.6.38 的内核增加了对 LZMA2 的支持，该算法同时也是 xz 使用的压缩算法。

版本 2.6.35 之后的内核包含的 SquashFS 增加了扩展文件属性支持。

生成 SquashFS 文件系统镜像的命令是：

```
mksquashfs rootfs ./rootfs.squashfs -b 128K -comp xz
```

### 5.4.3 EXT4

EXT4 的全称是 Fourth extended filesystem，是 Linux 系统下的日志文件系统，是 EXT3 文件系统的后继版本。

生成 EXT4 文件系统镜像的命令是：

```
make_ext4fs -l 256M -s rootfs.ext4 rootfs
```

### 5.4.4 UBIFS

无序区块映像文件系统（Unsorted Block Image File System，UBIFS）是一种用于固态硬盘存储设备的文件系统，UBIFS 最早在 2006 年由 IBM 与 Nokia 的工程师 Thomas Gleixner，Artem Bityutskiy 所设计，专门为了解决 MTD（Memory Technology Device）设备所遇到的瓶颈。由于 Nand 闪存容量的暴涨，YAFFS 等皆无法再去控制 Nand 闪存的空间。UBIFS 透过子系统 UBI 处理与 MTD 设备之间的动作。与 JFFS2 一样，UBIFS 建构于 MTD 设备之上，因而与一般的块设备不兼容。

UBIFS 在设计与性能上均较 YAFFS2、JFFS2 更适合大容量的 NAND FLASH。例如，UBIFS 支持 write-back，其写入的资料会被缓存，直到有必要写入时才写到闪存，大大地降低分散小区块数量并提高 I/O 效率。UBIFS 文件系统目录存储在闪存上，UBIFS 挂载时不需要扫描整个闪存的资料来重新创建文件目录。

生成 UBIFS 文件系统镜像的命令是：

```
mkfs.ubifs -F -q -r /home/usr/fs -m 2048 -e 126976 -c 2047 -o ubifs.img
```

```
ubinize -o ubi.img -m 2048 -p 128KiB ubinize.cfg
```

AEXRA CONFIDENTIAL FOR SIPEED

# 6 日志系统

## 6.1 日志系统简介

日志系统是记录操作系统、驱动、应用所产生的 Log 的一套服务程序。这套服务程序主要由 syslogd、klogd、axsyslogd 以及 axklogd 组成。klogd 记录 Linux 内核原生的内核 Log，syslogd 根据/etc/syslog.conf 的配置来记录系统 Log，syslogd 和 klogd 是 busybox 自带的一组 log 服务程序，通过/etc/init.d/S01syslogd 和/etc/init.d/S02klogd 服务来启动，自动保存 log。而 axsyslogd 和 axklogd 是本 SDK 开发的一组记录 SDK 自研实现的非开源驱动 log 和 应用程序 log 的服务程序，应用程序 log 直接由 axsyslogd 记录，而 axklogd 记录驱动的 log 并转发给 axsyslogd，所有 log 最终由 axsyslogd 统一写入到同一个 log 文件中。

**！ 注意：**

SDK 中的日志总体包含两个部分：

- Linux 系统原生日志，包含 Linux 内核原生日志及基础应用程序日志；
- SDK 自研驱动日志及 SDK 自研应用、动态库、静态库产生的日志。

本章着重介绍 SDK 自研驱动日志及 SDK 应用、动态库、静态库产生的日志的控制、抓取以及查看等。

- 本章中提到的 user log，如未特别表明，均指 SDK 自研应用、动态库、静态库等用户态程序所产生的日志。
- 本章中提到的 kernel log，如未特别表明，均指 SDK 自研驱动程序所产生的日志。
- 本章中提到的 SDK Log，如未特别表明，均指 user log + kernel log。

## 6.2 syslogd 和 klogd

syslogd 和 klogd 是 busybox 自带的一组 log 服务程序，通过/etc/init.d/S01syslogd 和 /etc/init.d/S02klogd 服务来启动，自动将 Linux 系统原生 log 保存到文件。klogd 主要功能是获取 Linux 原生内核 log 并转发给 syslogd 进行存储，而 syslogd 除了接受来自 klogd 的 log 外，还接收使用了 syslog 接口的应用程序输出的 log（比如 ftp、crond 等用户进程），统一保存到

log 文件中，log 文件的存储路径可以通过/etc/syslog.conf 来配置。SDK 中/etc/syslog.conf 文件的默认配置如下：

```
auth.*          /var/log/auth.log
authpriv.*      /var/log/authpriv.log
cron.*          /var/log/cron.log
daemon.*        /var/log/daemon.log
ftp.*           /var/log/ftp.log
lpr.*           /var/log/lpr.log
mail.*          /var/log/mail.log
mark.*          /var/log/mark.log
news.*          /var/log/news.log
security.*      /var/log/security.log
syslog.*        /var/log/syslog.log
kern.*          /opt/data/AXSyslog/kernel/kernel.log
user.*          /opt/data/AXSyslog/kernel/user.log
uucp.*          /var/log/uucp.log
local0.*        /var/log/local0.log
local1.*        /var/log/local1.log
local2.*        /var/log/local2.log
local3.*        /var/log/local3.log
local4.*        /var/log/local4.log
local5.*        /var/log/local5.log
local6.*        /var/log/local6.log
local7.*        /var/log/local7.log
*.emerg         *
```

Linux 内核原生 log 被保存到了/opt/data/AXSyslog/kernel 路径下，如果你需要变更路径可根据实际需求更改 user.\*和 kern.\*对应的路径。需特别说明的是 user.\*配置的实际是 linux 内核原生 log 的输出，这个行为是由 busybox 的实现决定的，不同版本的 busybox 的行为可能会略有差异（SDK 中使用的 busybox 版本是 busybox-1.35.0）。

## 6.3 axsyslogd 和 axklogd

axsyslogd 和 axklogd 是本 SDK 开发的一组记录本 SDK 自研驱动 log(kernel log)和应用 log(user log)的服务程序，axklogd 记录 kernel log 并转发给 axsyslogd 进行存储，而 axsyslogd 除了接收来自 axklogd 转发的 log 外，还会记录调用了 AX\_SYS\_Log API 的应用程序(包括动态库、静态库等)log，这两部分 log 被 axsyslogd 一起写入到同一个 log 文件中进行存储。axsyslogd 和 axklogd 是一套类似 syslogd 和 klogd 的 log 服务程序，被统一集成到 axbox 中，该套 log 服务程序涉及到的主要文件如下表所示：

| 文件                    | 说明                                                 |
|-----------------------|----------------------------------------------------|
| /etc/ax_syslog.conf   | axsyslogd 配置文件                                     |
| /etc/init.d/axsyslogd | axsyslogd 启动脚本                                     |
| /etc/init.d/axklogd   | axklogd 启动脚本                                       |
| /sbin/axsyslogd       | 保存 SDK 自研程序的 log 到文件系统，依赖 ax_syslog.conf 配置        |
| /sbin/axklogd         | 将 SDK 自研驱动程序的 kernel log 转发到 axsyslogd             |
| /bin/axdmesg          | 实时查看 SDK 的 kernel log 输出，功能类似 dmesg                |
| /bin/axbox            | 类似 busybox 的一个工具箱，包了 axsyslogd、axklogd、axdmesg 等工具 |

### 6.3.1 axsyslogd

axsyslogd 是本 SDK 自研开发的一个 log 服务程序，其主要职责有两个：

- 将 user log 和 kernel log 统一写入 log 文件；
- 根据 ax\_syslog.conf 配置文件，将各模块的 log 等级信息控制写入共享内存，以控制 user log 和 kernel log 的输出。

axsyslogd 对应由一个配置文件 ax\_syslog.conf，其默认路径是/etc/ax\_syslog.conf，但该文件的位置是可配置的，配置方法是修改工程对应的 dts 文件，以 AX630C\_emmc\_arm64\_k419 工程为例，其 dts 文件为：

```
linux-4.19.125/arch/arm64/boot/dts/axera/AX630C_emmc_arm64_k419.dts
```

其配置内容为：

```
axera_logctl {
    compatible = "axera,logctl";
    config = "/etc/ax_syslog.conf";
    logstate = /bits/ 8 <1 1>;
    loglevel = /bits/ 8 <4 4>;
};
```

如果您有变更 ax\_syslog.conf 文件位置的需求，可在对应的 dts 文件中去修改 config 的配置，关于 axera\_logctl 更详细的配置说明请参考[配置 dts](#) 章节，ax\_syslog.conf 文件内容的说明请参考[配置 ax\\_syslog.conf](#) 章节。

### 6.3.2 axklogd

axklogd 主要的功能是读取本 SDK 自研驱动程序的 kernel log 并转发到 axsyslogd 进程进行存储。需要注意的是，如果重启了 axsyslogd 进程，axklogd 进程也必须重启，且必须保持 axsyslogd 先启动、axklogd 后启动的顺序，否则 axklogd 无法将 log 转发给 axsyslogd，从而造成 log 丢失。

## 6.4 日志系统的部署

日志系统的部署，主要包括两个部分：

- 系统原生 log 的部署，即 syslogd 及 klogd 的部署；
- 本 SDK 自研 log 系统的部署，即 axsyslogd 及 axklogd 的部署。

6.4.1 系统原生日志的部署

Linux 系统原生日志主要通过 syslogd 和 klogd 进行维护。syslogd 和 klogd 是 busybox 自带的一组 log 服务程序，它们分别通过/etc/init.d/S01syslogd 和/etc/init.d/S02klogd 服务来启动。

这组日志服务程序设计到的主要资源文件如下表所示。

资源文件列表

| 文件                     | 说明                                                               |
|------------------------|------------------------------------------------------------------|
| /bin/busybox           | busybox 文件                                                       |
| /sbin/syslogd          | syslog 服务程序，实现 syslog、klog 的保存                                   |
| /sbin/klogd            | klog 服务程序，实现 linux 原生 kernel log 的读取并转发给 /sbin/syslogd 进行存储      |
| /etc/init.d/S01syslogd | syslogd 启动、停止脚本                                                  |
| /etc/init.d/S02klogd   | klogd 启动、停止脚本，必须在 syslogd 启动后才能启动该脚本，如果 syslogd 重启过，klogd 也必须重启。 |
| /etc/syslog.conf       | syslogd 配置文件                                                     |

资源部署

默认情况下，busybox、syslogd、klogd、S01syslogd、S02klogd 这些文件都是按照上表的路径部署的，如果您有路径的定制化需求，只需将 busybox、syslogd、klogd 加入到 PATH 环境变量可查询到的路径即可，而 S01syslogd、S02klogd 通过/etc/init.d/rcS 来开机自启动即可。

## 修改 log 保存路径

/etc/syslog.conf 文件需做特别说明：

```
user.* /opt/data/AXSyslog/kernel/user.log
```

该配置项指定了 linux 原生 kernel log 的输出路径，您可以修改配置项来更改 Log 保存路径，同时需要将/etc/init.d/rcS 文件中的：

```
mkdir -p /opt/data/AXSyslog/kernel
```

修改为您希望的 log 存储路径。

## 修改 log 保存大小

```
SYSLD_ARGS="-s 5120 -b 99 -S"
```

“-s 5120”配置项指定了 user.log 单个文件的大小为 5MB，“-b 99”指定了最多可保存 99 个 log 文件，当 log 文件超过 99 个时，log 文件将被循环覆盖。

## 修改 syslog.conf 文件路径

/etc/syslog.conf 是 busybox 系统内部指定的配置文件路径，如果有修改该路径的需求，有两种方式：

方式 1）：修改/etc/init.d/S01syslogd 脚本

将：

```
SYSLD_ARGS="-s 5120 -b 99 -S"
```

修改为：

```
SYSLD_ARGS="-f /mydir/syslog.conf -s 5120 -b 99 -S"
```

通过-f 指定配置文件路径，/mydir/syslog.conf 为新的配置文件路径(请根据实际情况修改)。

方式 2）：修改 busybox 源码

修改 syslogd/syslogd.c 的 parse\_syslogdcfg 函数：

```
static void parse_syslogdcfg(const char *file)
{
    char *t;
    logRule_t **pp_rule;
    /* tok[0] set of selectors */
    /* tok[1] file name */
    /* tok[2] has to be NULL */
    char *tok[3];
    parser_t *parser;

    parser = config_open2(file ? file : "/etc/syslog.conf",
                          file ? xfopen_for_read : fopen_for_read);
```

图 6-1

将默认的/etc/syslog.conf 文件路径改为您希望的路径，这样在不使用-f 参数的情况下，会默认通过该路径来读取配置。

6.4.2 SDK 自研日志系统的部署

本 SDK 自研日志系统对日志的管理工作主要由 axsyslogd 和 axklogd 完成。axsyslogd 和 axklogd 是本 SDK 开发的一组记录自研实现的 kernel log（非开源 ko）和 user log（应用程序、动态库、静态库等）的服务程序，axklogd 记录 kernel log，而 axsyslogd 记录 user log，这两部分 log 被统一写入到同一个 log 文件中。这组日志服务程序设计到的主要资源文件如下表所示。

资源文件列表

| 文件              | 说明                                                                   |
|-----------------|----------------------------------------------------------------------|
| /bin/axbox      | busybox 文件                                                           |
| /sbin/axsyslogd | syslog 服务程序，实现 syslog、klog 的保存                                       |
| /sbin/axklogd   | axklog 服务程序，实现 ax_printk 接口的 kernel log 的读取并转发给 /sbin/axsyslogd 进行存储 |

|                            |                                                                                                                                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /etc/init.d/axsyslogd      | axsyslogd 启动、停止脚本                                                                                                                                                                                |
| /etc/init.d/axklogd        | axklogd 启动、停止脚本，必须在 axsyslogd 启动后才能启动该脚本，如果 axsyslogd 重启过，klogd 也必须重启。                                                                                                                           |
| /etc/ax_syslog.conf        | axsyslogd 配置文件                                                                                                                                                                                   |
| /bin/axdmesg               | 功能与 dmesg 类似，可通过该指令查看最新的 ax_printk 接口输出的 kernel log.                                                                                                                                             |
| AX630C_emmc_arm64_k419.dts | 这里是以 AX630C_emmc_arm64_k419 工程为例，其 dts 配置文件路径是：linux-4.19.125/arch/arm64/boot/dts/axera/AX630C_emmc_arm64_k419.dts，其 axera_logctl 相关配置，配置了 user log、kernel log 的默认等级，以及 ax_syslog.conf 的存放路径等内容。 |

## 资源部署

axbox、axsyslogd、axklogd 和 axdmesg 只需存放到 PATH 可搜寻到的路径即可，axsyslogd 和 axklogd 需要通过/etc/init.d/rcS 来加入开机启动：

```
/etc/init.d/axsyslogd start
/etc/init.d/axklogd start
```

## 配置 dts

AX620E SDK 中，每个工程都有对应的 dts 配置文件，以 AX630C\_emmc\_arm64\_k419 工程为例，其 dts 配置文件路径是：

```
linux-4.19.125/arch/arm64/boot/dts/axera/AX630C_emmc_arm64_k419.dts
```

其中 axera\_logctl 的配置如下：

```
axera_logctl {  
    compatible = "axera,logctl";  
    config = "/etc/ax_syslog.conf";  
    logstate = /bits/ 8 <1 1>;  
    loglevel = /bits/ 8 <4 4>;  
};
```

其中，`config` 配置的是 `ax_syslog.conf` 配置文件路径，您可以修改为您需要的路径，比如：`/etc/init.d/my_syslog.conf`，这样 `axsyslogd` 会自动去读取 `/etc/init.d/my_syslog.conf` 文件的配置信息。

`logstate` 的配置中“`/bits/ 8`”表示该配置项 8 bit 位宽，保持不变即可。`<1 1>` 分别代表 user log 和 kernel log 的默认状态，1 表示默认开启 log，0 表示默认关闭 log，这在开机阶段还未加载 `ax_syslog.conf` 配置时有效，在加载 `ax_syslog.conf` 配置后将会被 `ax_syslog.conf` 中的配置覆盖。

`loglevel` 中 `<4 4>` 分别代表 user log 和 kernel log 的默认 log 等级，这在开机阶段还未加载 `ax_syslog.conf` 配置时有效，在加载 `ax_syslog.conf` 配置后将会被 `ax_syslog.conf` 中的配置覆盖。

## 配置 ax\_syslog.conf

`ax_syslog.conf` 配置文件默认内容如下：

```
# 第一部分功能：Log 保存配置  
AX_SYSLOG_enable = 1          #开启 axsyslog, 1:开启, 0:关闭。  
AX_SYSLOG_path = /opt/data    #log 保存路径。  
AX_SYSLOG_percent = 80        #磁盘空间使用百分比, 1) 使用时超过该值, Log 开始循环覆盖;  
2) 启动时超过该值, Log 可能被丢弃。  
AX_SYSLOG_strategy = 0        #磁盘空间受限后, 新 log 是覆盖旧 log, 还是丢弃。0:覆盖,  
1:丢弃。  
AX_SYSLOG_filesizemax = 50    #单个 Log 文件大小, 单位:MB。  
AX_SYSLOG_volume = 1024       #所有 log 占用的总磁盘空间, 单位:MB。  
AX_SYSLOG_minimalspace = 256 #当磁盘使用百分比超过 AX_SYSLOG_percent 时, 如果磁盘剩  
余值大于该值, 仍然可保存 log。  
# 第二部分功能：Log 等级配置
```

```
#模块 Log 等级设置
ulog_state = on
ulog_target = file
klog_state = on
#module id level(user) level(kernel)
ISP      1      4      4
CE        2      4      4
VO        3      4      4
VDSP      4      4      4
EFUSE     5      4      4
NPU       6      4      4
VENC      7      4      4
VDEC      8      4      4
JENC      9      4      4
JDEC     10      4      4
SYS      11      4      4
AENC     12      4      4
IVPS     13      4      4
...
```

该文件配置包含两部分功能：

## log 存储配置

log 存储配置，说明如下：

- 1> AX\_SYSLOG\_enable 1：保存 log，0：不保存 log（log 仍然可能在输出，只是未存储）；
- 2> AX\_SYSLOG\_path log 存储路径，不支持 ramfs 分区。例如，通过 mount -t ramfs /dev/xxx /log/dir 得到的/log/dir 无法将 Log 保存下来，原因是这种方式得到的分区无法获取分区大小，请避免使用该方式得到的分区，如果需要使用 ram 分区来保存 Log，推荐使用 tmpfs 分区；
- 3> AX\_SYSLOG\_percent 为避免磁盘被写 log 写满，通过该配置值，控制 Log 的输出量，当磁盘占用空间超过该值时，log 系统会进行循环覆盖或停止保存 log（由 AX\_SYSLOG\_strategy 决定）。

4> AX\_SYSLOG\_strategy log 存储策略, 磁盘空间受限后, 新 log 是覆盖旧 log, 还是丢弃。  
0:覆盖, 1:丢弃。

5> AX\_SYSLOG\_filesizemax 单个 log 文件的大小, 请务必保证至少可以保存两个 log 文件,  
即  $AX\_SYSLOG\_filesizemax * 2 \leq AX\_SYSLOG\_volume$ ;

6> AX\_SYSLOG\_volume 存储 log 的总大小, 当存储的 log 文件大小超过该值时, 开始循环覆盖或者丢弃 log (由 AX\_SYSLOG\_strategy 决定);

7> AX\_SYSLOG\_minimalspace log 系统会尽可能多的保存 Log, 以 2GB 的总空间为例, AX\_SYSLOG\_percent = 80, AX\_SYSLOG\_minimalspace = 256 的配置, 当磁盘写到 1638.4MB 时, 剩余的可用空间为 409.6MB, 这个值大于 AX\_SYSLOG\_minimalspace, 所以 Log 会继续保存, 直到可用空间小于 256MB, Log 开始循环覆盖。相反, 如果总空间为 1GB, AX\_SYSLOG\_percent = 80, AX\_SYSLOG\_minimalspace = 256 的配置, 那么当可用空间小于 AX\_SYSLOG\_minimalspace 时, Log 仍然会保存到磁盘的 80%, 然后开始循环覆盖。请根据你的实际磁盘空间来配置这两项值。

## log 等级配置

log 等级配置, 包含 user log 和 kernel log 等级的配置说明如下:

```
#模块 Log 等级设置
ulog_state = on
ulog_target = file
klog_state = on
```

ulog\_state: 配置 user log 的输出与否, 可选配置: on/off, on: 输出 Log, off: 关闭 Log。

ulog\_target: 配置 user log 的输出目的地, 可选配置: file/console/null, file: Log 保存到文件, console: Log 输出到终端, null: Log 被丢弃, 相当于配置了 ulog\_state=off。

klog\_state: 配置 kernel log 的输出与否, 可选配置: on/off, on: 输出 Log, off: 关闭 Log。

各模块 Log 等级配置:

```
#module id level(user) level(kernel)
ISP      1      4      4
CE       2      4      4
```

|     |   |   |   |
|-----|---|---|---|
| VO  | 3 | 4 | 4 |
| ... |   |   |   |

第一列：module 是模块名称，最大长度 11 字符(超过将会被截断)，可自定义命名；

第二列：是模块 id，模块的唯一标识，取值范围：0~255，其中 0~127 为 SDK 内部使用 id，128~255 共客户自定义使用；

第三列：level(user)是模块对应的 user log 输出等级，可配置的等级为 0-7，值越小，等级越高，0-7 对应的 Log 等级如下：

| 值 | 等级        |
|---|-----------|
| 0 | EMERGENCY |
| 1 | ALERT     |
| 2 | CRITICAL  |
| 3 | ERROR     |
| 4 | WARN      |
| 5 | NOTICE    |
| 6 | INFO      |
| 7 | DEBUG     |

第四列：level(kernel) 是模块对应的 kernel log 输出等级，可配置的等级为 0-7，值越小，等级越高，0-7 对应的 log 等级与 user log 一致。

6.4.3 只读文件系统的日志部署

SDK Log 存储的位置，根据配置/etc/ax\_syslog.config 文件：

```
AX_SYSLOG_path = /opt/data
```

配置可知位于/opt/data/ASyslog/syslog 目录，log 文件是以“日期\_时间.log”形式命名的，如：2023-05-11\_15-22-26.log。Log 文件的大小根据配置：

```
AX_SYSLOG_filesizemax = 50
```

SDK 中提供的默认值是 50MB，当 log 文件超过 50MB 时，会建立一个新的文件。当总的日志文件大小达到配置：

```
AX_SYSLOG_volume = 1024
```

时，将会循环覆盖，删除最早创建的 log 文件，并创建新的 log 文件，如此循环。

## user log 输出说明

user log 输出的格式如下：

```
系统时间 Axera: [内核时间] [模块名][log 类型][进程 id] Log 内容
```

其中：

系统时间：log 输出时的系统时间

[内核 Log 时间] 从开机开始计算的内核时间

[CPU] AX620E CPU 为 2 核，取值区间为 C0-C1

[log 类型]

M: eMergency

A: Alert

C: Critical

E: Error

W: Warning

N: Notic

I: Information

D: Debug

[进程 id] 输出 Log 所属进程 ID

[模块名] 输出 Log 所属模块

示例 Log 如下：

```
2023-05-09 19:43:36 AXera: [ 6783.744241] [AX_VDEC] [W] [1570] [tid:1613,
AxVdDot10] [AX_DecPicInfo_Print] [711] all pp disabled! 265
2023-05-09 19:43:36 AXera: [ 6783.744262] [AX_VDEC] [W] [1570] [tid:1618,
AxVdDewt11] [AX_VsiDecProcess] [1769] VdGrp=11, 1 DEC_STRM_PROCESSED
```

## kernel log 输出说明

通过 ax\_printk 输出的 Log 格式如下：

```
系统时间 kernel: [内核时间] [CPU] [模块名] [log 类型] [线程 id] Log 内容
```

其中：

系统时间：log 输出时的系统时间

[内核 Log 时间] 从开机开始计算的内核时间

[CPU] AX620E CPU 为 2 核，取值区间为 C0-C1

[log 类型]

M: eMergency

A: Alert

C: Critical

E: Error

W: Warning

N: Notice

I: Information

D: Debug

[线程 id] 输出 Log 所属线程 ID

[模块名] 输出 Log 所属模块

示例 Log 如下：

```
2023-05-09 19:43:36 kernel: [ 6783.908171] [C0][VO][E][1570]
[ __draw_chn_disable:176] layer0-chn58 status(0x81) invalid
2023-05-09 19:43:36 kernel: [ 6784.059040] [C0][ivps][E][1570]
[axvpp_wq_destroy:1436] error result:0x7ff
```

根据以上 user log 和 kernel log 的输出说明可知，我们保持了 user log 与 kernel log 时间的一致，包括系统时间的一致，以及内核时间的一致(这里的内核时间，是指从开机开始计算的时间)，这对对比分析 user log 与 kernel log 非常有帮助。

## 6.5 集成自定义模块 Log 到日志系统

### 6.5.1 集成应用程序 Log

要集成应用程序 Log 到 SDK 自研日志系统，主要涉及到 4 部分内容：

- 1) .h 头文件引用，主要包括：ax\_global\_type.h、ax\_sys\_api.h 和 ax\_sys\_log.h；
- 2) ax\_syslog.conf 配置文件，添加对应的配置；
- 3) API 调用，主要包括：AX\_SYS\_Init()、AX\_SYS\_Deinit()和 AX\_SYS\_LogPrint\_Ex()；
- 4) 库文件引用：libax\_sys.so 或 libax\_sys.a。

为更好说明如何集成应用程序 Log 到我们的日志系统，下面举例演示。

步骤 1：在 ax\_global\_type.h 文件的 AX\_MOD\_ID\_E 枚举中添加自定义模块 ID；ax\_global\_type.h 文件位于 SDK 的 msp/out/include 目录，其 AX\_MOD\_ID\_E 枚举包含了所有模块 ID 的定义：

```
typedef enum
{
    AX_ID_MIN      = 0x00,
    AX_ID_ISP      = 0x01,
    AX_ID_CE       = 0x02,
    AX_ID_VO       = 0x03,
    AX_ID_VDSP     = 0x04,
    . . . . .
    /* for customer*/
    AX_ID_CUST_MIN = 0x80, /* 128 */
    AX_ID_MAX      = 0xFF /* 255 */
} AX_MOD_ID_E;
```

AX\_ID\_CUST\_MIN 之前的 ID 是供 SDK 内部使用的，AX\_ID\_CUST\_MIN 及之后是供客户定制化使用的，所以增加模块的时候，请在 AX\_ID\_CUST\_MIN 之后增加，避免与 SDK 产生冲突。

步骤 2：在 ax\_syslog.conf 文件中添加配置；

步骤 3：在源文件中调用 AX\_SYS API；

API 的调用共涉及到 3 个：AX\_SYS\_Init()、AX\_SYS\_Deinit()、AX\_SYS\_LogPrint\_Ex()。其中 AX\_SYS\_Init()和 AX\_SYS\_Deinit()是系统初始化和去初始化，AX\_SYS\_LogPrint\_Ex()是 Log 输出 API。

步骤 4：在 Make file 中指定 libax\_sys 库路径。

libax\_sys.so/libax\_sys.a 位于 SDK 的 msp\out\\$(ARCH)\_\$(LIBC)\lib 目录，步骤 2 中引用的 AX\_SYS API 都位于 libax\_sys 库中。

demo 程序代码如下：

ax\_global\_type.h 文件,新增 AX\_ID\_CUST1、AX\_ID\_CUST2、AX\_ID\_CUST3 共 3 个 ID，其 ID 值从 128 开始递增：

```
typedef enum
{
```

```

    AX_ID_MIN      = 0x00,
    AX_ID_ISP      = 0x01,
    AX_ID_CE       = 0x02,
    AX_ID_VO       = 0x03,
    AX_ID_VDSP     = 0x04,
    .....
    /* for customer*/
    AX_ID_CUST_MIN = 0x80, /* 128 */
    AX_ID_CUST1 = AX_ID_CUST_MIN,
    AX_ID_CUST2,
    AX_ID_CUST3,
    .....
    AX_ID_MAX      = 0xFF /* 255 */
} AX_MOD_ID_E;

```

ax\_syslog.conf 文件:

```

#module id level(user) level(kernel)
ISP      1      4      4
CE       2      4      4
VO       3      4      4
VDSP     4      4      4
...
CUST1    128    4      4
CUST2    129    4      4
CUST3    130    4      4

```

**注意:**

**module** 列字符长度不能超过 11 字节，超出将被截断。

sample\_olog.c 文件:

```

#include <stdio.h>
#include <stdlib.h>

```

```
#include <string.h>
#include <signal.h>
#include <errno.h>
#include <unistd.h>
#include <pthread.h>
#include "ax_sys_log.h"
#include "ax_sys_api.h"
```

```
int id_list[] = {
    AX_ID_CUST1,
    AX_ID_CUST2,
    AX_ID_CUST3,
};
```

```
char* tag_list[] = {
    "CUST1",
    "CUST2",
    "CUST3",
};
```

```
int level_list[] = {
    SYS_LOG_EMERGENCY,
    SYS_LOG_ALERT,
    SYS_LOG_CRITICAL,
    SYS_LOG_ERROR,
    SYS_LOG_WARN,
    SYS_LOG_NOTICE,
    SYS_LOG_INFO,
    SYS_LOG_DEBUG,
};
```

```
static int gLoopExit = 0;

static AX_VOID __sigint(int iSigNo)
{
    printf("Catch signal %d\n", iSigNo);
    gLoopExit = 1;

    return ;
}

int main(int argc, char **argv)
{
    int ret = 0;
    int i;
    int cnt = sizeof(id_list)/sizeof(id_list[0]);
    int level;
    signal(SIGINT, __sigint);

    // init
    ret = AX_SYS_Init();
    if (AX_SUCCESS != ret) {
        printf("AX_SYS_Init failed! Error Code:0x%X\n", ret);
        return -1;
    }

    while(gLoopExit==0){
        for (level=SYS_LOG_DEBUG; level>=SYS_LOG_EMERGENCY; level--){
            for (i=0; i< cnt; i++){
                AX_SYS_LogPrint_Ex(level, tag_list[i], id_list[i], "This log's
id: %d leve:%d\n", id_list[i], level);
                if (gLoopExit) break;
            }
        }
    }
}
```

```
    }  
    if (gLoopExit) break;  
    usleep(1000000);  
}  
}  
  
//deinit  
ret = AX_SYS_Deinit();  
if (AX_SUCCESS != ret) {  
    printf("AX_SYS_Deinit failed! Error Code:0x%X\n", ret);  
    return -1;  
}  
return 0;  
}
```

Makefile 文件:

```
CUR_PATH      := $(shell pwd)  
HOME_PATH     := $(abspath $(CUR_PATH)/../../..)  
  
include $(HOME_PATH)/build/color.mk  
include $(HOME_PATH)/build/config.mak  
  
PRJ_OUT_HOME  := $(HOME_PATH)/build/out/$(PROJECT)  
OBJ_OUT_PATH  := $(PRJ_OUT_HOME)/objs  
MSP_OUT_PATH  := $(HOME_PATH)/msp/out  
SRC_PATH      := $(CUR_PATH)  
COMMON_PATH   := $(CUR_PATH)/../common  
LIB_PATH      := $(MSP_OUT_PATH)/lib  
SRC_RELATIVE_PATH := $(subst $(HOME_PATH)/,,$(SRC_PATH))  
TARGET_OUT_PATH := $(OBJ_OUT_PATH)/$(SRC_RELATIVE_PATH)  
ROOTFS_TARGET_PATH := $(MSP_OUT_PATH)  
HEADER_EXT    := $(HOME_PATH)/header/external
```

```
OUT_PATH          := $(HOME_PATH)/msp/out

# output
MOD_NAME          := sample_uleg
OUTPUT            := .obj

# source
SRCS              := $(wildcard $(SRC_PATH)/*.c)

CINCLUDE          += -I$(SRC_PATH) \
                    -I$(OUT_PATH)/include \
                    -I$(BASE_PATH)/sys \
                    -I$(COMMON_PATH)/include \
                    -I$(HEADER_EXT)

OBJS              := $(addprefix $(OUTPUT)/,$(notdir
$(patsubst %.c,%.o,$(SRCS))))
DEPS              := $(OBJS:%.o=%.d)

# exec
TARGET            := $(MOD_NAME)

# build flags
ifeq ($(debug),yes)
CFLAGS            += -Wall -O0 -ggdb3
else
CFLAGS            += -Wall -O2
endif

# dependency
CLIB              += -L$(OUT_PATH)/lib
CLIB              += -lax_sys
```

```
# CLIB          += -lm -lpthread

# install
INSTALL_BIN    := $(TARGET)

# link
LINK = $(CC)

include $(HOME_PATH)/build/rules.mak
```

编译后生成 `sample_ulo` 可执行程序，运行后的 Log 输出：

```
2023-05-12 13:52:05 AXera: [62574.756935] [CUST1][W][5130] This log's id: 128
leve:4
2023-05-12 13:52:05 AXera: [62574.756995] [CUST2][W][5130] This log's id: 129
leve:4
2023-05-12 13:52:05 AXera: [62574.757014] [CUST3][W][5130] This log's id: 130
leve:4
2023-05-12 13:52:06 AXera: [62575.757934] [CUST1][E][5130] This log's id: 128
leve:3
2023-05-12 13:52:06 AXera: [62575.757977] [CUST2][E][5130] This log's id: 129
leve:3
2023-05-12 13:52:06 AXera: [62575.757995] [CUST3][E][5130] This log's id: 130
leve:3
2023-05-12 13:52:07 AXera: [62576.758932] [CUST1][C][5130] This log's id: 128
leve:2
2023-05-12 13:52:07 AXera: [62576.758972] [CUST2][C][5130] This log's id: 129
leve:2
2023-05-12 13:52:07 AXera: [62576.758989] [CUST3][C][5130] This log's id: 130
leve:2
2023-05-12 13:52:08 AXera: [62577.759942] [CUST1][A][5130] This log's id: 128
leve:1
```

```
2023-05-12 13:52:08 AXera: [62577.759998] [CUST2][A][5130] This log's id: 129
leve:1
2023-05-12 13:52:08 AXera: [62577.760017] [CUST3][A][5130] This log's id: 130
leve:1
2023-05-12 13:52:09 AXera: [62578.760941] [CUST1][M][5130] This log's id: 128
leve:0
2023-05-12 13:52:09 AXera: [62578.760996] [CUST2][M][5130] This log's id: 129
leve:0
2023-05-12 13:52:09 AXera: [62578.761013] [CUST3][M][5130] This log's id: 130
leve:0
```

cat /proc/ax\_proc/logctl 查看当前 Log 等级配置:

| module | id  | level(U) | level(K) |
|--------|-----|----------|----------|
| ISP    | 1   | 4        | 4        |
| CE     | 2   | 4        | 4        |
| VO     | 3   | 4        | 4        |
| VDSP   | 4   | 4        | 4        |
| EFUSE  | 5   | 4        | 4        |
| NPU    | 6   | 4        | 4        |
| VENC   | 7   | 4        | 4        |
| VDEC   | 8   | 4        | 4        |
| JENC   | 9   | 4        | 4        |
| JDEC   | 10  | 4        | 4        |
| SYS    | 11  | 4        | 4        |
| AENC   | 12  | 4        | 4        |
| IVPS   | 13  | 4        | 4        |
| MIPI   | 14  | 4        | 4        |
| ADEC   | 15  | 4        | 4        |
| DMA    | 16  | 4        | 4        |
| VIN    | 17  | 4        | 4        |
| USER   | 18  | 4        | 4        |
| IVES   | 19  | 4        | 4        |
| SKEL   | 20  | 4        | 4        |
| IVE    | 21  | 4        | 4        |
| 3A     | 25  | 4        | 4        |
| AI     | 32  | 4        | 4        |
| A0     | 33  | 4        | 4        |
| SENSOR | 34  | 4        | 4        |
| CUST1  | 128 | 4        | 4        |
| CUST2  | 129 | 4        | 4        |
| CUST3  | 130 | 4        | 4        |

图 6-2

至此，应用程序 Log 的集成完成。

## 6.5.2 集成驱动程序 Log

要集成驱动程序 Log 到 SDK 自研日志系统，则相对更加简单，只需要三个步骤：

- 1) 在 ax\_global\_type.h 的 AX\_MOD\_ID\_E 中添加自定义模块 ID；
- 2) ax\_syslog.conf 配置文件，添加对应的配置；
- 3) 在驱动程序中调用 ax\_printk API 打印 Log。

为更好说明如何集成驱动程序 Log 到我们的日志系统，下面举例演示。

步骤 1：在 ax\_global\_type.h 文件的 AX\_MOD\_ID\_E 枚举中添加自定义模块 ID，方式同[集成应用程序 Log](#)介绍的一样，如果应用程序已经添加过对应的模块，则不用重复添加；

步骤 2：在驱动程序源码中调用 ax\_printk 接口输出 Log；

为更好说明如何集成驱动程序 Log 到我们的日志系统，下面举例演示。

demo 程序代码(文件路径：kernel\osdrv\klog)如下：

ax\_global\_type.h 文件，新增 AX\_ID\_CUST1、AX\_ID\_CUST2、AX\_ID\_CUST3 共 3 个 ID，操作同[集成应用程序 Log](#)中介绍的一样，如果应用程序已经添加过对应的模块，则不用重复添加。

Makefile 文件

```
MKFILE_PATH      := $(abspath $(lastword $(MAKEFILE_LIST)))
HOME_PATH         := $(abspath $(dir $(MKFILE_PATH))../..)
BUILD_PATH        := $(HOME_PATH)/build
ROOTFS_PATH       := $(HOME_PATH)/rootfs

include $(BUILD_PATH)/config.mak

OBJ_OUT_DIR       := $(BUILD_PATH)/out/$(PROJECT)/objs
KSRC              := $(abspath $(dir $(MKFILE_PATH)))
SRC_RELATIVE_PATH := $(subst $(HOME_PATH)/,,$(KSRC))
KBUILD_DIR        := $(OBJ_OUT_DIR)/$(SRC_RELATIVE_PATH)
```

```
KBUILD_MAKEFILE      := $(KBUILD_DIR)/Makefile
MSP_OUT_PATH          := $(HOME_PATH)/msp/out

EXTRA_CFLAGS += -I$(MSP_OUT_PATH)/include/

KDIR ?= $(HOME_PATH)/kernel/linux/$(KERNEL_DIR)
MODULE_NAME := test_klog

SRCS := testklog.c

$(MODULE_NAME)-objs := $(SRCS:%.c=%.o)

obj-m := $(MODULE_NAME).o

clean-objs := $(SRCS:%.c=%.o)
clean-objs += $(join $(dir $(SRCS)), $(patsubst %.c, %.o.cmd, $(notdir $(SRCS))))

include $(BUILD_PATH)/krules.mak
```

### testklog.c 文件

```
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/slab.h>
#include <linux/delay.h>
#include "ax_global_type.h"

#define TEST_KLOG_PROC_NAME "ax_proc/test_klog"

int id_list[] = {
```

```
    AX_ID_CUST1,  
    AX_ID_CUST2,  
    AX_ID_CUST3,  
};  
  
char* tag_list[] = {  
    "CUST1",  
    "CUST2",  
    "CUST3",  
};  
  
static int test_klog(void)  
{  
    int ret = 0;  
    int i, level;  
    int cnt = sizeof(id_list)/sizeof(id_list[0]);  
    for (level=SYS_LOG_DEBUG; level>=SYS_LOG_EMERGENCY; level--){  
        for(i=0; i < cnt; i++){  
            ax_printk(id_list[i], tag_list[i], level, "This log's id:%d,  
level:%d", id_list[i], level);  
        }  
    }  
    return ret;  
}  
  
static ssize_t logctl_proc_write(struct file *file, const char *buffer,  
size_t count, loff_t *f_pos)  
{  
    char *tmp = kzalloc((count + 1), GFP_KERNEL);  
    char cmd[64];
```

```
int ret;

if (!tmp)
    return -ENOMEM;
if (copy_from_user(tmp, buffer, count)) {
    pr_err("@%s copy_from_user failed!!!\n", __FUNCTION__);
    kfree(tmp);
    return -EFAULT;
}

ret = sscanf(tmp, "%s", cmd);
if (0 == strcmp(cmd, "test")){
    test_klog();
}

kfree(tmp);
tmp = NULL;

return count;
}

static int axlogctl_proc_show(struct seq_file *p, void *v)
{
    return 0;
}

static int logctl_proc_open(struct inode *inode, struct file *file)
{
    return single_open(file, axlogctl_proc_show, NULL);
}
```

```
static const struct proc_ops proc_axlogctl_operations = {
    .proc_flags = PROC_ENTRY_PERMANENT,
    .proc_open  = logctl_proc_open,
    .proc_read  = seq_read,
    .proc_write = logctl_proc_write,
    .proc_release = single_release,
};

static int __init test_klog_proc_init(void)
{
    proc_create(TEST_KLOG_PROC_NAME, S_IRUSR, NULL,
&proc_axlogctl_operations);
    return 0;
}

static void __exit test_klog_proc_exit(void)
{
    remove_proc_entry(TEST_KLOG_PROC_NAME, NULL);
}

module_init(test_klog_proc_init);
module_exit(test_klog_proc_exit);

MODULE_AUTHOR("axera");
MODULE_LICENSE("GPL");
```

编译后生成 test\_klog.ko 驱动程序，insmod 后会生成 /proc/ax\_proc/test\_klog 节点，执行：

```
echo test > /proc/ax_proc/test_klog
```

即可看到 /opt/data/AXSyslog/syslog 目录下的 Log 文件中查看到 Log 输出：

```
2023-05-12 14:26:32 kernel: [ 61.314683] [C0][CUST1][W][538] This log's
id:128, level:4
```

```
2023-05-12 14:26:32 kernel: [ 61.314693] [C0][CUST2][W][538] This log's
id:129, level:4
2023-05-12 14:26:32 kernel: [ 61.314695] [C0][CUST3][W][538] This log's
id:130, level:4
2023-05-12 14:26:32 kernel: [ 61.314696] [C0][CUST1][E][538] This log's
id:128, level:3
2023-05-12 14:26:32 kernel: [ 61.314698] [C0][CUST2][E][538] This log's
id:129, level:3
2023-05-12 14:26:32 kernel: [ 61.314700] [C0][CUST3][E][538] This log's
id:130, level:3
2023-05-12 14:26:32 kernel: [ 61.314702] [C0][CUST1][C][538] This log's
id:128, level:2
2023-05-12 14:26:32 kernel: [ 61.314703] [C0][CUST2][C][538] This log's
id:129, level:2
2023-05-12 14:26:32 kernel: [ 61.314705] [C0][CUST3][C][538] This log's
id:130, level:2
2023-05-12 14:26:32 kernel: [ 61.314707] [C0][CUST1][A][538] This log's
id:128, level:1
2023-05-12 14:26:32 kernel: [ 61.314708] [C0][CUST2][A][538] This log's
id:129, level:1
2023-05-12 14:26:32 kernel: [ 61.314710] [C0][CUST3][A][538] This log's
id:130, level:1
2023-05-12 14:26:32 kernel: [ 61.314711] [C0][CUST1][M][538] This log's
id:128, level:0
2023-05-12 14:26:32 kernel: [ 61.314713] [C0][CUST2][M][538] This log's
id:129, level:0
2023-05-15 14:26:32 kernel: [ 61.314715] [C0][CUST3][M][538] This log's id:130,
level:0
```

至此，驱动程序 Log 的集成完成。

## 6.6 动态修改 Log 等级

### 6.6.1 动态修改 Linux 内核原生 Log 等级

对于 Linux 原生内核 log，即通过 `printk` 输出的 log，可以通过 `/proc/sys/kernel/printk` 节点来动态控制输出到串口的 linux log 等级。默认情况下，Linux 原生内核 Log 的等级配置为 4(warning)，这可以通过 `cat /proc/sys/kernel/printk` 来查看：

```
# cat /proc/sys/kernel/printk
4      4      1      7
```

如果要修改输出到终端的 Log 等级，比如修改到等级 7，可以通过下面指令来实现：

```
echo 7 > /proc/sys/kernel/printk
```

关于 `printk` log 等级设置的更多用法，可参考内核文档 `linux-5.15.73/Documentation/core-api/printk-basics.rst`。

### 6.6.2 SDK 自研日志系统 log 等级控制流程

本 SDK 自研日志系统 Log 等级控制的基本流程如下图所，配置文件 `ax_syslog.conf` 文件包含了 log 等级配置基本信息。

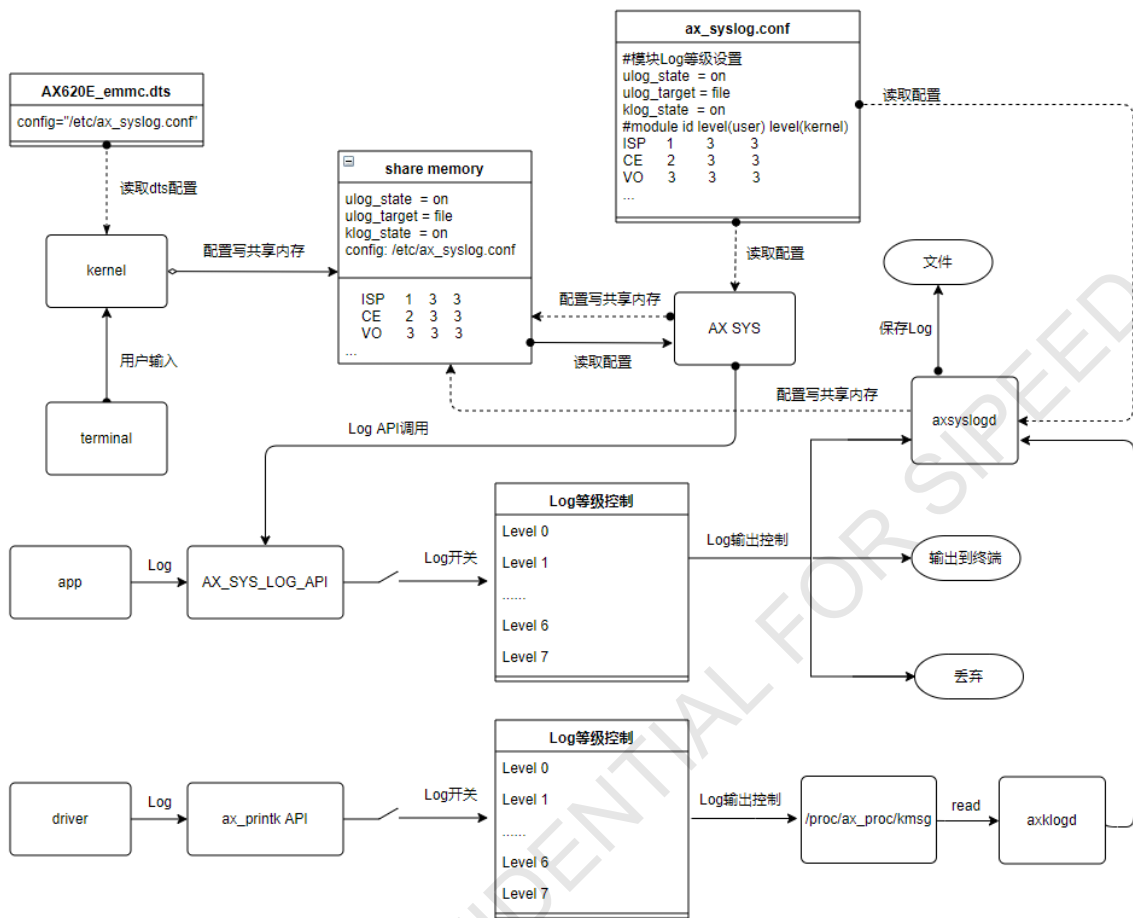


图 6-3

### 6.6.3 动态修改 user log 等级

我们的 log 系统支持按模块进行 log 等级修改，每个模块都有自己对应的 ID，比如 ISP、VO 和 NPU 他们对应的模块 ID 分别是 1、3、6。确定模块对应的 ID 值的方式有两种：

1. 通过 `cat /etc/ax_syslog.conf` 文件查看；
2. 通过 `cat /proc/ax_proc/logctl` 节点查看。

在 `ax_syslog.conf` 配置文件中，我们有一段这样的配置：

```
#模块 Log 等级设置
ulog_state = on
```

```
ulog_target = file
klog_state = on
#module id level(user) level(kernel)
ISP      1      4      4
CE       2      4      4
VO       3      4      4
VDSP     4      4      4
.....
```

以 ISP 模块为例，其 id 为 1，user log 等级为 4，这表示 ISP 模块 user log 等级为 0-4 的 Log 能够被输出，而等级为 5-7 的 Log 将被忽略。如果您需要修改模块的默认等级，可以在 ax\_syslog.conf 配置文件修改并保存。

为了方便对 user log 等级控制，我们提供了根据模块 id 来控制 log 等级的指令。指令格式如下：

```
echo ulog [id] [level] > /proc/ax_proc/logctl
```

其中 ulog 表示控制 user log 等级，id 为模块 ID，比如 ISP 的模块 ID 为 1，level 为控制 log 输出的等级，而 /proc/ax\_proc/logctl 为控制 log 等级的设备节点。为更好地展示如何控制 user log 的等级，我们举例说明。

示例 1：把 ISP 模块的 user log 输出等级设置为 7，可通过指令：

```
echo ulog 1 7 > /proc/ax_proc/logctl
```

来实现，修改后可通过：

```
cat /proc/ax_proc/logctl
```

查看设置是否生效：

| module | id | level(U) | level(K) |
|--------|----|----------|----------|
| ISP    | 1  | 7        | 4        |
| CE     | 2  | 4        | 4        |
| VO     | 3  | 4        | 4        |
| VDSP   | 4  | 4        | 4        |
| EFUSE  | 5  | 4        | 4        |
| NPU    | 6  | 4        | 4        |
| VENC   | 7  | 4        | 4        |
| VDEC   | 8  | 4        | 4        |
| JENC   | 9  | 4        | 4        |

图 6-4

user log 等级设置为 7 后，所有等级的 ISP user log 输出均可看到。

示例 2： 将所有模块的 user log 等级修改为 2

```
echo ulog all 2 > /proc/ax_proc/logctl
```

查看修改结果：

| module | id | level(U) | level(K) |
|--------|----|----------|----------|
| ISP    | 1  | 2        | 4        |
| CE     | 2  | 2        | 4        |
| VO     | 3  | 2        | 4        |
| VDSP   | 4  | 2        | 4        |
| EFUSE  | 5  | 2        | 4        |
| NPU    | 6  | 2        | 4        |
| VENC   | 7  | 2        | 4        |
| VDEC   | 8  | 2        | 4        |
| JENC   | 9  | 2        | 4        |
| JDEC   | 10 | 2        | 4        |
| SYS    | 11 | 2        | 4        |
| AENC   | 12 | 2        | 4        |
| IVPS   | 13 | 2        | 4        |
| MIPI   | 14 | 2        | 4        |
| ADEC   | 15 | 2        | 4        |
| DMA    | 16 | 2        | 4        |
| VIN    | 17 | 2        | 4        |
| USER   | 18 | 2        | 4        |
| IVES   | 19 | 2        | 4        |
| SKEL   | 20 | 2        | 4        |
| IVE    | 21 | 2        | 4        |
| 3A     | 25 | 2        | 4        |
| AI     | 32 | 2        | 4        |
| A0     | 33 | 2        | 4        |

图 6-5

示例 3： 关闭/打开 user log 输出

```
echo ulog off > /proc/ax_proc/logctl
```

```
-----  
klog state: on  
ulog state: off  
ulog_target: console  
-----
```

图 6-6

这种情况下，user log 将不再输出（不只是 log 不保存到文件，在应用程序中 log 的就已经不输出了）。

打开 user log 输出：

```
echo ulog on > /proc/ax_proc/logctl
```

示例 4：切换 user log 输出到 console 终端

```
echo ulog console > /proc/ax_proc/logctl
```

如果要切换输出到文件，可通过指令

```
echo ulog file> /proc/ax_proc/logctl
```

来实现。

## 6.6.4 动态修改 kernel log 等级

kernel log 等级的控制与 user log 等级的控制方式基本一致，只需将命令中的 ulog 改成 klog，最大的差异在于 user log 能够控制输出到 console 终端，但 kernel log 只能输出到日志文件。

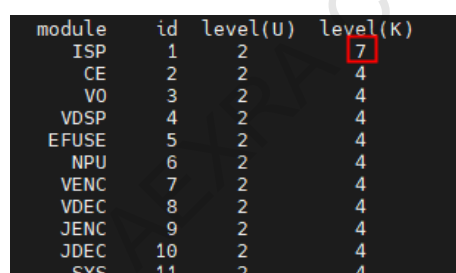
以下举例演示如何控制 kernel log 的输出。

示例 1：把 ISP 模块的 log 输出等级设置为 7，可通过指令：

```
echo klog 1 7 > /proc/ax_proc/logctl
```

来实现，修改后可通过：

```
cat /proc/ax_proc/logctl
```



| module | id | level(U) | level(K) |
|--------|----|----------|----------|
| ISP    | 1  | 2        | 7        |
| CE     | 2  | 2        | 4        |
| VO     | 3  | 2        | 4        |
| VDSP   | 4  | 2        | 4        |
| EFUSE  | 5  | 2        | 4        |
| NPU    | 6  | 2        | 4        |
| VENEC  | 7  | 2        | 4        |
| VDEC   | 8  | 2        | 4        |
| JENC   | 9  | 2        | 4        |
| JDEC   | 10 | 2        | 4        |
| SYS    | 11 | 2        | 4        |

图 6-7

示例 2：将所有模块的 kernel log 等级修改为 3

```
echo klog all 3 > /proc/ax_proc/logctl
```

查看修改结果：

| module | id | level(U) | level(K) |
|--------|----|----------|----------|
| ISP    | 1  | 2        | 3        |
| CE     | 2  | 2        | 3        |
| VO     | 3  | 2        | 3        |
| VDSP   | 4  | 2        | 3        |
| EFUSE  | 5  | 2        | 3        |
| NPU    | 6  | 2        | 3        |
| VENC   | 7  | 2        | 3        |
| VDEC   | 8  | 2        | 3        |
| JENC   | 9  | 2        | 3        |
| JDEC   | 10 | 2        | 3        |
| SYS    | 11 | 2        | 3        |
| AENC   | 12 | 2        | 3        |
| IVPS   | 13 | 2        | 3        |
| MIPI   | 14 | 2        | 3        |
| ADEC   | 15 | 2        | 3        |
| DMA    | 16 | 2        | 3        |
| VIN    | 17 | 2        | 3        |
| USER   | 18 | 2        | 3        |
| IVES   | 19 | 2        | 3        |
| SKEL   | 20 | 2        | 3        |
| IVE    | 21 | 2        | 3        |
| 3A     | 25 | 2        | 3        |
| AI     | 32 | 2        | 3        |
| A0     | 33 | 2        | 3        |
| SENSOR | 34 | 2        | 3        |

图 6-8

示例 3：关闭/打开 kernel log 输出

```
echo klog off > /proc/ax_proc/logctl
```

查看结果

```
-----  
klog_state: off  
ulog_state: on  
ulog_target: file  
-----
```

图 6-9

关闭 kernel log 后将不会有任何 kernel log 输出。

重新打开 kernel log 可以执行

```
echo klog on > /proc/ax_proc/logctl
```

# 7 SPI NAND NOR Flash 工程指南

## 7.1 SPI NOR Flash 工程构建

### 7.1.1 修改 uboot 的分区信息

修改 uboot 的配置文件 `include/configs/ax620e_common.h`, 即宏定义 `MTDPARTS_SPINOR` 关联的 `NOR_MTDPARTS`:

```
#define NOR_MTDPARTS "mtdparts=spi4.0:" FLASH_PARTITIONS
#define NAND_MTDPARTS "mtdparts=spi4.0:" FLASH_PARTITIONS

/* bootargs for eMMC */
#define BOOTARGS_EMMC DEBUG_BOOT_ARGS KERNEL_BOOTARGS
/* bootargs for SD */
#define BOOTARGS_SD OS_MEM_ARGS " console=ttyS0,115200n8 earlycon=uart8250,mmio32,0x4880000 init=/sbin/init root=/dev/mmcblk1p2 rw rootdelay=3 rootfstype=ext4"

#define BOOTARGS_SPINAND OS_MEM_ARGS " console=ttyS0,115200n8 earlycon=uart8250,mmio32,0x4880000 board_id=0,boot_reason=0x1,noinitrd\
root="ROOTFS_DEV" rw rootfstype="ROOTFS_TYPE" ubi.mtd=9,2048 root=ubi0:rootfs init=/linuxrc " NAND_MTDPARTS

/* bootargs for SPI NOR Flash */
#define MTDIDS_SPINOR "nor0=spi4.0"
#define MTDPARTS_SPINOR NOR_MTDPARTS
#define BOOTARGS_SPINOR KERNEL_BOOTARGS
/* bootargs for SPI NAND Flash */
#define MTDIDS_DEFAULT "nand0=spi4.0"
#define MTDPARTS_DEFAULT NAND_MTDPARTS
```

图 7-1

关联的 PARTITION size 定义在 `build/projects/AX620Q_nor_arm32_k419/project.mak` 中, 可根据实际的 SPI NOR Flash 的 size 进行修改 (当前为 16MB):

其中 `NORFLASH_SIZE := 16M` 定义了当前项目 NorFlash 的大小 (当前为 16MB, 如有变化需调整此定义值), `opt` 分区的大小 (`OPT_PARTITION_SIZE`) 由 NorFlash 及其他分区的大小调用函数 `AutoCalculateRest` 自动计算而来 (如分区有变化, 需同时调整函数 `AutoCalculateRest` 调用的传入参数)。

```
#partition size config
NORFLASH_SIZE           := 16M
SPL_PARTITION_SIZE      := 128K
UBOOT_PARTITION_SIZE    := 384K
ENV_PARTITION_SIZE      := 64K
DTB_PARTITION_SIZE      := 64K
ifeq ($(LIBC), uclibc)
KERNEL_PARTITION_SIZE    := 2432K
ROOTFS_PARTITION_SIZE    := 2112K
CUSTOMER_PARTITION_SIZE  := 1M
else
KERNEL_PARTITION_SIZE    := 2432K
ROOTFS_PARTITION_SIZE    := 3008K
CUSTOMER_PARTITION_SIZE  := 1M
endif
OPT_PARTITION_SIZE       := $(call AutoCalculateRest, \
$(NORFLASH_SIZE) \
$(SPL_PARTITION_SIZE) \
$(UBOOT_PARTITION_SIZE) \
$(ENV_PARTITION_SIZE) \
$(DTB_PARTITION_SIZE) \
$(KERNEL_PARTITION_SIZE) \
$(ROOTFS_PARTITION_SIZE) \
$(CUSTOMER_PARTITION_SIZE))
AUTO_FIT_PARTITION       := OPT
```

图 7-2

**！ 注意：**

spi4.0，其中 spi4 表示 device name，可以在设备树的配置中查询到；而.0 则表示 chip select 0，如果为.1，则表示 chip select 1。

**🔑 说明**

如果需要修改分区，只需要修改宏 MTDPARTS\_SPINOR 和宏 BOOTARGS\_SPINOR 关联的 NOR\_MTDPARTS 宏关于分区的信息即可。

## 7.1.2 配置 kernel 以支持 squashfs 文件系统

```
> File system > Miscellaneous filesystems
SquashFS 4.0 - Squashed file system support
```

## 7.1.3 生成 rootfs 镜像

```
mksquashfs rootfs rootfs.img -b 64K -comp xz
```

### 7.1.4 生成 spl 镜像

Romcode 启动阶段使用 Quad SPI 功能，需要查阅 SPI NOR Flash 的物料规格书 SR 寄存器 QE bit，如规格书定义如下 S9 为 QE bit，对应 RDSR 命令为 35h，WRSR 命令为 31h：

## 6. STATUS REGISTER

| S23      | S22  | S21  | S20      | S19      | S18 | S17      | S16      |
|----------|------|------|----------|----------|-----|----------|----------|
| HOLD/RST | DRV1 | DRV0 | Reserved | Reserved | LPE | Reserved | Reserved |

| S15  | S14 | S13 | S12 | S11 | S10  | S9 | S8   |
|------|-----|-----|-----|-----|------|----|------|
| SUS1 | CMP | LB3 | LB2 | LB1 | SUS2 | QE | SRP1 |

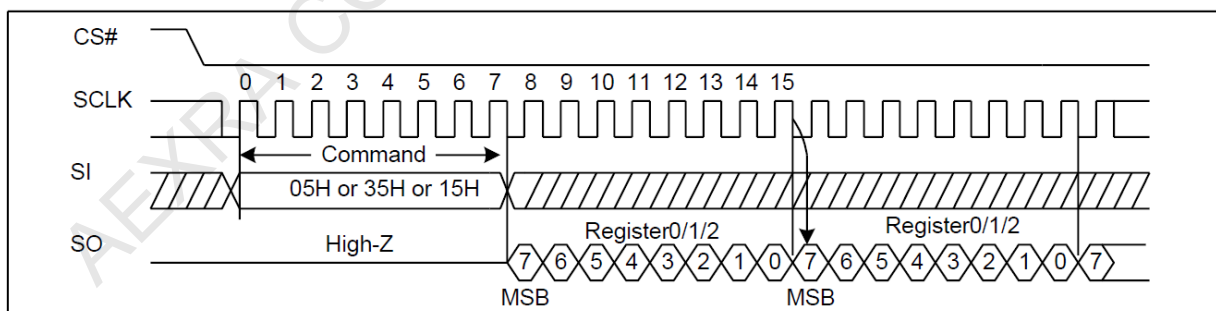
| S7   | S6  | S5  | S4  | S3  | S2  | S1  | S0  |
|------|-----|-----|-----|-----|-----|-----|-----|
| SRP0 | BP4 | BP3 | BP2 | BP1 | BP0 | WEL | WIP |

图 7-3

### 7.4. Read Status Register (RDSR) (05H or 35H or 15H)

The Read Status Register (RDSR) command is for reading the Status Register. The Status Register may be read at any time, even while a Program, Erase or Write Status Register cycle is in progress. When one of these cycles is in progress, it is recommended to check the Write in Progress (WIP) bit before sending a new command to the device. It is also possible to read the Status Register continuously. For command code "05H"/"35H"/"15H", the SO will output Status Register bits S7~S0/ S15-S8/ S23-S16.

Figure6. Read Status Register Sequence Diagram



### 7.5. Write Status Register (WRSR) (01H or 31H or 11H)

The Write Status Register (WRSR) command allows new values to be written to the Status Register. Before it can be accepted, a Write Enable (WREN) command must previously have been executed. After the Write Enable (WREN) command has been decoded and executed, the device sets the Write Enable Latch (WEL).

The Write Status Register (WRSR) command has no effect on S20, S19, S17, S16, S15, S10, S1 and S0 of the Status

图 7-4

需对应修改 spl 签名脚本: build/tools/imgsign/spl\_AX620E\_sign.py, 对应配置 spl img header 的 nand\_nor\_cfg 字段 bit8~15、bit16~23、bit31-24 分别为 RDSR 命令、WRSD 命令、QE bit 位:

```
struct img_header{
    u32 check_sum;//0x00
    u32 magic_data;//0x04, should be 0x55543322
    u32 capability;
    u32 img_size;
    u32 fw_size;
    u32 img_check_sum;
    u32 fw_check_sum;
    /*
     * [1:0]: nand page_size, page_size = 2^(nand_nor_cfg[1:0] + 11), 2KB ~
16KB
     * [4:2]: nand block_size, block_size = 2^(nand_nor_cfg[4:2] + 16), 64KB ~
8MB
     * [6:5]: nand oob_size, oob_size = 2^(nand_nor_cfg[6:5] + 6), 64Bytes ~
512Bytes
     * [7]: nand bad block marker size, if nand_nor_cfg[7] is 0, the bad block
marker
     * is 1 byte, if nand_type[7] is 1, bad block marker is 2 bytes.
     * [15: 8]: nor QE read status register cmd
     * [23:16]: nor QE write status register cmd
     * [31:24]: nor QE bit offset
     */
    u32 nand_nor_cfg;
...
};
...
hdr.sig_header = 0x01000800
if PKG_SIZE == 0x40000:
    hdr.nand_nor_cfg = 0x06010509
else:
    hdr.nand_nor_cfg = 0x01313504
hdr.ocm_start_addr = 0x03000400
```

## 7.1.5 双 spi-nor flash 支持

3. Build/projects/AX620Q\_nor\_arm32\_k419/pinmux/AX620Q\_DEMO\_pinmux.h 文件配置 CS1 的 pinmux 为 GPIO 功能控制,例如使用 GPIO0\_A4 管脚配置如下:

| File | File                                                                     |
|------|--------------------------------------------------------------------------|
| 1    | 1 0x0230000C, 0x00040003, /* PadName = VI_D0 Fuction = I2S0_DIN0 */      |
| 2    | 2 0x02300018, 0x00040003, /* PadName = VI_D1 Fuction = I2S0_SCLK */      |
| 3    | 3 0x02300024, 0x00040003, /* PadName = VI_D2 Fuction = I2S0_DOUT */      |
| 4    | 4 0x02300030, 0x00040003, /* PadName = VI_D3 Fuction = I2S0_MCLK */      |
| 5    | 5 0x0230003C, 0x00040003, /* PadName = VI_D4 Fuction = I2S0_DIN1 */      |
| 6    | 6 0x02300040, 0x00040003, /* PadName = VI_D4 Fuction = GPIO0_A4 */       |
| 7    | 7 0x02300044, 0x00040003, /* PadName = VI_CLK0 Fuction = I2S0_LRCK */    |
| 8    | 8 0x02300048, 0x00000003, /* PadName = I2C0_SCL Fuction = I2C0_SCL */    |
| 9    | 9 0x0230004C, 0x00000003, /* PadName = I2C0_SDA Fuction = I2C0_SDA */    |
| 10   | 10 0x02300050, 0x00000003, /* PadName = I2C1_SCL Fuction = I2C1_SCL */   |
| 11   | 11 0x02300054, 0x00000003, /* PadName = I2C1_SDA Fuction = I2C1_SDA */   |
| 12   | 12 0x02300058, 0x00000003, /* PadName = UART0_TXD Fuction = UART0_TXD */ |
| 13   | 13 0x0230005C, 0x00000003, /* PadName = UART0_RXD Fuction = UART0_RXD */ |
| 14   | 14 0x02300060, 0x00000003, /* PadName = UART1_TXD Fuction = UART1_TXD */ |
| 15   | 15 0x02300064, 0x00000003, /* PadName = UART1_RXD Fuction = UART1_RXD */ |

图 7-5

4. kernel/linux/linux-4.19.125/arch/arm/boot/dts/axera/AX620E.dtsi 将原 spi4 增加配置对应的 cs1 gpio 节点:

```

1214 1214 » » » resets = <flash_reset 22 0x14 1>,
1215 1215 » » » <flash_reset 21 0x14 1>;
1216 1216 » » » reset-names = "rst", "hrst";
1217 1217 » » » status = "disabled";
1218 1218 » » » };
1219 1219
1220 1220 » » spi4: spi@1A00000 {
1221 1221 » » compatible = "snps,dwc-ssi-1.03a";
1222 1222 » » reg = <0x0 0x1A00000 0x0 0x10000>;
1223 1223 » » interrupts = <GIC_SPI 8 IRQ_TYPE_LEVEL_HIGH>;
1224 1224 » » cs-gpios = <&ax_gpio2 29 0>; /*GPIO2-A29 for ahb_ssi's cs0 */
1225 1225 » » //cs-gpios = <&ax_gpio2 25 0>; /*GPIO2-A25 for ahb_ssi's cs1 */
1226 1226 » » cs-gpios = <&ax_gpio2 29 0>; /*GPIO2-A29 for ahb_ssi's cs0 */
1227 1227 » » <&ax_gpio0 4 0>; /*GPIO2-A25 for ahb_ssi's cs1 */
1228 1228 » » pinctrl-names = "default";
1229 1229 » » #address-cells = <1>;
1230 1230 » » #size-cells = <0>;
1231 1231 » » clocks = <&cpu_clk AX620X_CLK_H_SSI_EB>,
1232 1232 » » <&cpu_clk AX620X_HCLK_SPI_EB>;
1233 1233 » » clock-names = "flashclk", "pclk";
1234 1234 » » resets = <&cpu_reset 18 0x10 1>,
1235 1235 » » <&cpu_reset 17 0x10 1>;
1236 1236 » » reset-names = "rst", "prst";
1237 1237 » » dma = <&dma_per 1> <&dma_per 0>;

```

图 7-6

5. kernel/linux/linux-4.19.125/arch/arm/boot/dts/axera/AX620Q\_nor\_arm32\_k419.dts

增加 cs1 的 spi-nor 配置

```
0 » spi_flash@0 {
1 »     » #address-cells = <1>;
2 »     » #size-cells = <1>;
3 »     » compatible = "jedec,spi-nor";
4 »     » reg = <0>;
5 »     » rx-sample-delay-ns = <6>;
6 »     » spi-max-frequency = <104000000>;
7 »     » spi-tx-bus-width = <4>;
8 »     » spi-rx-bus-width = <4>;
9 » };
0 » spi_flash@1 {
1 »     » #address-cells = <1>;
2 »     » #size-cells = <1>;
3 »     » compatible = "jedec,spi-nor";
4 »     » reg = <1>;
5 »     » rx-sample-delay-ns = <6>;
6 »     » spi-max-frequency = <104000000>;
7 »     » spi-tx-bus-width = <4>;
8 »     » spi-rx-bus-width = <4>;
9 » };
0 » };
1 »
```

图 7-7

6. 如需 uboot 场景使用, 由于当前使用的原生 uboot-2020.4 版本不支持 mtd 下挂双相同设备, 可以参考如下 diff 修改, 通过 sf probe 1、sf read/write/erase 命令使用



uboot-spi.diff.zip

图 7-8

7. 如需 AXDL 烧录 axp 支持, 可参考如下 diff 修改



build-axdl.diff.zip



uboot-axdl.diff.zip

图 7-9

## 7.1.6 NFS 挂载

NOR Flash 工程受限于 Flash size, 对 linux kernel 做了部分功能的裁剪, 其中就包括了 NFS 功能。正常情况下 NFS 功能无法使用, 需要先运行 /soc/scripts/nfs\_start.sh 脚本后才可 mount nfs。

/soc/scripts/nfs\_start.sh 脚本完成的主要功能是加载 NFS 相关的 ko，包括：

grace.ko、lockd.ko、nfs.ko、nfsv2.ko、nfsv3.ko 和 sunrpc.ko。

这些文件存放在/soc/ko 目录。

## 7.1.7 SPI NOR Flash SecondSource 物料支持

1.根据物料规格书修改 uboot 的 drivers\mtd\spi\spi-nor-ids.c 和 kernel 的 drivers\mtd\spi-nor\spi-nor.c 文件：

```
chenchi@axcnsbussrv06p:~/mc20e/2ndsource/ax620e_trunk/boot/uboot/u-boot-2020.04/drivers/mtd/spi$ ls -l
total 184
-rw-rw-r-- 1 chenchi chenchi 2303 Feb 28 11:29 fsl_espi_spl.c
-rw-rw-r-- 1 chenchi chenchi 6767 Feb 28 11:29 Kconfig
-rw-rw-r-- 1 chenchi chenchi 591 Feb 28 11:29 Makefile
-rw-rw-r-- 1 chenchi chenchi 15382 Feb 28 11:29 sandbox.c
-rw-rw-r-- 1 chenchi chenchi 19031 Feb 28 11:29 sf_dataflash.c
-rw-rw-r-- 1 chenchi chenchi 2640 Feb 28 11:29 sf_internal.h
-rw-rw-r-- 1 chenchi chenchi 3089 Feb 28 11:29 sf_mtd.c
-rw-rw-r-- 1 chenchi chenchi 4155 Feb 28 11:29 sf_probe.c
-rw-rw-r-- 1 chenchi chenchi 2510 Feb 28 11:29 sf_uclass.c
-rw-rw-r-- 1 chenchi chenchi 69769 Feb 28 11:29 spi-nor-core.c
-rw-rw-r-- 1 chenchi chenchi 16815 Feb 28 11:29 spi-nor-ids.c
-rw-rw-r-- 1 chenchi chenchi 19481 Feb 28 11:29 spi-nor-tiny.c
```

```
chenchi@axcnsbussrv06p:~/mc20e/2ndsource/ax620e_trunk/kernel/linux/linux-4.19.125/drivers/mtd/spi-nor$ ls -l
total 324
-rw-rw-r-- 1 chenchi chenchi 25557 Mar 5 20:45 aspeed-smc.c
-rw-rw-r-- 1 chenchi chenchi 21093 Mar 5 20:45 atmel-quadspi.c
-rw-rw-r-- 1 chenchi chenchi 39098 Mar 5 20:45 cadence-quadspi.c
-rw-rw-r-- 1 chenchi chenchi 31227 Mar 5 20:45 fsl-quadspi.c
-rw-rw-r-- 1 chenchi chenchi 12825 Mar 5 20:45 hisi-sfc.c
-rw-rw-r-- 1 chenchi chenchi 23397 Mar 5 20:45 intel-spi.c
-rw-rw-r-- 1 chenchi chenchi 649 Mar 5 20:45 intel-spi.h
-rw-rw-r-- 1 chenchi chenchi 2228 Mar 5 20:45 intel-spi-pci.c
-rw-rw-r-- 1 chenchi chenchi 1458 Mar 5 20:45 intel-spi-platform.c
-rw-rw-r-- 1 chenchi chenchi 4652 Mar 5 20:45 Kconfig
-rw-rw-r-- 1 chenchi chenchi 611 Mar 5 20:45 Makefile
-rw-rw-r-- 1 chenchi chenchi 14163 Mar 5 20:45 mtk-quadspi.c
-rw-rw-r-- 1 chenchi chenchi 11694 Mar 5 20:45 nxp-spifi.c
-rw-rw-r-- 1 chenchi chenchi 88610 Mar 5 20:45 spi-nor.c
-rw-rw-r-- 1 chenchi chenchi 17899 Mar 5 20:45 stm32-quadspi.c
```

2. 增加对应 device id 物料配置:一般按字段定义配置 Name、Manufacture ID、Device ID、容量、QUAD 模式即可：

82695: [AX620E-7476] UBOOT: Fudan Microelectronics FM25Q128A spinor support — [u-boot-2020.04/drivers/mtd/spi/spi-nor-ids.c](#)

Base = [gitweb](#) → Patchset 1 = [gitweb](#) | [DOWNLOAD](#)

```

@@
321 #ifndef CONFIG_SPI_FLASH_XMC
322 * /* XMC (Wuhan Xinxin Semiconductor Manufacturing Corp.) */
323 * { INFO("XM25QH64A", 0x207017, 0, 64 * 1024, 128, SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ) },
324 * { INFO("XM25QH128A", 0x207018, 0, 64 * 1024, 256, SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ) },
325 #endif
326 #ifndef CONFIG_SPI_FLASH_ZBIT
327 * /* ZBIT (Zbit Semiconductor Manufacturing Corp.) */
328 * { INFO("ZB25VQ64", 0x5e4017, 0, 64 * 1024, 128, SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ) },
329 * { INFO("ZB25VQ128", 0x5e4018, 0, 64 * 1024, 256, SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ) },
330 #endif
331 #endif
332 * { },
333 };
334
335 #endif
336 #endif
337
338 #ifndef CONFIG_SPI_FLASH_FMSH
339 * /* FMSH (Shanghai Fudan Microelectronics Corp.) */
340 * { INFO("FM25Q128A", 0x14018, 0, 64 * 1024, 256, SECT_4K | SPI_NOR_DUAL_READ | SPI_NOR_QUAD_READ) },
341 #endif
342 #endif
343
344 #endif
345 #endif
346 #endif
347
348 #endif
349 #endif
350 #endif
351 #endif
352 #endif
353 #endif
354 #endif
355 #endif
356 #endif
357 #endif
358 #endif
359 #endif
360 #endif
361 #endif
362 #endif
363 #endif
364 #endif
365 #endif
366 #endif
367 #endif
368 #endif
369 #endif
370 #endif
371 #endif
372 #endif
373 #endif
374 #endif
375 #endif
376 #endif
377 #endif
378 #endif
379 #endif
380 #endif
381 #endif
382 #endif
383 #endif
384 #endif
385 #endif
386 #endif
387 #endif
388 #endif
389 #endif
390 #endif
391 #endif
392 #endif
393 #endif
394 #endif
395 #endif
396 #endif
397 #endif
398 #endif
399 #endif
400 #endif
401 #endif
402 #endif
403 #endif
404 #endif
405 #endif
406 #endif
407 #endif
408 #endif
409 #endif
410 #endif
411 #endif
412 #endif
413 #endif
414 #endif
415 #endif
416 #endif
417 #endif
418 #endif
419 #endif
420 #endif
421 #endif
422 #endif
423 #endif
424 #endif
425 #endif
426 #endif
427 #endif
428 #endif
429 #endif
430 #endif
431 #endif
432 #endif
433 #endif
434 #endif
435 #endif
436 #endif
437 #endif
438 #endif
439 #endif
440 #endif
441 #endif
442 #endif
443 #endif
444 #endif
445 #endif
446 #endif
447 #endif
448 #endif
449 #endif
450 #endif
451 #endif
452 #endif
453 #endif
454 #endif
455 #endif
456 #endif
457 #endif
458 #endif
459 #endif
460 #endif
461 #endif
462 #endif
463 #endif
464 #endif
465 #endif
466 #endif
467 #endif
468 #endif
469 #endif
470 #endif
471 #endif
472 #endif
473 #endif
474 #endif
475 #endif
476 #endif
477 #endif
478 #endif
479 #endif
480 #endif
481 #endif
482 #endif
483 #endif
484 #endif
485 #endif
486 #endif
487 #endif
488 #endif
489 #endif
490 #endif
491 #endif
492 #endif
493 #endif
494 #endif
495 #endif
496 #endif
497 #endif
498 #endif
499 #endif
500 #endif
501 #endif
502 #endif
503 #endif
504 #endif
505 #endif
506 #endif
507 #endif
508 #endif
509 #endif
510 #endif
511 #endif
512 #endif
513 #endif
514 #endif
515 #endif
516 #endif
517 #endif
518 #endif
519 #endif
520 #endif
521 #endif
522 #endif
523 #endif
524 #endif
525 #endif
526 #endif
527 #endif
528 #endif
529 #endif
530 #endif
531 #endif
532 #endif
533 #endif
534 #endif
535 #endif
536 #endif
537 #endif
538 #endif
539 #endif
540 #endif
541 #endif
542 #endif
543 #endif
544 #endif
545 #endif
546 #endif
547 #endif
548 #endif
549 #endif
550 #endif
551 #endif
552 #endif
553 #endif
554 #endif
555 #endif
556 #endif
557 #endif
558 #endif
559 #endif
560 #endif
561 #endif
562 #endif
563 #endif
564 #endif
565 #endif
566 #endif
567 #endif
568 #endif
569 #endif
570 #endif
571 #endif
572 #endif
573 #endif
574 #endif
575 #endif
576 #endif
577 #endif
578 #endif
579 #endif
580 #endif
581 #endif
582 #endif
583 #endif
584 #endif
585 #endif
586 #endif
587 #endif
588 #endif
589 #endif
590 #endif
591 #endif
592 #endif
593 #endif
594 #endif
595 #endif
596 #endif
597 #endif
598 #endif
599 #endif
600 #endif
601 #endif
602 #endif
603 #endif
604 #endif
605 #endif
606 #endif
607 #endif
608 #endif
609 #endif
610 #endif
611 #endif
612 #endif
613 #endif
614 #endif
615 #endif
616 #endif
617 #endif
618 #endif
619 #endif
620 #endif
621 #endif
622 #endif
623 #endif
624 #endif
625 #endif
626 #endif
627 #endif
628 #endif
629 #endif
630 #endif
631 #endif
632 #endif
633 #endif
634 #endif
635 #endif
636 #endif
637 #endif
638 #endif
639 #endif
640 #endif
641 #endif
642 #endif
643 #endif
644 #endif
645 #endif
646 #endif
647 #endif
648 #endif
649 #endif
650 #endif
651 #endif
652 #endif
653 #endif
654 #endif
655 #endif
656 #endif
657 #endif
658 #endif
659 #endif
660 #endif
661 #endif
662 #endif
663 #endif
664 #endif
665 #endif
666 #endif
667 #endif
668 #endif
669 #endif
670 #endif
671 #endif
672 #endif
673 #endif
674 #endif
675 #endif
676 #endif
677 #endif
678 #endif
679 #endif
680 #endif
681 #endif
682 #endif
683 #endif
684 #endif
685 #endif
686 #endif
687 #endif
688 #endif
689 #endif
690 #endif
691 #endif
692 #endif
693 #endif
694 #endif
695 #endif
696 #endif
697 #endif
698 #endif
699 #endif
700 #endif
701 #endif
702 #endif
703 #endif
704 #endif
705 #endif
706 #endif
707 #endif
708 #endif
709 #endif
710 #endif
711 #endif
712 #endif
713 #endif
714 #endif
715 #endif
716 #endif
717 #endif
718 #endif
719 #endif
720 #endif
721 #endif
722 #endif
723 #endif
724 #endif
725 #endif
726 #endif
727 #endif
728 #endif
729 #endif
730 #endif
731 #endif
732 #endif
733 #endif
734 #endif
735 #endif
736 #endif
737 #endif
738 #endif
739 #endif
740 #endif
741 #endif
742 #endif
743 #endif
744 #endif
745 #endif
746 #endif
747 #endif
748 #endif
749 #endif
750 #endif
751 #endif
752 #endif
753 #endif
754 #endif
755 #endif
756 #endif
757 #endif
758 #endif
759 #endif
760 #endif
761 #endif
762 #endif
763 #endif
764 #endif
765 #endif
766 #endif
767 #endif
768 #endif
769 #endif
770 #endif
771 #endif
772 #endif
773 #endif
774 #endif
775 #endif
776 #endif
777 #endif
778 #endif
779 #endif
780 #endif
781 #endif
782 #endif
783 #endif
784 #endif
785 #endif
786 #endif
787 #endif
788 #endif
789 #endif
790 #endif
791 #endif
792 #endif
793 #endif
794 #endif
795 #endif
796 #endif
797 #endif
798 #endif
799 #endif
800 #endif
801 #endif
802 #endif
803 #endif
804 #endif
805 #endif
806 #endif
807 #endif
808 #endif
809 #endif
810 #endif
811 #endif
812 #endif
813 #endif
814 #endif
815 #endif
816 #endif
817 #endif
818 #endif
819 #endif
820 #endif
821 #endif
822 #endif
823 #endif
824 #endif
825 #endif
826 #endif
827 #endif
828 #endif
829 #endif
830 #endif
831 #endif
832 #endif
833 #endif
834 #endif
835 #endif
836 #endif
837 #endif
838 #endif
839 #endif
840 #endif
841 #endif
842 #endif
843 #endif
844 #endif
845 #endif
846 #endif
847 #endif
848 #endif
849 #endif
850 #endif
851 #endif
852 #endif
853 #endif
854 #endif
855 #endif
856 #endif
857 #endif
858 #endif
859 #endif
860 #endif
861 #endif
862 #endif
863 #endif
864 #endif
865 #endif
866 #endif
867 #endif
868 #endif
869 #endif
870 #endif
871 #endif
872 #endif
873 #endif
874 #endif
875 #endif
876 #endif
877 #endif
878 #endif
879 #endif
880 #endif
881 #endif
882 #endif
883 #endif
884 #endif
885 #endif
886 #endif
887 #endif
888 #endif
889 #endif
890 #endif
891 #endif
892 #endif
893 #endif
894 #endif
895 #endif
896 #endif
897 #endif
898 #endif
899 #endif
900 #endif
901 #endif
902 #endif
903 #endif
904 #endif
905 #endif
906 #endif
907 #endif
908 #endif
909 #endif
910 #endif
911 #endif
912 #endif
913 #endif
914 #endif
915 #endif
916 #endif
917 #endif
918 #endif
919 #endif
920 #endif
921 #endif
922 #endif
923 #endif
924 #endif
925 #endif
926 #endif
927 #endif
928 #endif
929 #endif
930 #endif
931 #endif
932 #endif
933 #endif
934 #endif
935 #endif
936 #endif
937 #endif
938 #endif
939 #endif
940 #endif
941 #endif
942 #endif
943 #endif
944 #endif
945 #endif
946 #endif
947 #endif
948 #endif
949 #endif
950 #endif
951 #endif
952 #endif
953 #endif
954 #endif
955 #endif
956 #endif
957 #endif
958 #endif
959 #endif
960 #endif
961 #endif
962 #endif
963 #endif
964 #endif
965 #endif
966 #endif
967 #endif
968 #endif
969 #endif
970 #endif
971 #endif
972 #endif
973 #endif
974 #endif
975 #endif
976 #endif
977 #endif
978 #endif
979 #endif
980 #endif
981 #endif
982 #endif
983 #endif
984 #endif
985 #endif
986 #endif
987 #endif
988 #endif
989 #endif
990 #endif
991 #endif
992 #endif
993 #endif
994 #endif
995 #endif
996 #endif
997 #endif
998 #endif
999 #endif
1000 #endif

```

四线模式支持情况：仅支持 data 四线，未支持 cmd/addr 四线模式

特别说明：以上软件适配只是 bringup 简单调通，硬件可靠性需要高低温压测，可以咨询我司 HW 研发 AVL 物料验证支持流程；如遇特殊物料无法支持，也需走 AVL 验证支持流程由我司研发资源调试。

## 7.2 SPI NAND Flash 工程创建

### 7.2.1 检查 AX620Q\_nand\_arm32\_k419\_defconfig 文件

确保以下两个宏：

- CONFIG\_CMD\_NAND=y
- CONFIG\_ENV\_IS\_IN\_NAND=y

### 7.2.2 修改 uboot 的分区信息

修改 uboot 的配置文件 include/configs/ax620e\_common.h，即宏定义 MTDPARTS\_DEFAULT 关联的 NAND\_MTDPARTS：

```

#define NOR_MTDPARTS "mtdparts=spi4.0:" FLASH_PARTITIONS
#define NAND_MTDPARTS "mtdparts=spi4.0:" FLASH_PARTITIONS

/* bootargs for eMMC */
#define BOOTARGS_EMMC DEBUG_BOOT_ARGS KERNEL_BOOTARGS
/* bootargs for SD */
#define BOOTARGS_SD OS_MEM_ARGS " console=ttyS0,115200n8 earlycon=uart8250,mmio32,0x4880000 init=/sbin/init root=/dev/mmcblk1p2 rw ro

#define BOOTARGS_SPINAND OS_MEM_ARGS " console=ttyS0,115200n8 earlycon=uart8250,mmio32,0x4880000 board_id=0,boot_reason=0x1,noinitrd\
root="ROOTFS_DEV" rw rootfstype="ROOTFS_TYPE" ubi.mtd=9,2048 root=ubi0:rootfs init=/linuxrc " NAND_MTDPARTS

/* bootargs for SPI NOR Flash */
#define MTDIDS_SPINOR "nor=spi4.0"
#define MTDPARTS_SPINOR NOR_MTDPARTS
#define BOOTARGS_SPINOR KERNEL_BOOTARGS

/* bootargs for SPI NAND Flash */
#define MTDIDS_DEFAULT "nand=spi4.0"
#define MTDPARTS_DEFAULT NAND_MTDPARTS

```

图 7-10

关联的 PARTITION size 定义在 build/projects/AX620Q\_nand\_arm32\_k419/project.mak 中，可根据实际的 SPI NAND Flash 的 size 进行修改（当前为 128MB）：

```

#partition size config
SPL_PARTITION_SIZE      := 1M
DDRINIT_PARTITION_SIZE  := 512K
UBOOT_PARTITION_SIZE    := 1280K
ENV_PARTITION_SIZE      := 512K
PARAM_PARTITION_SIZE    := 4M
DTB_PARTITION_SIZE      := 512K
KERNEL_PARTITION_SIZE   := 8M
ROOTFS_PARTITION_SIZE   := 112M
AUTO_FIT_PARTITION      := ROOTFS

```

图 7-11

#### 注意：

spi4.0，其中 spi4 表示 device name，可以在设备树的配置中查询到；而.0 则表示 chip select 0，如果为.1，则表示 chip select 1。

#### 说明：

如果需要修改分区，只需要修改宏 MTDPARTS\_DEFAULT 和宏 CONFIG\_BOOTARGS 关于分区的信息即可。

修改之后，编译 uboot，重新烧录新的 uboot 的带 signed 的 bin 文件，启动至 uboot 命令行界面。如下图所示：

```
AXERA-UBOOT=>
AXERA-UBOOT=>mtddparts

device nand0 <spi4.0>, # parts = 6
#: name      size      net size      offset      mask_flags
0: spi       0x00080000    0x00080000    0x00000000    0
1: uboot     0x00100000    0x00100000    0x00080000    0
2: kernel-dtb 0x00100000    0x00100000    0x00180000    0
3: kernel    0x02000000    0x02000000    0x00280000    0
4: param     0x00080000    0x00080000    0x02280000    0
5: rootfs    0x0c000000    0x0c000000    0x02300000    0

active partition: nand0,0 - (spi) 0x00080000 @ 0x00000000

defaults:
mtdids : nand0=spi4.0
mtdparts: mtdparts=spi4.0:512K(spi),1M(uboot),1M(kernel-dtb),32M(kernel),512K(param),192M(rootfs)
AXERA-UBOOT=>
```

图 7-12 启动至 uboot 命令行界面

可以看到分区信息已经生效。

## 7.2.3 生成 ubi rootfs

需要在 uboot 和 kernel 中分别使能对 UBIFS 的支持。

### 配置 uboot

执行 `make menuconfig` 命令以使能如下宏定义：

```
CONFIG_CMD_NAND
CONFIG_CMD_UBI
CONFIG_CMD_UBIFS
CONFIG_RBTREE
CONFIG_MTD_DEVICE
CONFIG_MTD_PARTITIONS
CONFIG_CMD_MTDPARTS
CONFIG_LZO
```

设置启动信息：

```
#setenv bootargs 'console=tty0,115200n8 noinitrd ip=off mem=256M
rootwait=1 rw ubi.mtd=6, 2048 rootfstype=ubifs root=ubi0:rootfs
init=/init'
```

## 配置 kernel

使用 `make menuconfig` 命令进行如下配置：

```
Enabling UBI support on MTD devices.
    Device Drivers --->
        Memory Technology Device (MTD) support --->
Enable UBI - Unsorted block images --->
    Enabling UBIFS file-system support.
        File systems --->
            Miscellaneous filesystems --->
                UBIFS file system support
```

### 7.2.4 制作 UBIFS

制作 UBIFS 需要两个工具：`mkfs.ubifs` 和 `ubinize`。

`mkfs.ubifs`：用于制作 UBIFS image。

`ubinize`：根据 UBIFS image 制作 `ubi.img`，`ubi.img` 是通过 `uboot` 直接烧写到 NAND Flash 分区上的。

Demo 板上的 spi nand flash 的型号是 xxx，page size 2048 bytes，block size 为  $64 * 2048 = 131072$  bytes，block count 为 1024。制作针对 demo 板的 UBIFS，执行如下两条命令：

#### 命令一：

```
mkfs.ubifs -F -q -r /home/usr/fs -m 2048 -e 126976 -c 2047 -o ubifs.img
```

参数说明：

-F：使能"white-space-fixup"，如果是通过 u-boot 烧写需要使能此功能。

-r：待制作的文件系统目录。

-m：最小输入输出大小，一般为 page\_size，当前 spinand 的 page\_size 为 2048byte。

-e：LEB size 逻辑可擦除块大小， $block\_size - 2 * (page\_size) = 128K - 4K = 126976$ 。

-c: 文件系统所占用的最大 block 数，一般小于等于 (block\_count-1)。

-o: 输出的 ubifs.img 文件。

## 命令二:

```
ubinize -o ubi.img -m 2048 -p 128KiB ubinize.cfg
```

参数说明:

-p: block size

-m: SPI NAND FLASH 的最小读写单元，一般为 page size

-o: 输出的 ubi.img 文件

ubinize.cfg: 为 ubinize 所需的配置文件

内容如下:

```
[ubifs]
mode=ubi
image=ubifs.img
vol_id=0
vol_size=100MiB
vol_type=dynamic
vol_alignment=1
vol_name=rootfs
```

## 7.2.5 SPI NAND Flash SecondSource 物料支持

1. 根据物料规格书修改 uboot 和 kernel 的 drivers\mtd\nand\spi\厂商名.c 文件

```
chenchi@axcnsbussrv03p:~/ax620_trunk/boot/uboot/u-boot-2020.04/drivers/mtd/nand/spi$ ls -l
total 68
-rw-rw-r-- 1 chenchi chenchi 28876 Mar  8 10:57 core.c
-rw-rw-r-- 1 chenchi chenchi  4180 Mar  8 10:57 gigadevice.c
-rw-rw-r-- 1 chenchi chenchi   187 Mar  8 10:57 Kconfig
-rw-rw-r-- 1 chenchi chenchi  4122 Mar  8 10:57 macronix.c
-rw-rw-r-- 1 chenchi chenchi   142 Mar  8 10:57 Makefile
-rw-rw-r-- 1 chenchi chenchi  3301 Mar  8 10:57 micron.c
-rw-rw-r-- 1 chenchi chenchi  5774 Mar  8 10:57 winbond.c
```

```
chenchi@axcnshbussrv03p:~/ax620_trunk/kernel/linux/linux-4.19.125/drivers/mtd/nand/spi$ ls -l
total 56
-rw-rw-r-- 1 chenchi chenchi 27293 Mar  8 10:58 core.c
-rw-rw-r-- 1 chenchi chenchi 3915 Mar  8 10:58 gigadevice.c
-rw-rw-r-- 1 chenchi chenchi 185 Mar  8 10:58 Kconfig
-rw-rw-r-- 1 chenchi chenchi 3859 Mar  8 10:58 macronix.c
-rw-rw-r-- 1 chenchi chenchi 141 Mar  8 10:58 Makefile
-rw-rw-r-- 1 chenchi chenchi 3255 Mar  8 10:58 micron.c
-rw-rw-r-- 1 chenchi chenchi 4894 Mar  8 10:58 winbond.c
```

## 2. 关闭 SPINAND\_PAGE\_READ\_FROM\_CACHE\_QUADIO\_OP 定义

四线模式支持情况：仅支持 data 四线，未支持 cmd/addr 四线模式：

```
static SPINAND_OP_VARIANTS(read_cache_variants,
>  * SPINAND_PAGE_READ_FROM_CACHE_QUADIO_OP(0, 2, NULL, 0),
>  * SPINAND_PAGE_READ_FROM_CACHE_X4_OP(0, 1, NULL, 0),
>  * SPINAND_PAGE_READ_FROM_CACHE_DUALTO_OP(0, 1, NULL, 0),
>  * SPINAND_PAGE_READ_FROM_CACHE_X2_OP(0, 1, NULL, 0),
>  * SPINAND_PAGE_READ_FROM_CACHE_OP(true, 0, 1, NULL, 0),
>  * SPINAND_PAGE_READ_FROM_CACHE_OP(false, 0, 1, NULL, 0));

22 static SPINAND_OP_VARIANTS(read_cache_variants,
23  * //SPINAND_PAGE_READ_FROM_CACHE_QUADIO_OP(0, 2, NULL, 0),
24  * SPINAND_PAGE_READ_FROM_CACHE_X4_OP(0, 1, NULL, 0),
25  * SPINAND_PAGE_READ_FROM_CACHE_DUALTO_OP(0, 1, NULL, 0),
26  * SPINAND_PAGE_READ_FROM_CACHE_X2_OP(0, 1, NULL, 0),
27  * SPINAND_PAGE_READ_FROM_CACHE_OP(true, 0, 1, NULL, 0),
28  * SPINAND_PAGE_READ_FROM_CACHE_OP(false, 0, 1, NULL, 0));
29
```

## 3. 增加对应 device id 物料配置

e — ☒ u-boot-2020.04/drivers/mtd/nand/spi/gigadevice.c

File 2

[SHOW BLAME](#) | [EDI](#)

```
g_ecc_get_status)),
tage:1.8v */
i, 2048, 1, 1, 1),

ad_cache_variants,
ite_cache_variants,
date_cache_variants),

:yg_ooblayout,
g_ecc_get_status)),

119  * gd5fxgq4xexxg_ecc_get_status)),
120  * SPINAND_INFO("GD5F2GQ5RExxH", 0x42, /* voltage:1.8v */
121  * NAND_MEMORG(1, 2048, 128, 64, 2048, 1, 1, 1),
122  * NAND_ECCREQ(8, 512),
123  * SPINAND_INFO_OP_VARIANTS(&read_cache_variants,
124  * &write_cache_variants,
125  * &update_cache_variants),
126  * 0,
127  * SPINAND_ECCINFO(&gd5fxgq4xexxg_ooblayout,
128  * gd5fxgq4xexxg_ecc_get_status)),
129  * SPINAND_INFO("GD5F2GM7xExxG", 0x82, /* voltage:1.8v */
130  * NAND_MEMORG(1, 2048, 64, 64, 2048, 1, 1, 1),
131  * NAND_ECCREQ(8, 512),
132  * SPINAND_INFO_OP_VARIANTS(&read_cache_variants,
133  * &write_cache_variants,
134  * &update_cache_variants),
135  * SPINAND_HAS_QE_BIT,
136  * SPINAND_ECCINFO(&gd5fxgq4xexxg_ooblayout,
137  * gd5fxgq4xexxg_ecc_get_status)),
```

linux-4.19.125/drivers/mtd/nand/spi/gigadevice.c

File 2

[SHOW BLAME](#) | [ED](#)

```

108 »         return -EINVAL;
109 }
110
ble[] = {
111 static const struct spinand_info gigadevice_spinand_table[] = {
112     SPINAND_INFO("GD5F2GQ5RExxH",
113     »         0x42,
114     »         NAND_MEMORG(1, 2048, 128, 64, 2048, 1, 1, 1),
115     »         NAND_ECCREQ(8, 512),
116     »         SPINAND_INFO_OP_VARIANTS(&read_cache_variants,
117     »         »         »         &write_cache_variants,
118     »         »         »         &update_cache_variants),
119     »         0,
120     »         SPINAND_ECCINFO(&gd5fxgq4_variant2_ooblayout,
121     »         »         gd5fxgq4uexxg_ecc_get_status)),
122     SPINAND_INFO("GD5F2GM7xExxG",
123     »         0x82,
124     »         NAND_MEMORG(1, 2048, 64, 64, 2048, 1, 1, 1),
125     »         NAND_ECCREQ(8, 512),
126     »         SPINAND_INFO_OP_VARIANTS(&read_cache_variants,
127     »         »         »         &write_cache_variants,
128     »         »         »         &update_cache_variants),
129     »         SPINAND_HAS_QE_BIT,
130     »         SPINAND_ECCINFO(&gd5fxgq4_variant2_ooblayout,
131     »         »         gd5fxgq4uexxg_ecc_get_status)),

```

#### 4. 字段说明:

##### 1) Manufacture ID:

在物料规格书中可查找,

```

const struct spinand_manufacturer gigadevice_spinand_manufacturer = {
    .id = SPINAND_MFR_GIGADEVICE,
    .name = "GigaDevice",
    .ops = &gigadevice_spinand_manuf_ops,
};

```

Manufacture ID

```

7: #include <linux/device.h>
8: #include <linux/kernel.h>
9: #include <linux/mtd/spinand.h>
10:
11: #define SPINAND_MFR_GIGADEVICE 0xC8

```

##### 2) Device ID:

在物料规格书中可查找,

```

static const struct spinand_info gigadevice_spinand_table[] = {
    SPINAND_INFO("GD5F2GQ5RExxH", 型号
Device ID 0x42,
    NAND_MEMORG(1, 2048, 128, 64, 2048, 1, 1, 1),
    NAND_ECCREQ(8, 512),
    SPINAND_INFO_OP_VARIANTS(&read_cache_variants,
                             &write_cache_variants,
                             &update_cache_variants),
    0,
    SPINAND_ECCINFO(&gd5fxgq4_variant2_ooblayout,
                    gd5fxgq4uexxg_ecc_get_status)),
};

```

### 3) NAND\_MEMORG:

```

NAND_MEMORG(1, 2048, 128, 64, 2048, 1, 1, 1),|

#define NAND_MEMORG(bpc, ps, os, ppe, epl, ppl, lpt, nt) \
{
    .bits_per_cell = (bpc),
    .pagesize = (ps),
    .oobsize = (os),
    .pages_per_eraseblock = (ppe),
    .eraseblocks_per_lun = (epl),
    .planes_per_lun = (ppl),
    .luns_per_target = (lpt),
    .ntargets = (nt),
}

```

bits\_per\_cell: 填 1, NAND 均为 SLC, 1 cell 存 1 bit

pagesize、oobsize: 根据规格书填写, 例如

#### ◆ Page Size

- Internal ECC On (ECC\_EN=1, default):

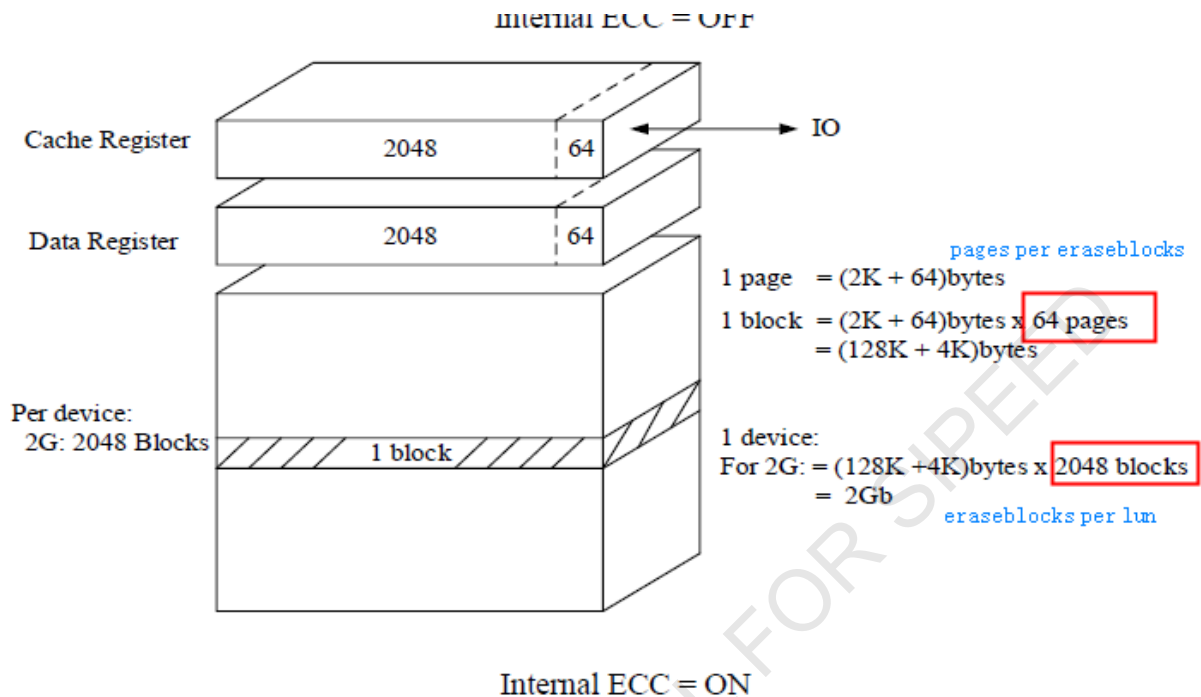
Page Size: 2048-Byte+64-Byte OOB Size

- Internal ECC Off (ECC\_EN=0):

Page Size: 2048-Byte+128-Byte

pages\_per\_eraseblock: 一般为 64

eraseblocks\_per\_lun、planes\_per\_lun、luns\_per\_target: 根据规格书填写



- 4) SPINAND\_INFO\_OP\_VARIANTS: JEDEC 标准一致，同主 source 移植即可
- 5) SPINAND\_HAS\_QE\_BIT: 根据规格书确认是否支持四线模式，大多数物料都支持
- 6) SPINAND\_ECCINFO: ECC 纠错接口每个厂商 default 的接口一般通用（例如 gigadevice 的 gd5fxgq4uexxg\_ecc\_get\_status 接口），但一定要根据规格书确认是否可以直接使用还是另外开发接口

```
static int gd5fxgq4uexxg_ecc_get_status(struct spinand_device *spinand,
                                         u8 status)
{
    u8 status2;
    struct spi_mem_op op = SPINAND_GET_FEATURE_OP(GD5FXGQ4UEXXG_REG_STATUS2,
                                                    &status2);
    int ret;

    switch (status & STATUS_ECC_MASK) {
    case STATUS_ECC_NO_BITFLIPS:
        return 0;
    }

    #define SPINAND_MFR_GIGADEVICE 0xC8
    #define GD5FXGQ4XA_STATUS_ECC_1_7_BITFLIPS (1 << 4)
    #define GD5FXGQ4XA_STATUS_ECC_8_BITFLIPS (3 << 4)
    #define GD5FXGQ4UEXXG_REG_STATUS2 0xF0
}
```

```

28:
29: /* status register */
30: #define REG_STATUS      0xc0
31: #define STATUS_BUSY     BIT(0)
32: #define STATUS_ERASE_FAILED BIT(2)
33: #define STATUS_PROG_FAILED BIT(3)
34: #define STATUS_ECC_MASK  GENMASK(5, 4)
35: #define STATUS_ECC_NO_BITFLIPS (0 << 4)
36: #define STATUS_ECC_HAS_BITFLIPS (1 << 4)
37: #define STATUS_ECC_UNCOR_ERROR (2 << 4)
38:

```

## 12.1 Get Features (0FH) and Set Features (1FH)

The GET FEATURES (0FH) and SET FEATURES (1FH) commands are used to monitor the device status and alter the device behavior. These commands use a 1-byte feature address to determine which feature is to be read or modified. Feature such as OTP can be enabled or disabled by setting specific feature bits (shown in the below table). The status register is mostly read, except WEL, which is a writable bit with the WRITE ENABLE (06H) command.

When a feature is set, it remains active until the device is power cycled or the feature is written to. Unless otherwise specified in the following table, once the device is set, it remains set, even if a RESET (FFH) command is issued.

Table 12-1. Features Settings

| Register   | Addr. | 7        | 6        | 5        | 4        | 3        | 2        | 1        | 0        |
|------------|-------|----------|----------|----------|----------|----------|----------|----------|----------|
| Protection | A0H   | BRWD     | Reserved | BP2      | BP1      | BP0      | INV      | CMP      | Reserved |
| Feature    | B0H   | OTP_PRT  | OTP_EN   | Reserved | ECC_EN   | BPL      | Reserved | Reserved | QE       |
| Status     | C0H   | Reserved | Reserved | ECCS1    | ECCS0    | P_FAIL   | E_FAIL   | WEL      | OIP      |
| Feature    | D0H   | Reserved | DS_S1    | DS_S0    | Reserved | Reserved | Reserved | Reserved | Reserved |
| Status     | F0H   | Reserved | Reserved | ECCSE1   | ECCSE0   | BPS      | Reserved | Reserved | Reserved |

特别说明：以上软件适配只是 bringup 简单调通，硬件可靠性需要高低温压测，可以咨询我司 HW 研发 AVL 物料验证支持流程；如遇特殊物料无法支持，也需走 AVL 验证支持流程由我司研发资源调试。

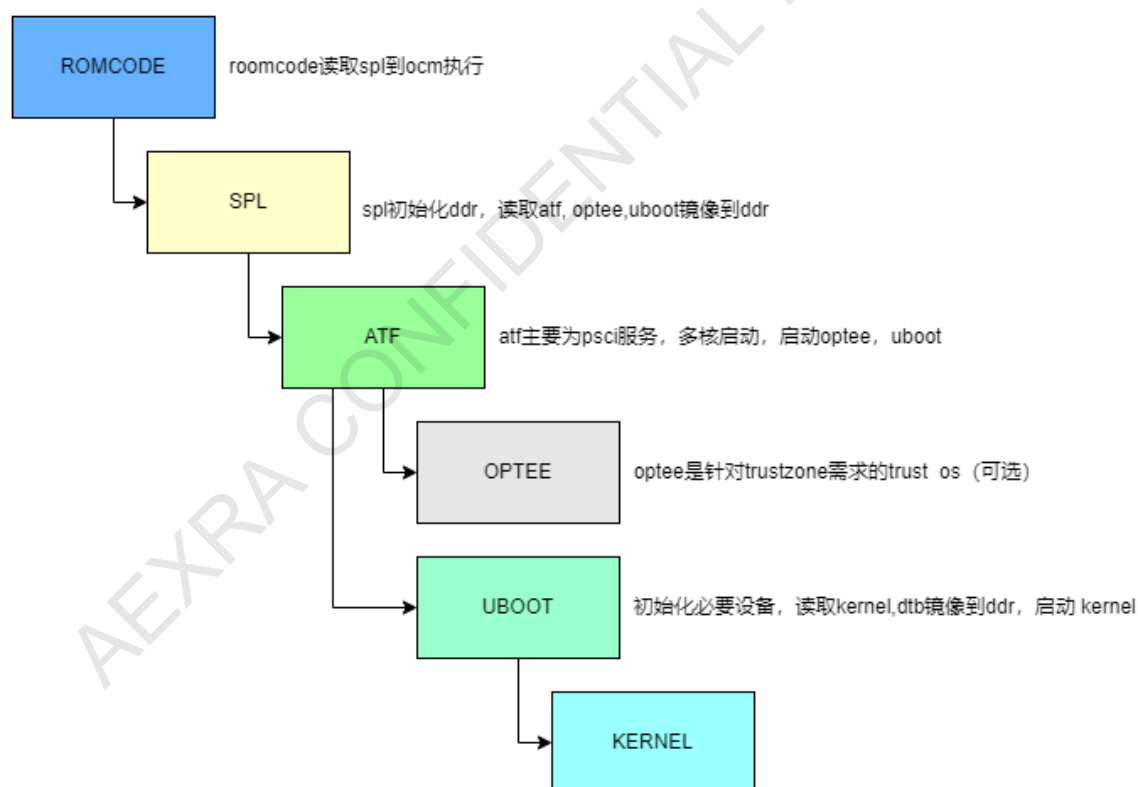
## 8 U-Boot 移植应用开发指南

### 8.1 概述

Linux 的启动需要有 bootloader 来引导，AX620E 系列单板的 bootloader 可选使用 U-Boot，当选用的外围芯片信号与参考设计不同时，需要修改 U-Boot 的相关配置文件。

#### 8.1.1 AX620E 启动流程

AX620E 芯片 64 位启动流程如下：

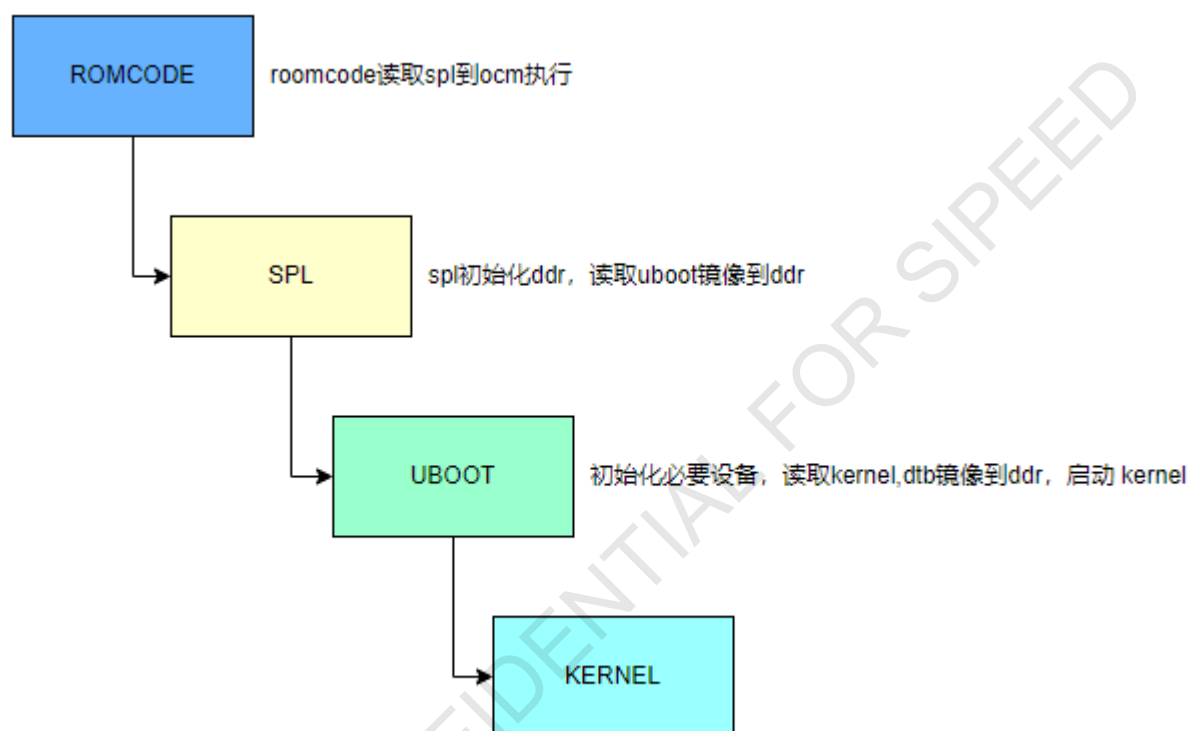


启动流程

如上图，romcode 读取 spl 镜像到 ocm（内部的 sarm）中运行，而 spl 获得执行权后，在启动

流程中需要初始化 ddr、读取 atf, optee, uboot 镜像到 ddr 上，atf 获得执行权后，完成 optee（可选）、uboot 的启动引导，U-Boot 获得执行权限后，完成 kernel 的加载与启动。

AX620E 芯片 32 位启动流程如下：



如上图，romcode 读取 spl 镜像到 ocm（内部的 sarm）中运行，而 spl 获得执行权后，在启动流程中需要初始化 ddr、读取 uboot 镜像到 ddr 上，U-Boot 获得执行权限后，完成 kernel 的加载与启动。

本文档主要介绍在 AX620E 单板上如何移植、烧写 U-Boot 以及相关 debug。

## 8.1.2 U-Boot 目录结构

boot/uboot 目录如下：

```
Makefile  README.md  u-boot-2020.04
```

Uboot 目录结构

其中 Makefile 是 AX 为项目工程编译添加的 u-boot 顶层 Makefile, 项目工程完整编译时, Make 系统会根据该 Makefile 描述进行 u-boot 的编译和打包工作。

u-boot-2020.04 为 AX 对开源 u-boot 修改后的代码, 主要的目录、文件如下所示, 详细可以参考该目录下的 README 文件。

u-boot-2020.04 主要目录文件

| 目录名       | 描述                     |
|-----------|------------------------|
| api/      | 与硬件无关的 API             |
| arch/     | 与架构体系相关的 code          |
| board/    | 各种单板的相关的定制代码           |
| cmd/      | 命令相关代码                 |
| common/   | 通用代码                   |
| configs/  | 配置文件                   |
| disk/     | 磁盘分区相关代码               |
| doc/      | 文档                     |
| drivers/  | 设备驱动代码                 |
| dts/      | 设备树 Kconfig 和 Makefile |
| env/      | Env 环境变量相关代码           |
| examples/ | 示例代码                   |

| 目录名       | 描述            |
|-----------|---------------|
| fs/       | 文件系统实现代码      |
| include/  | 头文件           |
| lib/      | 库文件           |
| Licenses/ | 许可证相关文件       |
| Makefile  | 主 Makefile 文件 |
| net/      | 网络实现代码        |
| README    | readme 帮助文件   |
| scripts/  | 脚本文件          |
| test/     | 测试代码          |
| tools/    | 工具文件夹         |
| .config   | 配置文件，编译后生成    |

UBOOT 提供 `unsigned char get_chip_type_id(void)` 来获取 CHIP TYPE，CHIP TYPE 定义在 `u-boot-2020.04/arch/arm/include/asm/arch-axera/boot_mode.h` 中，`get_chip_type_id` 定义在 `u-boot-2020.04/arch/arm/include/asm/arch-axera/ax620e.h` 中。下面是接口和芯片类型的具体定义：

```
typedef enum chip_type {
    NONE_CHIP_TYPE = 0x0,
    AX620Q_CHIP = 0x1,
    RESERVED_CHIP = 0x2,
    AX620V200_CHIP = 0x3,
    AX630C_CHIP = 0x4,
    AX620E_CHIP_MAX = 0x5,
```

```
} chip_type_e;

unsigned char get_chip_type_id(void)
{
    return misc_info->chip_type; /*返回值为 chip_type_e 类型*/
}
```

## 8.2 移植 U-Boot

上一节介绍了 U-Boot 的启动流程，以及 U-Boot 的主要代码目录、文件，本节主要介绍基于 AX620E 芯片，如何在 U-Boot 中添加新的 board，或者不添加新 board 直接修改参考 board。

### 8.2.1 基于 AX630C\_emmc\_arm64\_k419 新建 board

如果不想基于 AX630C\_emmc\_arm64\_k419 参考 board 直接修改，则需要新建自己的 board，下面介绍如何完全新建 test\_board。下面的小结中的操作除非特别说明，默认操作的根节点为 u-boot-2020.04。

#### 添加 board 图形配置选项

在 Kconfig 中添加新建 board 的选项，以便于图形化配置，修改 arch/arm/mach-axera/ax620e/Kconfig（一般同系列产品的不同 board 都会添加到此处）。

```
if ARCH_AXERA
.....

config TARGET_TEST_BOARD      <—添加新 board config 项
    bool "AX620E test board"
    help
        AX620E test board is a evaluation board for AXERA AX620E,
        with full function and phisical connectors support like
        usb2.0 host ports, JTAG, MAC, SDcard, HDMI, USB-2-serial...

source "board/axera/test_board/Kconfig"
```

```
.....  
endif
```

## 添加 board 默认配置文件

在 configs 目录下创建默认配置文件，复制 AX630C\_emmc\_arm64\_k419\_defconfig。

```
cd configs  
cp AX630C_emmc_arm64_k419_defconfig test_board_defconfig
```

然后根据实际的硬件设计对其中的部分 CONFIG 内容进行修改，常见的修改点：

```
CONFIG_SYS_TEXT_BASE= 0x5C000000  
CONFIG_DEFAULT_DEVICE_TREE="AX620E_emmc"    -->修改为"test_board"  
CONFIG_CONS_INDEX=1  
CONFIG_TARGET_AX620E_EMMC=y    -->修改为 CONFIG_TARGET_TEST_BOARD=y
```

### 说明：

**CONFIG\_SYS\_TEST\_BASE**

为代码段地址。

**CONFIG\_DEFAULT\_DEVICE\_TREE**

指定 dts 文件的名称，编译 uboot 的时候会同时编译 dts，编译系统据此到 arch/arm/dts 目录寻找同名带.dts 后缀的文件，生成.dtb 文件。

**CONFIG\_CONS\_INDEX**

debug 使用的串口号，需要根据硬件设计确认使用的串口号。

**CONFIG\_TARGET\_TEST\_BOARD=y**

名称可以自定义，但是需要与新增的 board config 项同名。

## 添加 dts 文件

在 arch/arm/dts 目录下创建新 board 的 dts 文件，直接复制后再进行修改：

```
cd arch/arm/dts  
cp AX620E_emmc.dts test_board.dts
```

对新创建的 `test_board.dts` 进行修改, 可以根据具体 `board` 的硬件设计, 添加或者删除部分设备描述。

修改 `arch/arm/dts/Makefile`, 添加新 `board` 的编译:

```
dtb-$(CONFIG_TARGET_TEST_BOARD) += \  
test_board.dtb
```

 说明:

`test_board.dtb`, 名字需要与 `defconfig` 中 `CONFIG_DEFAULT_DEVICE_TREE` 的值匹配一致。

## 添加 `board` 对应的板级文件夹

U-Boot 中每个 `board` 都有一个对应的文件夹存放板级文件, 该目录中可以放诸如 `board` 的 `pin` 配置文件等, 直接复制后再进行修改:

```
cd board/axera  
cp -r ax620e_emmc test_board
```

对 `test_board` 目录里的文件再做相关修改。

## 修改文件名

将 `copy` 过来的文件下的文件重新命名:

```
cd test_board  
mv AX620e_emmc.c test_board.c
```

## 修改 `Makefile`

文件重新命名后, `Makefile` 也要做同步修改:

```
obj-y += ax620e_emmc.o      ->test_board.o  
obj-y += pinmux.o
```

## 修改 Kconfig

Kconfig 也需要同步修改：

```
if TARGET_AX620E_EMMC          —>TARGET_TEST_BOARD, 要与 defconfig 中一致

config SYS_BOARD
    default "ax620e_emmc "      ->"test_board"

config SYS_VENDOR
    default "axera"

config SYS_CONFIG_NAME
    default "ax620e_emmc "      ->"test_board"

config BOARD_SPECIFIC_OPTIONS # dummy
    def_bool y

endif
```

## 配置 Pinmux

Pinmux 主要是配置 pin 的 function select、上下拉、驱动能力等配置，操作步骤如下所示：

使用 pinmux 配置表格，在表格中进行选择配置，之后点击“Export”按钮，在表格目录下会

生成.h 头文件。

ax620e\_EVB\_PIN\_MUX - Excel

文件开始插入页面布局公式数据审阅视图告诉我想要做什么...

剪贴板

格式刷

等线11A A

自动换行

合并后居中

常规

条件格式

套用表格格式

单元格样式

插入

删除

格式

剪贴板

字体

对齐方式

数字

样式

单元格

K1

|    | A         | B                    | C            | D         | L               | M                | N     | O     | P     | Q       | R            |
|----|-----------|----------------------|--------------|-----------|-----------------|------------------|-------|-------|-------|---------|--------------|
| 1  |           | Board ID = 0 for EVB |              |           |                 |                  |       |       |       |         |              |
| 2  |           |                      |              |           |                 |                  |       |       |       |         |              |
| 3  |           | Export               |              |           |                 |                  |       |       |       |         |              |
| 4  | group ID  | PAD name             | base address | offset    | func_sel[18:16] | function         | pu[7] | pd[6] | st[4] | ds[3:0] | notes        |
| 5  | G1        | VI_D0                | 0x2300000    | 0xC       | 4               | I2S0_DIN0        | 0     | 0     | 0     | 3       |              |
| 6  |           | VI_D1                |              | 0x18      | 4               | I2S0_SCLK        | 0     | 0     | 0     | 3       |              |
| 7  |           | VI_D2                |              | 0x24      | 4               | I2S0_DOUT        | 0     | 0     | 0     | 3       |              |
| 8  |           | VI_D3                |              | 0x30      | 4               | I2S0_MCLK        | 0     | 0     | 0     | 3       |              |
| 9  |           | VI_D4                |              | 0x3C      | 4               | I2S0_DIN1        | 0     | 0     | 0     | 3       |              |
| 10 |           | VI_D5                |              | 0x48      | 0               | VI_D5            | 0     | 0     | 0     | 3       |              |
| 11 |           | VI_D6                |              | 0x54      | 0               | VI_D6            | 0     | 0     | 0     | 3       |              |
| 12 |           | VI_D7                |              | 0x60      | 0               | VI_D7            | 0     | 0     | 0     | 3       |              |
| 13 |           | VI_D8                |              | 0x6C      | 0               | VI_D8            | 0     | 0     | 0     | 3       |              |
| 14 |           | VI_D9                |              | 0x78      | 0               | VI_D9            | 0     | 0     | 0     | 3       |              |
| 15 | G6        | VI_CLK0              | 0x2304000    | 0x84      | 4               | I2S0_LRCK        | 0     | 0     | 0     | 3       |              |
| 16 |           | I2C0_SCL             |              | 0xC       | 0               | I2C0_SCL         | 1     | 0     | 0     | 3       |              |
| 17 |           | I2C0_SDA             |              | 0x18      | 0               | I2C0_SDA         | 1     | 0     | 0     | 3       |              |
| 18 |           | I2C1_SCL             |              | 0x24      | 0               | I2C1_SCL         | 1     | 0     | 0     | 3       |              |
| 19 |           | I2C1_SDA             |              | 0x30      | 0               | I2C1_SDA         | 1     | 0     | 0     | 3       |              |
| 20 |           | UART0_TXD            |              | 0x3C      | 0               | UART0_TXD        | 1     | 0     | 0     | 3       |              |
| 21 |           | UART0_RXD            |              | 0x48      | 0               | UART0_RXD        | 1     | 0     | 0     | 3       |              |
| 22 |           | UART1_TXD            |              | 0x54      | 0               | UART1_TXD        | 1     | 0     | 0     | 3       |              |
| 23 |           | UART1_RXD            |              | 0x60      | 0               | UART1_RXD        | 1     | 0     | 0     | 3       |              |
| 24 |           | UART2_TXD            |              | 0x6C      | 0               | UART2_TXD        | 1     | 0     | 0     | 3       |              |
| 25 | UART2_RXD | 0x78                 | 0            | UART2_RXD | 1               | 0                | 0     | 3     |       |         |              |
| 26 | UART3_TXD | 0x84                 | 0            | UART3_TXD | 1               | 0                | 0     | 3     |       |         |              |
| 27 | UART3_RXD | 0x90                 | 0            | UART3_RXD | 1               | 0                | 0     | 3     |       |         |              |
| 28 |           | EMAC_PTP_PPS0        | 0x2304000    | 0xC       | 6               | GPIO1_A8         | 1     | 0     | 0     | 3       | sensor_rstn1 |
| 29 |           | EMAC_PTP_PPS1        |              | 0x18      | 4               | SEN_VSYNC_D0     | 1     | 0     | 0     | 3       | sensor_vs    |
| 30 |           | EMAC_PTP_PPS2        |              | 0x24      | 2               | SD_CARD_DETECT_N | 1     | 0     | 0     | 3       |              |
| 31 |           | EMAC_PTP_PPS3        |              | 0x30      | 6               | GPIO1_A11        | 1     | 0     | 0     | 3       | sensor_rstn2 |
| 32 |           | RGMII_MDCK           |              | 0x3C      | 0               | RGMII_MDCK       | 0     | 0     | 0     | 3       |              |
| 33 |           | RGMII_MDIO           |              | 0x48      | 0               | RGMII_MDIO       | 0     | 0     | 0     | 3       |              |
| 34 |           | EPHY_CLK             |              | 0x54      | 0               | EPHY_CLK         | 0     | 0     | 0     | 3       |              |
| 35 |           | EPHY_RSTN            |              | 0x60      | 0               | EPHY_RSTN        | 0     | 0     | 0     | 3       |              |
| 36 |           | EPHY_LED0            |              | 0x6C      | 0               | EPHY_LED0        | 0     | 0     | 0     | 3       |              |

Pinmux 配置表格

将产生的头文件添加到项目编译目录中。

/build/projects/test\_board/pinmux/test\_board\_pinmux.h

对 pinmux.c 进行修改。

```
#include <asm/io.h>
#include <linux/bitops.h>
#include <asm/arch-axera/ax620e.h>
#include <asm/arch/boot_mode.h>

#define DPHYTX_BASE      0x230A000UL
#define DPHY_REG_LEN     0x1000
#define DPHYTX_SW_RST_SET 0x46000B8
#define DPHYTX_SW_RST_SHIFT BIT(6)
```

```
#define DPHYTX_MIPI_EN      0x23F110C
#define PINMUX_FUNC_SEL    GENMASK(18, 16)

static unsigned int ax620Q_EVB_pinmux[] = {
#include "AX620Q_EVB_pinmux.h"
};

static unsigned int ax620Q_Demo_pinmux[] = {
#include "AX620Q_DEMO_pinmux.h"
};

static unsigned int ax620V200_EVB_pinmux[] = {
#include "AX630C_AX620V200_EVB_pinmux.h"
};

static unsigned int ax620V200_Demo_pinmux[] = {
#include "AX630C_AX620V200_DEMO_pinmux.h"
};

static unsigned int ax630C_EVB_pinmux[] = {
#include "AX630C_AX620V200_EVB_pinmux.h"
};

static unsigned int ax630C_Demo_pinmux[] = {
#include "AX630C_AX620V200_DEMO_pinmux.h"
};

static unsigned int test_board_pinmux[] = {
#include "test_board_pinmux.h" —>添加
};

struct pinmux {
    unsigned int *data;
    unsigned int size;
};

static struct pinmux ax620E_pinmux_tbl[AX620E_BOARD_MAX] = {
    [AX620Q_EVB] =
        {ax620Q_EVB_pinmux,
```

```

        sizeof(ax620Q_EVB_pinmux) / sizeof(unsigned int)),
[AX620Q_DEMO] =
    {ax620Q_Demo_pinmux,
    sizeof(ax620Q_Demo_pinmux) / sizeof(unsigned int)},
[AX620V200_EVB] =
    {ax620V200_EVB_pinmux,
    sizeof(ax620V200_EVB_pinmux) / sizeof(unsigned int)},
[AX620V200_DEMO] =
    {ax620V200_Demo_pinmux,
    sizeof(ax620V200_Demo_pinmux) / sizeof(unsigned int)},
[AX630C_EVB] =
    {ax630C_EVB_pinmux,
    sizeof(ax630C_EVB_pinmux) / sizeof(unsigned int)},
[AX630C_DEMO] =
    {ax630C_Demo_pinmux,
    sizeof(ax630C_Demo_pinmux) / sizeof(unsigned int)},
[test_board] =
    {test_board_pinmux,
    sizeof(test_board_pinmux) / sizeof(unsigned int)},
};

int pinmux_init(void)
{
    int i;
    int index = get_board_id();
    u8 is_dphytx;

    if (index < 0 || index > AX620E_BOARD_MAX - 1)
        return 0;

    for (i = 0; i < ax620E_pinmux_tbl[index].size; i += 2) {
        is_dphytx = ax620E_pinmux_tbl[index].data[i] - DPHYTX_BASE <
DPHY_REG_LEN ? 1 : 0;
        //when dphytx select gpio func 1.set reset 2.mipi disable 3.func sel &
config
        if (is_dphytx && (ax620E_pinmux_tbl[index].data[i + 1] &
PINMUX_FUNC_SEL)) {
            writel(DPHYTX_SW_RST_SHIFT, DPHYTX_SW_RST_SET);
            writel(0, DPHYTX_MIPI_EN);
        }
    }
}

```

```
    }  
    writel(ax620E_pinmux_tbl[index].data[i + 1],  
           (long)ax620E_pinmux_tbl[index].data[i]);  
}  
  
return 0;  
}
```

#### 说明:

- 添加 test\_board 头文件 test\_board\_pinmux.h 到结构体中。
- Pin 配置是与硬件 board 设计强相关的，需要与硬件确认好。

## 配置 Memory 大小

单板的 memory 配置可能不尽相同，需要根据硬件设计或者软件的使用，配置 memory 的使用大小，修改 test\_board.c 文件中如下函数：

```
int dram_init(void)  
{  
    gd->ram_size = 0x80000000; —>指定 ram size, 默认 2G, 请根据实际 DDR 大小进行调整  
  
    return 0;  
}  
  
int dram_init_banksize(void)  
{  
    gd->bd->bi_dram[0].start = 0x40000000; —>指定 ram start addr, uboot 使用的  
SARM 起始地址, 为固定地址  
    gd->bd->bi_dram[0].size = 0x80000000; —>默认 2G, 请根据实际 DDR 大小进行调整  
  
    return 0;  
}
```

🔑 说明：

1. 注意 uboot 是运行在 ddr 上的，ddr 的起始地址是 0x40000000。
2. 如果用户想要修改 uboot 的运行地址，可以通过修改上述描述的信息进行修改；

比如：想要运行在 ddr 首地址（0x40000000）到 1024M 的空间上，配置如下：

```
gd->ram_size = 0x40000000;  
gd->bd->bi_dram[0].start = 0x40000000;  
gd->bd->bi_dram[0].size = 0x40000000;
```

3. arm64 位版本为优化开机速度我们默认关闭了 uboot 代码段的 relocate，如果需要 uboot 代码段 relocate 请删除下面配置： CONFIG\_UBOOT\_SKIP\_RELOCATE=y

## 添加 board 对应的头文件

在 include/configs 目录下创建新 board 的头文件，直接复制后再进行修改：

```
cd include/configs  
cp ax620e_emmc.h test_board.h
```

在 test\_board.h 头文件中会 include 同级目录下的 ax620e\_common.h，该头文件大多数是以“CONFIG\_”开头的宏定义，主要是做 uboot 功能裁剪用。ax620e\_common.h 作为所有 ax620e 系列芯片 board 共用的一个配置头文件，board 差异化的配置放在 test\_board.h 中。

## 编译修改

前面已经完成了新 board 的创建工作，已经可以直接进行编译 u-boot.bin 了，ARM64 的情况下在 u-boot-2020.04 目录下执行如下命令：

```
make CROSS_COMPILE=aarch64-none-linux-gnu- test_board_defconfig  
make CROSS_COMPILE=aarch64-none-linux-gnu- -j4
```

ARM32 的情况下在 u-boot-2020.04 目录下执行如下命令：

```
make CROSS_COMPILE=arm-linux-gnueabi- test_board_defconfig  
make CROSS_COMPILE=arm-linux-gnueabi- -j4
```

如果编译通过，则会在 u-boot-2020.04 目录下生成 u-boot.bin。

uboot 顶层 Makefile 修改，项目工程编译时会传递 PROJECT 变量，可以根据该变量指定编译

哪个 board 和打包相关文件。

```
.PHONY: all prepare uboot
```

```
UBOOT_DIR ?= u-boot-2020.04
```

```
DEFAULT_CONFIG_STR := _defconfig
```

```
CONFIG ?=$(PROJECT)$(DEFAULT_CONFIG_STR)
```

—>此处的 `PROJECT_NAME` 为项目工程定义，因此需要在项目的 `project list` 里添加

—>`test_board`

```
install:
```

```
@$(ECHO) -e $(GREEN) "uboot $@..." $(DONE)
```

```
@$(MKDIR) $(BUILD_PATH)/out/$(PROJECT) /
```

```
@$(MKDIR) $(BUILD_PATH)/out/$(PROJECT)/debug
```

## 8.2.2 基于 AX620E 参考 board 直接修改

U-Boot 中很多文件夹和文件几乎不用关注，基于 AX620E 参考 board（AX630C\_emmc\_arm64\_k419）直接修改的话，相对于完全新建 board 的话则要简单快捷。

### 修改文件

如 8.2.1 节新建 board 中涉及的相关文件，比方说直接修改 AX630C\_emmc\_arm64\_k419 的 board 文件，主要可能修改的几个目录、文件如下：

#### 1. configs/AX630C\_emmc\_arm64\_k419

主要定义以 `CONFIG_XXX` 相关的功能特性开关。

#### 2. arch/arm/mach-axera/ax620e

`arch` 目录下有很多 CPU 架构的目录，我们使用的是 `arm` 系列，`arch/arm` 目录又有许多 `mach-xx` 的目录，这些是不同厂商的多种系列 `cpu` 有关文件目录，只需要关注 `mach-axera/AX620E` 目录中的文件。

#### 3. arch/arm/dts/AX620E\_emmc.dts、AX620E.dtsi

uboot 目前也逐渐开始向 kernel 看齐，支持 `dts`，该目录为 board 对应的 `dts` 文件。

#### 4. board/axera/ax620e\_emmc /

不同 board 的外围器件、管脚配置可能不同，board 相关的代码目录。

#### 5. include/configs/ ax620e\_emmc.h、ax620e\_common.h

board 的配置文件（诸如 ddr 地址，uart 地址等）。

## 编译

在 boot/uboot 目录，操作如下：

```
make p=AX630C_emmc_arm64_k419 all
```

该命令其实包含了 2 个操作：

```
make CROSS_COMPILE=aarch64-none-linux-gnu- AX630C_emmc_arm64_k419  
make CROSS_COMPILE=aarch64-none-linux-gnu- -j4
```

编译成功后，会在 u-boot-2020.04 目录下生成 u-boot.bin。

再次执行：

```
make p=AX630C_emmc_arm64_k419 install
```

该 install 命令操作会做如下 1 个操作：

拷贝 u-boot.bin 到工程项目的 target 目录：build/out/AX630C\_emmc\_arm64\_k419。

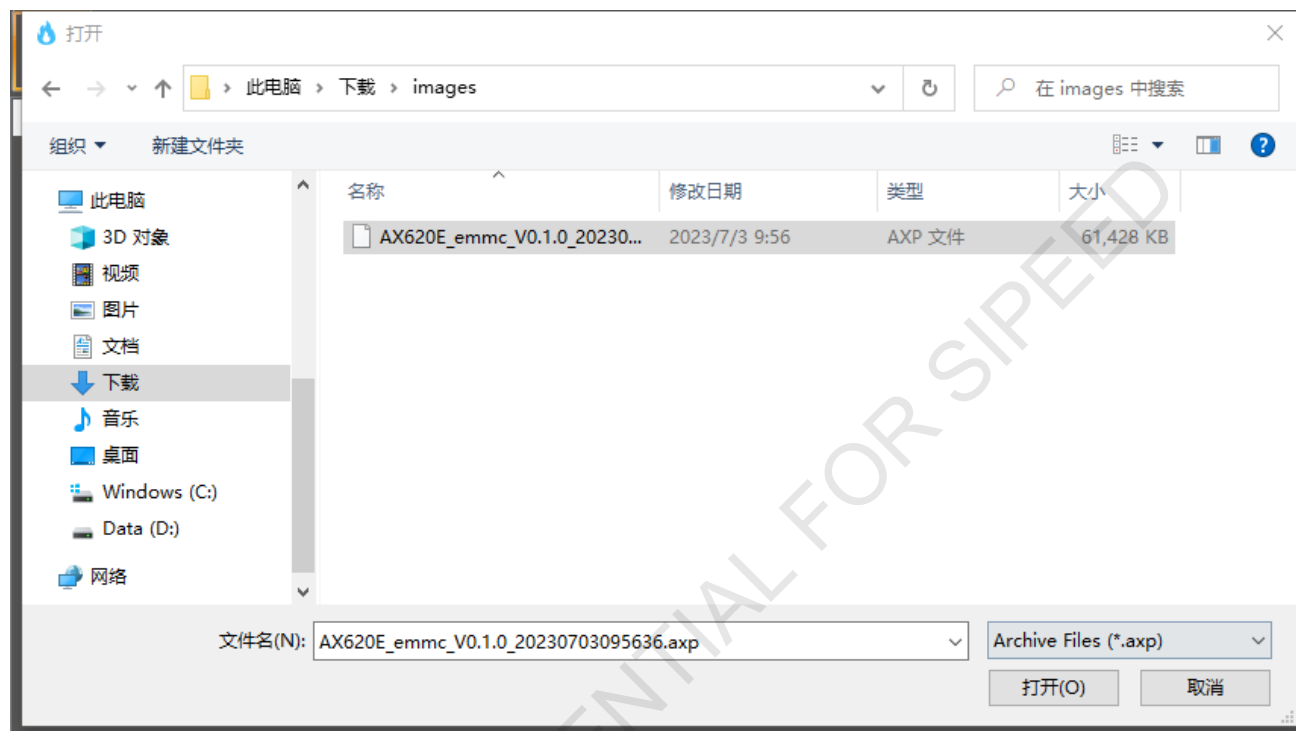
## 8.3 烧写 U-Boot

上一节已经生成了 u-boot.bin，本节介绍如何将 u-boot 镜像烧写入存储中。

### 8.3.1 AXDL 烧写 u-boot.bin

AXDL 是运行在 Windows 平台上的桌面应用程序，主要用于 FLASH 分区、镜像下载等功能。对于板子第一次烧写，需要使用该工具进行烧写，之后的烧写可以使用多种手段。烧写步骤如下：

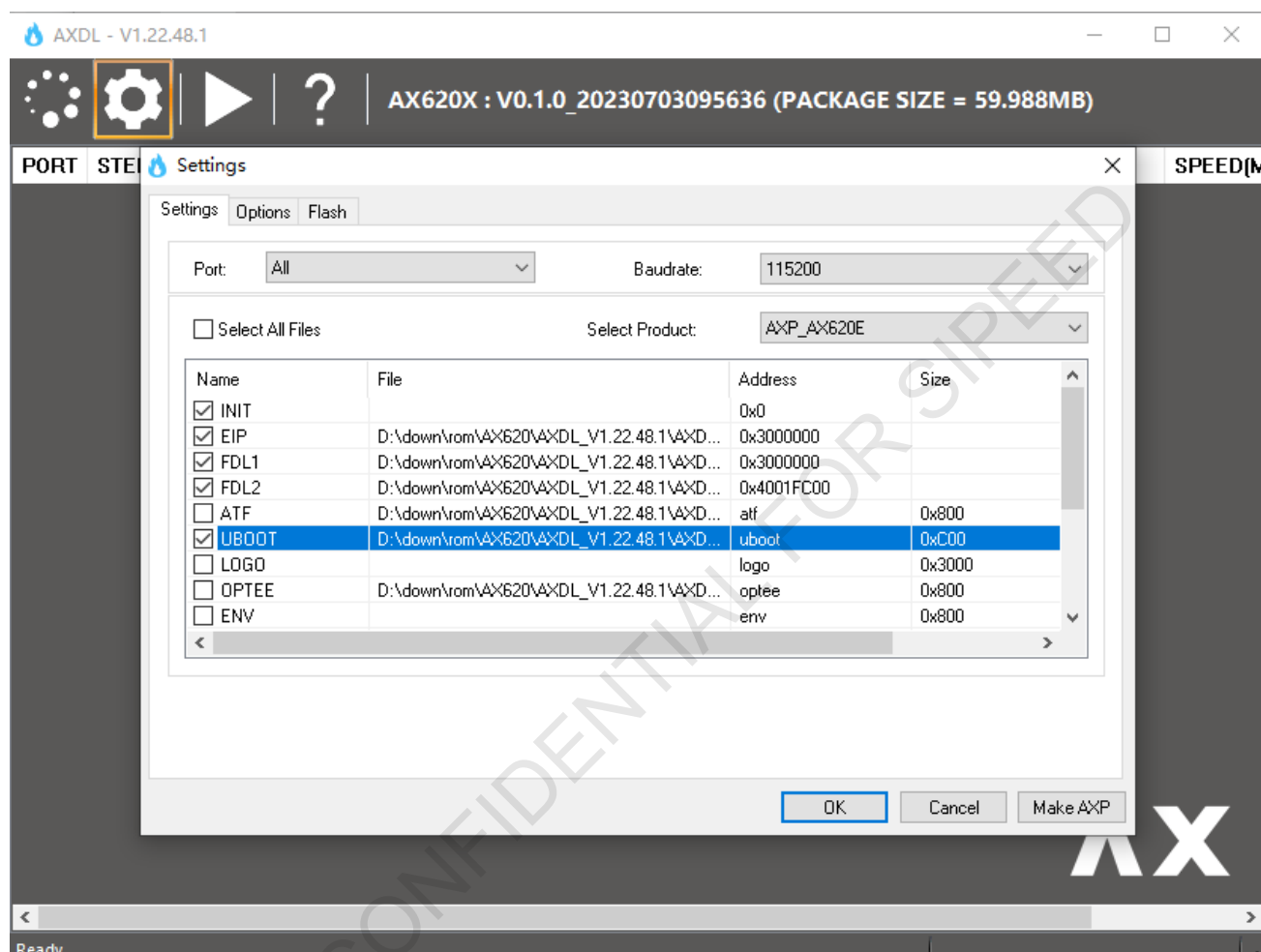
**步骤1** AXDL 工具栏点击加载按钮，选择.axp 镜像包文件。



选择.axp 镜像包

**步骤2** 工具栏点击“设置”按钮，在“Settings”页面 AXDL 已将 axp 镜像包的镜像文件释放到本地 temp 目录并进行配置。鼠标双击或右键单击镜像文件激活文件选

择框选择要烧写的 u-boot.bin，其他的镜像可不勾选。



替换 u-boot\_signed.bin 镜像

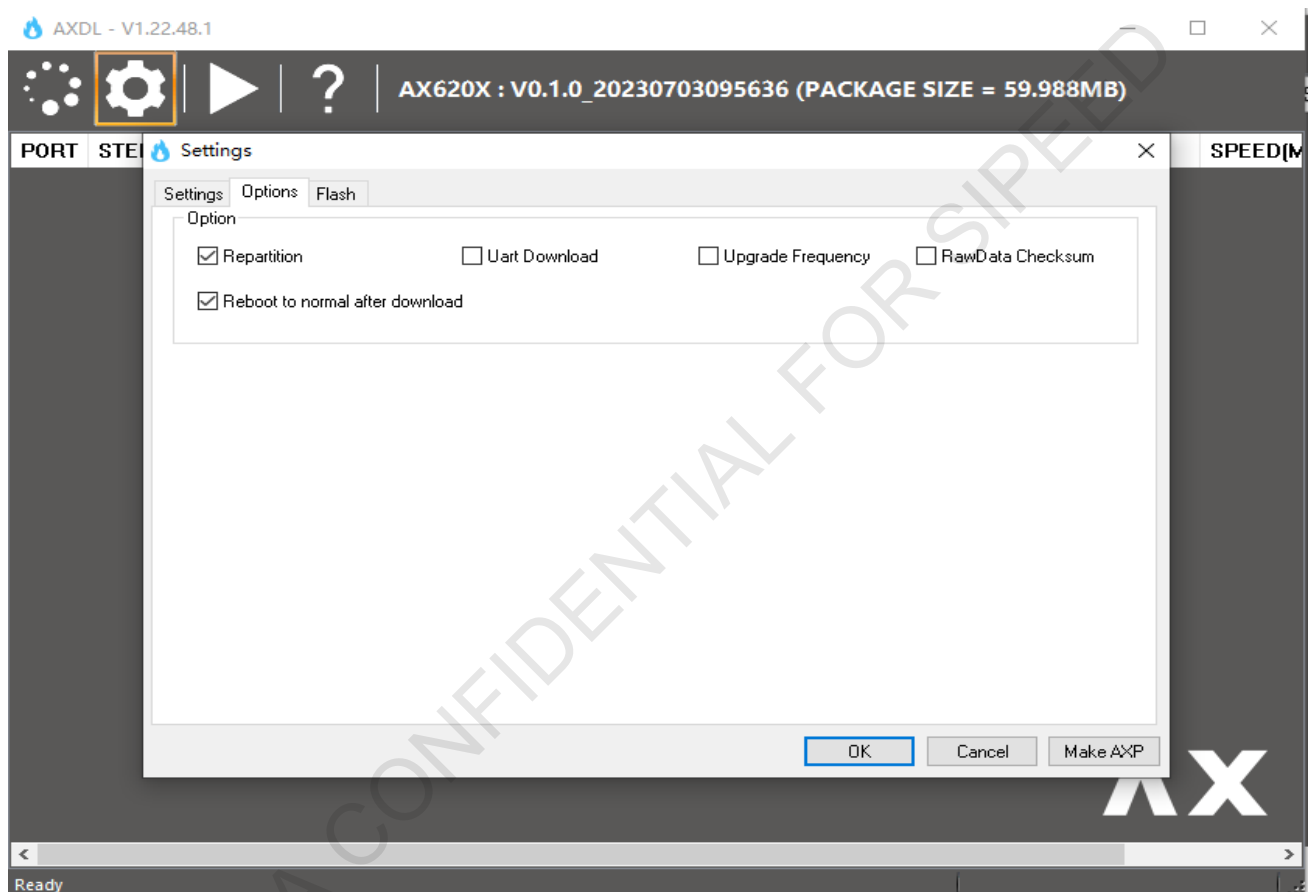
**步骤3** 点击“OK”确认设置，在工具栏点击“开始”按钮  启动下载。此时给板子上电，下载工具将自动与单板握手交互，开始下载。

**说明：**

如果只是单独 debug U-boot，则只需要在工具中勾选 UBOOT 即可，UBOOT\_BK 为 uboot 的备份分区。

### 8.3.2 AXDL repartiton

AXDL repartiton 是下载工具中的一个 option，如下图所示，该选项有个特殊功能，就是重写分区表：



#### Repartition option

若不勾选该选项，在./u-boot-2020.04/include/configs/ax620e\_common.h 文件中修改 bootargs 来进行分区表的修改，代码中会解析 bootargs 中的配置获取分区信息，fdl 下载和 uboot 启动过程会根据改分区信息进行下载和启动：

build/projects/AX630C\_emmc\_arm64\_k419/partition\_ab.mak 中定义了 bootargs：

```
KERNEL_BOOTARGS := "$(OS_MEM) console=ttyS0,115200n8  
earlycon=uart8250,mmio32,0x4880000  
board_id=0,boot_reason=0x0,initcall_debug=0 loglevel=8 \
```

```
usbcore.autosuspend=-1 root=$(ROOTFS_DEV) rootfstype=$(ROOTFS_TYPE) rw
rootwait blkdevparts=mmcblk0:$(FLASH_PARTITIONS) "
```

./u-boot-2020.04/include/configs/ax620e\_common.h 中 bootargs 如下

```
/* bootargs for eMMC */
```

```
#define BOOTARGS_EMMC DEBUG_BOOT_ARGS KERNEL_BOOTARGS
```

```
#define BOOTARGS_SPINOR KERNEL_BOOTARGS
```

若勾选了该选项，下载和启动将使用 AXP 包中 AX6xx\_xx.xml 里面描述的分区信息进行分区使用，如下图所示：

```
<Partitions strategy="1" unit="2">
  <Partition gap="0" id="spl" size="768" />
  <Partition gap="0" id="ddrinit" size="512" />
  <Partition gap="0" id="atf" size="256" />
  <Partition gap="0" id="atf_b" size="256" />
  <Partition gap="0" id="uboot" size="1536" />
  <Partition gap="0" id="uboot_b" size="1536" />
  <Partition gap="0" id="env" size="1024" />
  <Partition gap="0" id="optee" size="1024" />
  <Partition gap="0" id="optee_b" size="1024" />
  <Partition gap="0" id="logo" size="6144" />
  <Partition gap="0" id="logo_b" size="6144" />
  <Partition gap="0" id="dtb" size="1024" />
  <Partition gap="0" id="dtb_b" size="1024" />
  <Partition gap="0" id="kernel" size="65536" />
  <Partition gap="0" id="kernel_b" size="65536" />
  <Partition gap="0" id="rootfs" size="2097152" />
  <Partition gap="0" id="param" size="5120" />
  <Partition gap="0" id="soc" size="1048576" />
  <Partition gap="0" id="opt" size="0xffffffff" />
</Partitions>
```

#### AX630C\_emmc\_arm64\_k419.xml 分区信息

##### ！ 注意：

- uboot 分区大小不得超过 2M, 不建议修改 uboot, env 分区大小；
- 不推荐取消 repartition。只有需要单独下载某些 img, 且知道分区没有改变的时候, 才可以取消 repartition。

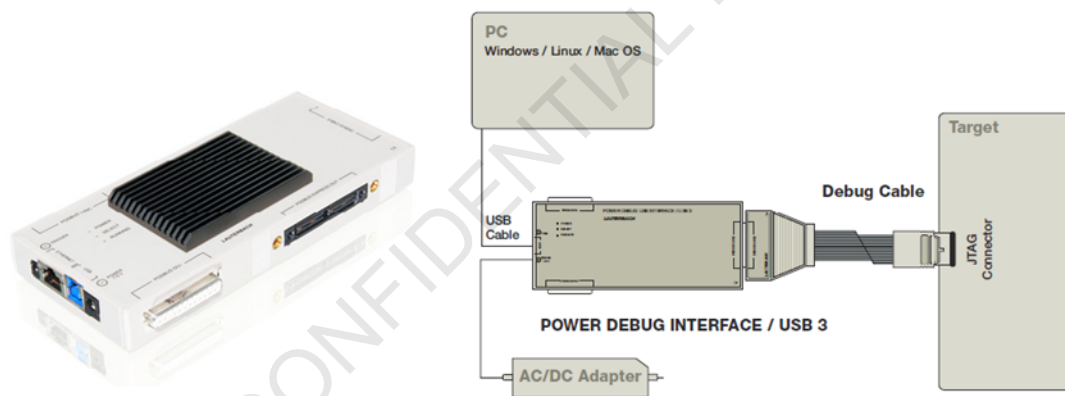
## 8.4 U-Boot Debug 方法

U-Boot 的 debug 方法，最简单的 debug 手段就是直接使用串口，代码中加入需要跟踪的打印语句，然后根据串口打印的 log 来判断 code 执行的流程，出问题时候的状态信息等。但如果有的条件下，JTAG 工具一般都具备单步跟踪、断点等功能，使用 JTAG 工具来 debug 会比较方便，提高调试效率。

本章将简单介绍下使用 Lauterbach 的 TRACE32 来进行 U-Boot 的调试。

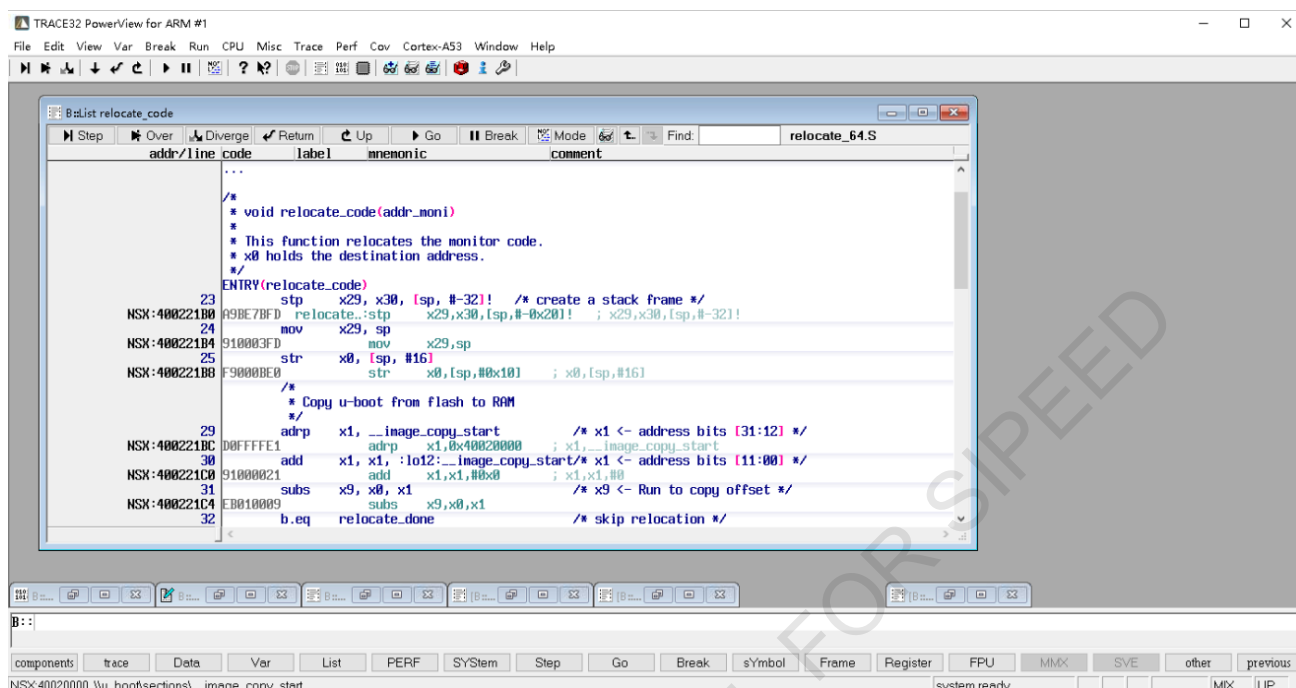
### 8.4.1 TRACE32 设备工具简介

Lauterbach 的 TRACE32 设备如下图所示：



设备和连接示意图

上图左半部分为调试盒子硬件实物，右侧为 debug 时的连接示意图，运行 TRACE32 debug 程序的电脑端通过 USB 线与盒子连接，盒子通过 cable 连接 target 单板。通过电脑上的 Debug 程序就可以进行 code 的 debug，程序界面如下图，程序的安装等细节请参考 Lauterbach 的文档。



TRACE32 软件界面

## 8.4.2 使用 TRACE32 调试 U-Boot 程序

调试步骤如下所示：

**步骤1** 单板 strappin 配置进入 USB 下载模式。

**步骤2** AX620E 项目 U-Boot 代码运行在 ddr 中。因此 trace 连接上单板后，可以加载 debug 的 u-boot.bin 到 0x5C000000 地址，加载符号表，并且设置断点到 U-Boot 的第一条指令地址，即可开始进行代码调试 debug，如[图 8-10](#) ~ 0 所示。

**说明：**

- u-boot bin 镜像路径：boot/uboot/u-boot-2020.04/u-boot.bin。
- u-boot 符号表路径：boot/uboot/u-boot-2020.04/u-boot。
- AX620E 系列的 trace32 脚 ax620e\_base.cmm，内容如后文所示。
- 注意 method 选择为 Onchip 类型。
- U-Boot 会对代码进行 relocate，会与编译链接地址不一致，因此在 relocate 前的代码跟踪可以在[图 812](#)加载符号表之后，搜索全局变量 gd。而 relocate 后，代码被搬到 ddr 的高端

地址运行，因此需要通过 0 到 0 获取 relocate 偏移，然后重新加载符号表。0 中 reloc\_off 即为 relocate 的偏移量，relocate 后需要 method 选择为 Onchip 类型，否则 break 不住。

ax620e\_base.cmm 内容:

```
RESet
SYStem.RESET
SYStem.CPU CortexA53
SYStem.CONFIG CoreNumber 1.
CORE.NUMber 1.

SYStem.CONFIG DEBUGACCESSPORT 0.
SYStem.CONFIG APBACCESSPORT 0.
SYStem.CONFIG MEMORYACCESSPORT 1.
SYStem.CONFIG AXIACCESSPORT 1.

SYStem.CONFIG COREDEBUG Base DAP:0x80610000 ;DAP:0x80710000 ;DAP:0x80610000
DAP:0x80710000
SYStem.CONFIG CTI      Base DAP:0x80620000 ;DAP:0x80720000 ;DAP:0x80620000
DAP:0x80720000
SYStem.CONFIG BMC      Base DAP:0x80630000 ;DAP:0x80730000 ;DAP:0x80630000
DAP:0x80730000
SYStem.CONFIG ETM      Base DAP:0x80640000 ;DAP:0x80740000 ;DAP:0x80640000
DAP:0x80740000

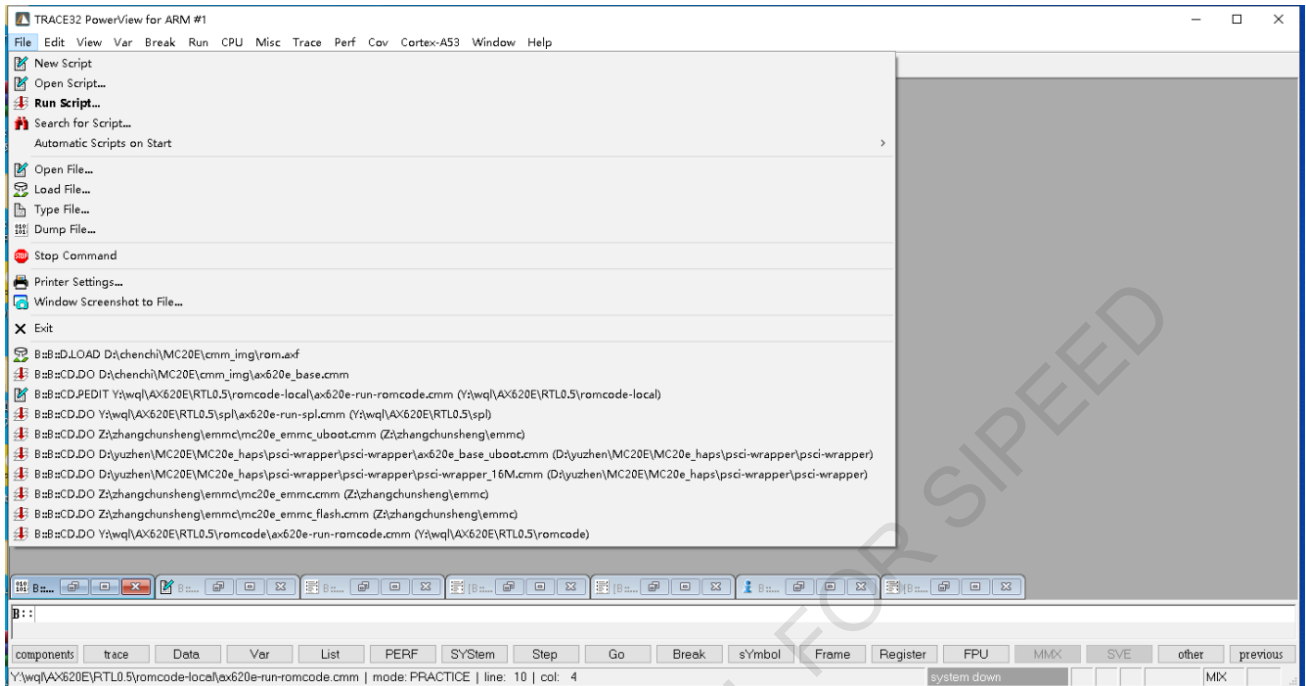
SYStem.CONFIG GICD Type GIC400
SYStem.CONFIG GICD Base AD:0x01851000
SYStem.CONFIG GICC Base AD:0x01852000
SYStem.CONFIG GICH Base AD:0x01854000
SYStem.CONFIG GICV Base AD:0x01856000

SYStem.JtagClock 5MHz

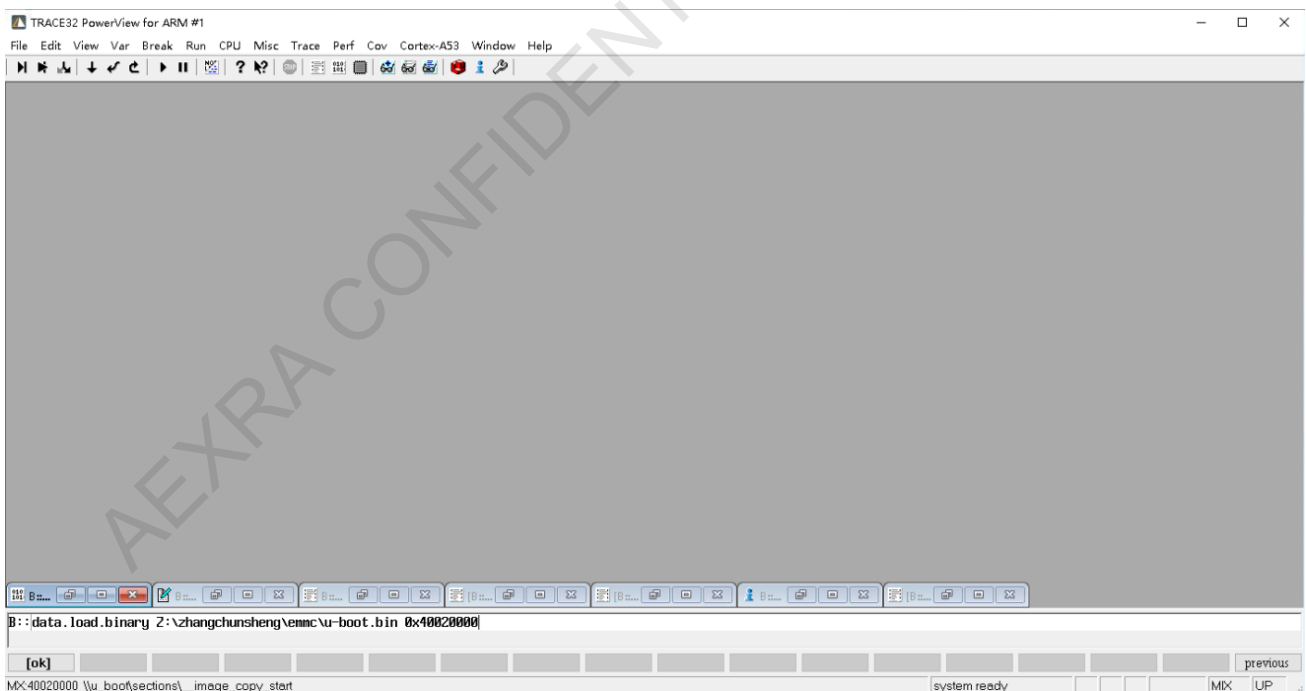
SYStem.CONFIG.DEBUGPORTTYPE SWD
SYStem.Option PWRREQ ON

SYStem.Mode Up

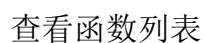
ENDDO
```



运行 cmm 脚本连接单板



加载 u-boot.bin 到运行地址





可参考如下 uboot\_debug.cmm 的加载过程：

```
do ax620e_base.cmm
&uboot_addr= 0x5C000000
&image_path="Y:\AX620E\build\out\AX620E_emmc\objs\boot\uboot\u-boot-2020.04"
data.load.binary &image_path\u-boot.bin &uboot_addr
register.set pc &uboot_addr
symbol.SourcePATH.Translate "/home/AX620E/boot/uboot/u-boot-2020.04"
"Y:\AX620E\boot\uboot\u-boot-2020.04"

data.load.elf &image_path\u-boot /nocode
register.set pc &uboot_addr
data.load.elf Y:\AX620E\build\out\AX620E_haps\objs\boot\uboot\u-boot-
2020.04\u-boot 0x3fec8000 /nocode
;go
```

## 8.5 附录

### 8.5.1 网络支持

#### 基础功能

AX620E 参考 board 的 U-Boot 中支持网络功能，有些 U-Boot 的场景可以使用网络功能。

举例使用网络给 u-boot 传输镜像，步骤如下所示：

- 步骤1** 在本地或者 Linux 服务器端建立 tftp server 和 DHCP server，具体可参考网上案例（例如 tftpd64）。
- 步骤2** 在 u-boot 启动过程中按键，进入 cmd 模式。
- 步骤3** 输入 DHCP 分配 IP 地址。
- 步骤4** 在板子 cmd 命令行输入 tftpboot 0x40200000 10.126.11.146:xxx.bin 命令，其

中，10.126.11.146 为服务器端 IP 地址，请根据具体的 Server IP 修改。

如上，通过单板连入网络，将 server 端 bin 文件传输到开发板内存地址 x40200000。

提供两个安装包，供参考使用；

Windows dhcp server 软件下载官网：<http://www.dhcpserver.de/cms/>



Tftpd32.rar

Windows tftp server 软件包：

## 扩展功能——TFTP 网络升级

### 实现原理

1. SDK 新增开机 tftp\_ota 启动模式场景，循环处理 2、3 完成所有升级镜像的写入
2. 通过 tftpboot 命令将 Linux 服务器端布置正确的升级镜像文件通过网络传输到板端内存
3. 将内存数据写入镜像文件对应的 flash 分区(支持的 flash 有 emmc、nand、nor)

### 使用步骤

**步骤 1** 将需要升级的镜像文件上传到 Linux 服务器

升级文件选择：

对于 emmc project 编译 sdk 包，从路径/build/out/AX6xx\_xxx/images/sd\_update\_pack 目录下获取。

**步骤 2** 在 uboot 或内核 console 设置 env 配置要升级的文件

(1) SDK env 支持情况：

uboot console 可直接使用 uboot 原生 env 命令；

kernel console 可使用 fw\_printenv 查看全部 env 变量，fw\_setenv 写单项 env 变量

(2) 配置 env 以进行 tftp 升级:

setenv ota\_server\_ip server 服务器 IP 地址 (必须存在)

setenv ota\_server\_path server 服务器路径 (可以不存在, 直接使用 server 服务器根目录)

setenv ota\_repartition true (分区有变更时需要配置, 全包升级建议配置)

setenv ota\_xml [xml 后缀的文件名]

setenv ota\_[partname] [镜像文件名] (单个或多个)

setenv ota\_ready true

**步骤 3** 重启下次开机 uboot 根据 env 配置自动进行 tftp 升级软件镜像, 或者直接执行升级命令 axera\_ota 进行升级。

Eg:

```
setenv ota_server_ip 192.168.1.8
setenv ota_xml AX620E.xml
setenv ota_spl spl.bin
setenv ota_uboot uboot.bin
setenv ota_uboot_b uboot_b.bin
setenv ota_atf atf.img
setenv ota_dtb dtb.img
setenv ota_optee optee.img
setenv ota_kernel kernel.img
setenv ota_rootfs rootfs.img
setenv ota_param param.img
setenv ota_opt opt.img
setenv ota_soc soc.omg
setenv ota_repartition enable
saveenv
axera_ota
```

## 重要说明

升级过程出现中断的情况:

(1) 不支持掉电

如果正好在升级 u-boot 分区对应的 bin 文件时未完成掉电，则无法再次启动，必须通过其它方式刷机（如 AXDL 工具升级）；

如果升级其它文件时掉电升级失败，且客户 u-boot 方案默认设计没有关闭 bootdelay，则还可以继续通过串口进入 u-boot console 重新配置 env 进行升级

## （2）升级过程中断网

升级过程中断网导致失败，不会影响 eMMC 数据的写入，如客户 u-boot 方案默认设计没有关闭 bootdelay，则还可以继续通过串口进入 u-boot console 重新配置 env 进行升级

### 🔑 注意事项：

- 请确保 XML 文件中的分区信息与 uboot 代码中的分区信息定义保持一致，否则升级后可能无法启动；
- ota 升级设置中若使能了 ota\_repartition 选项，在升级过程中会使用 xml 中的分区信息进行烧录，建议设置使能；

## 8.5.2 SD 卡升级

AX620E 参考 board 的 U-Boot 支持 SD 卡升级，设计方式如下：

软件能正常启动时，按电源键启动时，旧软件检测到 SD 卡存放升级文件，则会对板子上的软件版本进行升级。其中 SD 卡仅支持 FAT32 文件系统，支持 emmc project、nand project、nor project。

具体操作步骤如下：

**步骤1** SD 卡格式化为 FAT32 格式。

**步骤2** SD 卡根目录下拷贝升级的镜像文件，对于 emmc project 编译 sdk 包，从路径 /build/out/AX6xx\_xxx/images/sd\_update\_pack 目录下获取镜像内容。

**步骤3** 插入 SD 卡，按键重启，开始升级，升级成功后会重启。

### 🔑 说明：

升级成功后，会在 env 分区中添加升级成功标记（写 sduptate=finish 字段）防止重启后反复触发 SD 卡升级，不掉电不会消失，如果后续再进行升级，需要清除该字段或擦除 env 再升级。

Windows 10 操作系统对小于 32G 的 SD 卡支持 FAT32 格式化，但是对于大于 32G 卡默认支持 exfat 格式化，没有 FAT32 格式化选项。建议用小于 32G 的 SD 卡升级，或者网上下载工具对大于 32G 的卡进行 FAT32 格式化。

在 SD 卡升级中，SD 卡的分区个数要小于等于 2 个，若 SD 分出了大于 2 个的分区，将升级文件放到大于序号 2 的分区中，将不支持 SD 卡升级，用于升级的 SD 卡需要做成小于等于 2 个分区的。

也可通过 uboot 命令行进行手动触发 sd\_update。

#### 🔑 注意事项：

请确保 XML 文件中的分区信息与 uboot 代码中的分区信息定义保持一致，否则升级后可能无法启动。

### 8.5.3 SD 卡启动

AX620E 支持 ROM 从 SD 卡启动，可以启动到文件系统，目前仅支持 emmc project，具体操作步骤如下：

- 步骤1** 电脑插入 SD 卡（容量为 32G），将卡格式化为 FAT32 格式，格式化后，将卡插入板子，输入命令：`fdisk /dev/mmcblk1p1`，设置成两个分区，一个 FAT32 分区，一个 EXT4 分区，可去网页搜索 linux sd 卡建立分区类似博客有介绍；
- 步骤2** 拔出板端的 SD 卡，插入电脑，对于 emmc project 编译 sdk 包，从路径 `/build/out/AX6xx_xxx/images/sd_boot_pack` 目录下获取。
- 步骤3** 在 SD 卡里建一个名为 files 的文件夹，`sudo mount -t ext4 -o loop rootfs.ext4 rootfs/` 将 `rootfs.ext4` 文件解压出来，并将里面的全部内容拷贝到 SD 卡的 files 文件夹里，再将 SD 卡插入板端，输入命令，将 SD 卡分区 1 的内容挂载到 mnt 文件夹上：  
`mount /dev/mmcblk1p1 /mnt;`
- 步骤4** 进入 mnt 目录：`cd /mnt`，在 mnt 目录建立一个 rootfs 文件夹：`mkdir /mnt/rootfs`，挂载分区 2 到 rootfs 文件夹上：`mount /dev/mmcblk1p2 /rootfs`，拷贝/mnt 的所有内容到 /mnt/rootfs 文件夹中：`cp /mnt/* /mnt/rootfs;`
- 步骤5** 将板端断电，插入 sd 卡，上电进入 usb 下载，将从 SD 卡启动，板端能正常开机即

为 PASS。

## 8.5.4 SD 卡裸卡升级

AX620E 支持 ROM 从 SD 卡启动，可以启动到 uboot 之后进行 SD 升级，从而实现裸卡升级，具体操作步骤如下：

根据 5.3 和 5.4 章节，使用 `copy sd_update_pack` 中的所有 `img`，再将 `sd_boot_pack` 中的 `boot.bin` `copy` 出来，组成一个用于裸卡升级的包，放置到 sd 卡中，插入 sd 卡，上电进入 `usb` 下载，将从 SD 卡启动，启动到 `uboot` 后可以看到会进行 sd 卡升级。

 说明：

Uboot 升级之后会在 `env` 分区中设置一个 `flag`，建议测试之前进行 `flash erase all`。

## 8.5.5 U 盘升级

AX620E 参考 board 的 U-Boot 支持 U 盘升级，设计方式如下：

软件能正常启动时，按电源键启动时，读取 `env` 标记 `usbupdate` 如果是 `ready`，则会对板子上的软件版本进行升级。其中 U 盘仅支持 `FAT32` 文件系统，支持 `emmc project`、`nand project`、`nor project`。

具体操作步骤如下：

**步骤1** U 盘格式化为 `FAT32` 格式。

**步骤2** U 盘根目录下拷贝升级的镜像文件，对于 `emmc project` 编译 `sdk` 包，从路径 `/build/out/AX6xx_xxx/images/sd_update_pack` 目录下获取镜像内容。

**步骤3** 在 `uboot` 或内核 `console` 设置 `env` 配置要升级的文件。

配置 `env` 以进行 U 盘升级：

```
setenv usbupdate ready
```

**步骤4** 插入 U 盘，按键重启，开始升级，升级成功后会重启。

 说明：

Windows 10 操作系统对小于 32G 的 U 盘支持 FAT32 格式化，但是对于大于 32G 默认支持 exfat 格式化，没有 FAT32 格式化选项。建议用小于 32G 的 U 盘升级，或者网上下载工具对大于 32G 的 U 盘进行 FAT32 格式化。

也可通过 uboot 命令行进行手动触发 `usb_storage_update`。

**注意：**

请确保 XML 文件中的分区信息与 uboot 代码中的分区信息定义保持一致，否则升级后可能无法启动。

## 8.5.6 支持修改 xml 修改 partition 的大小

以 AX630C\_emmc\_arm64\_k419 为例支持的分区分布如下：

```
768K(spl), 512K(ddrinit), 256K(atf), 256K(atf_b), 1536K(uboot), 1536K(uboot_b), 1024K(env), 1024K(optee), 1024K(optee_b), 6144K(logo), 6144K(logo_b), 1024K(dtb), 1024K(dtb_b), 65536K(kernel), 65536K(kernel_b), 2097152K(rootfs), 5120K(param), 1048576K(soc), 4330752K(opt)
```

所有镜像大小是按照以上大小制作的，有些想要修改分区大小，比如想要将 `opt` 分区修改成 2048M，我们提供了一种方法不用修改代码重新编译下载包。

具体操作步骤如下：

**步骤1** 修改 5.2 章节中提到的 AX620E\_emmc\_arm64\_k419.xml 中分区的 size。

```
<Partitions strategy="1" unit="2">

  <Partition gap="0" id="spl" size="768" />
  <Partition gap="0" id="ddrinit" size="512" />
  <Partition gap="0" id="atf" size="256" />
  <Partition gap="0" id="atf_b" size="256" />
  <Partition gap="0" id="uboot" size="1536" />
  <Partition gap="0" id="uboot_b" size="1536" />
  <Partition gap="0" id="env" size="1024" />
  <Partition gap="0" id="optee" size="1024" />
  <Partition gap="0" id="optee_b" size="1024" />
  <Partition gap="0" id="logo" size="6144" />
  <Partition gap="0" id="logo_b" size="6144" />
  <Partition gap="0" id="dtb" size="1024" />
  <Partition gap="0" id="dtb_b" size="1024" />
  <Partition gap="0" id="kernel" size="65536" />
  <Partition gap="0" id="kernel_b" size="65536" />
  <Partition gap="0" id="rootfs" size="2097152" />
  <Partition gap="0" id="param" size="5120" />
  <Partition gap="0" id="soc" size="1048576" />
  <Partition gap="0" id="opt" size="0xffffffff" />
</Partitions>
```

分区大小修改

**步骤2** 将要下载的 opt.ext4 镜像进行解包；

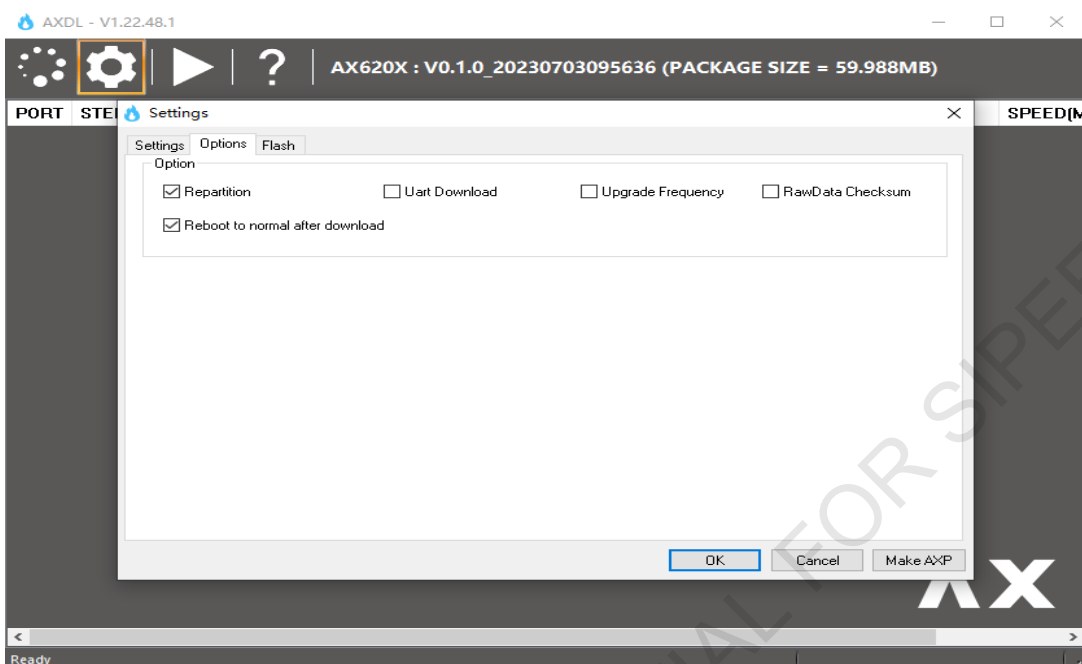
```
sudo mount -t ext4 -o loop opt.ext4 opt/
```

**步骤3** 之后再使用 tools/mkext4fs/make\_ext4fs 重新打包成 2G 大小的镜像；

```
./tools/mkext4fs/make_ext4fs -l 2048M $(BUILD_PATH) /opt.ext4 ./opt
```

**步骤4** 配置 AXDL 下载工具中 option,将 repartition 选项勾上, 然后使用 AXDL 工具下载

新打包的 opt.img;



下载工具配置

### 8.5.7 文件系统 SPARSE 压缩格式镜像

AX620E 参考 board 的 U-Boot 支持烧录 sparse 压缩格式文件系统镜像功能。目前仅支持 eMMC 分区文件系统 Sparse 镜像烧录。

**步骤1** 制作 sparse 压缩格式文件系统镜像。

当前 SDK 包代码中 rootfs/Makefile 脚本文件已通过如下 -s 命令可以制作 ext4 sparse 格式镜像：

```
@$(HOME_PATH)/tools/mkext4fs/make_ext4fs -l 256M -s
$(BUILD_PATH)/out/$(PROJECT)/rootfs_sparse.ext4
$(BUILD_PATH)/out/$(PROJECT)/rootfs
```

```

@$(HOME_PATH)/tools/mkext4fs/make_ext4fs -l 256M $(BUILD_PATH)/out/$(PROJECT)/rootfs.ext4 $(BUILD_PATH)/out/$(PROJECT)/rootfs
@$(HOME_PATH)/tools/mkext4fs/make_ext4fs -l 256M -s $(BUILD_PATH)/out/$(PROJECT)/rootfs_sparse.ext4 $(BUILD_PATH)/out/$(PROJECT)/rootfs
rm -rf $(BUILD_PATH)/out/$(PROJECT)/rootfs.tar
rm -rf $(BUILD_PATH)/out/$(PROJECT)/rootfs

@echo -e $(GREEN) "make param partition" $(DONE)
mkdir -p $(BUILD_PATH)/out/$(PROJECT)/param
@$(HOME_PATH)/tools/mkext4fs/make_ext4fs -l 5M $(BUILD_PATH)/out/$(PROJECT)/param.ext4 $(BUILD_PATH)/out/$(PROJECT)/param
@$(HOME_PATH)/tools/mkext4fs/make_ext4fs -l 5M -s $(BUILD_PATH)/out/$(PROJECT)/param_sparse.ext4 $(BUILD_PATH)/out/$(PROJECT)/param
rm -rf $(BUILD_PATH)/out/$(PROJECT)/param

@echo -e $(GREEN) "make opt partition" $(DONE)
@$(HOME_PATH)/tools/mkext4fs/make_ext4fs -l 1024M $(BUILD_PATH)/out/$(PROJECT)/opt.ext4 $(BUILD_PATH)/out/$(PROJECT)/opt
@$(HOME_PATH)/tools/mkext4fs/make_ext4fs -l 1024M -s $(BUILD_PATH)/out/$(PROJECT)/opt_sparse.ext4 $(BUILD_PATH)/out/$(PROJECT)/opt
rm -rf $(BUILD_PATH)/out/$(PROJECT)/opt

```

制作 sparse 镜像脚本命令

**步骤2** AXP 包默认打包 Raw 格式镜像，外部客户可在 AXDL 烧录工具手动选择替换 out 目录下生成的 sparse 格式同名文件启动正常烧录镜像流程即可（内部 DailyBuild 发布版本路径 img 压缩包中同时包含 sparse/raw 两种格式镜像）。

 建议：

由于 eMMC 器件具有顺序写性能优于随机写的普遍特性，而 sparse 格式下载流程会解析文件系统镜像 group 数据分段写入，因此文件系统使用冗余度越高 sparse 格式镜像可以尽量减少数据写入量，烧录性能更优；反之文件系统冗余度越低则不建议制作 sparse 格式镜像而是 raw 格式镜像尽量利用 eMMC 顺序写特性提升烧录性能。

## 8.5.8 U-boot 加密功能

AX620E 参考 board 的 SPL 和 UBOOT 都可以支持数据加密功能，防止存储在 ROM 上的 bootloader 数据被通过硬件手段获取后反汇编。目前仅支持 eMMC/Nand Flash 方案 u-boot.bin 数据加密。

1. SDK 默认未使能加密功能，AXP 默认打包由 u-boot.bin 直接进行 secureboot 签名的 u-boot\_signed.bin。
2. 使用同一 key 烧写 efuse 和编译生成 u-boot\_enc\_signed.bin。
  - a) key 文件在 boot/uboot/u-boot-2020.04/aes-256.key，长度为 256bit，默认值为 0，可以由客户修改，编译 uboot 时会根据 aes-256.key 值将 u-boot.bin 文件加密成 u-boot\_enc.bin，再由 secureboot 签名为 u-boot\_enc\_signed.bin。
  - b) 在 uboot 编译的 Makefile 中，已经添加生成加密的镜像文件了，可在 out 生成 img 中看到 u-boot\_enc\_signed.bin。默认使用 AES key 为 0000000000000000000000000000000000000000000000000000000000000000，客户可根据自己 fuse 烧入的 AES key 进行修改生成 img 是解密用的 AES key。

```

60 install:
61 * @$(ECHO) -e $(GREEN) "uboot $@" ... " $(DONE)
62 * @$(MKDIR) $(IMAGE_OUTDIR)
63 * @$(MKDIR) $(IMAGE_OUTDIR)/debug
64 * $(CP) $(KBUILD_OUTDIR)/$(UBOOT_ELF) $(IMAGE_OUTDIR)/debug/
65 * #$(CP) $(KBUILD_OUTDIR)/$(UBOOT_BIN) $(IMAGE_OUTDIR)/$(FDL2_BIN)
66 * $(CP) $(KBUILD_OUTDIR)/$(UBOOT_BIN) $(IMAGE_OUTDIR)/$(UBOOT_BIN)
67 * $(CP) $(UBOOT_DIR)/tools/logos/axera_logo.bmp $(IMAGE_OUTDIR)
68
69 ifeq ($(strip $(SIGN_USE_RSA3072)),true)
70 * @python3 $(TOOL_PATH)/sec_boot_AX620E_sign.py -i $(KBUILD_OUTDIR)/$(UBOOT_BIN) -o $(BUILD_PATH)/out/$(PROJECT)/images/$(UBOOT_SIGN_BIN) -pub $(HOME_PATH)/tools/imsign/key_3072/pubkey
71 * #$(CP) $(BUILD_PATH)/out/$(PROJECT)/images/$(FDL2_SIGN_BIN) $(BUILD_PATH)/out/$(PROJECT)/images/$(UBOOT_SIGN_BIN)
72 else
73 * @python3 $(TOOL_PATH)/sec_boot_AX620E_sign.py -i $(KBUILD_OUTDIR)/$(UBOOT_BIN) -o $(BUILD_PATH)/out/$(PROJECT)/images/$(UBOOT_SIGN_BIN) -pub $(TOOL_PATH)/public.pem -prv $(TOOL_PATH)/
74 * #$(CP) $(BUILD_PATH)/out/$(PROJECT)/images/$(UBOOT_SIGN_BIN) $(BUILD_PATH)/out/$(PROJECT)/images/$(FDL2_SIGN_BIN)
75 endif

```

### 生成加密镜像

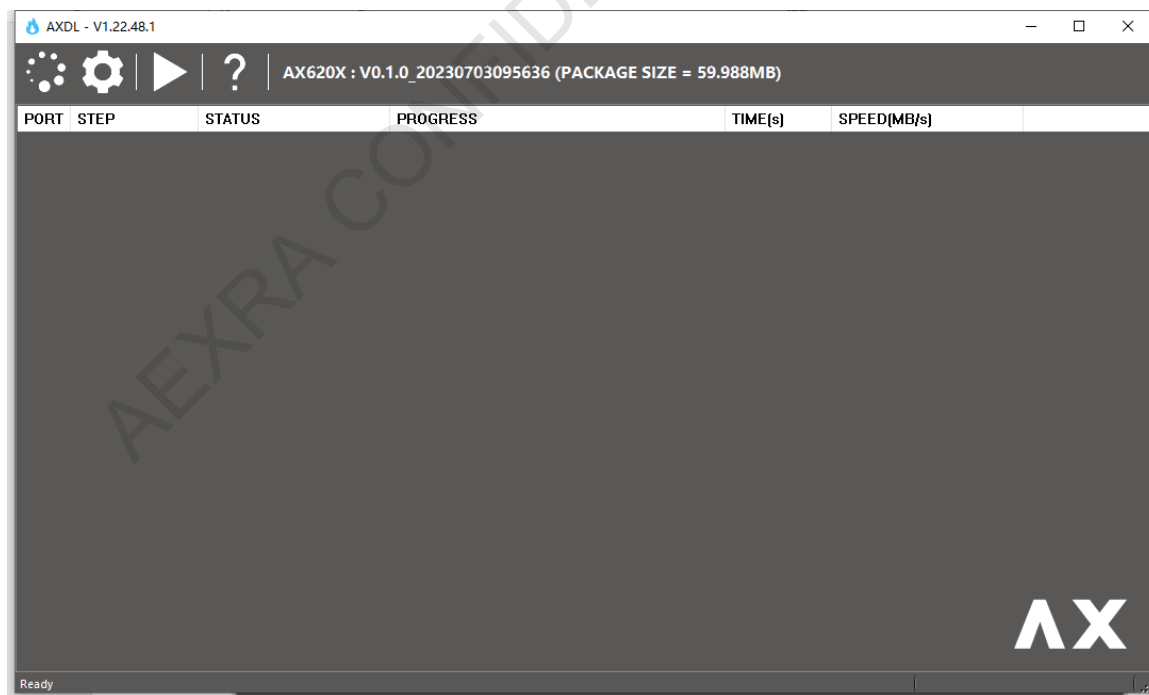
- c) 使用与 a) 相同的 aes-256.key 烧写 efuse block 22、23、24、25、26、27、28、29，efuse driver 可参考调用 boot\uboot\uboot-2020.04\arch\arm\mach-axera\AX620E\efuse\_drv.c 文件的接口：int ax\_efuse\_write(int blk, int data);

## 8.5.9 回读分区数据

AX620E 参考 board 的 U-Boot 支持回读分区数据功能，目前仅支持 eMMC/NAND 分区数据回读。

具体步骤如下：

AXDL 烧录工具加载 AXP 包。

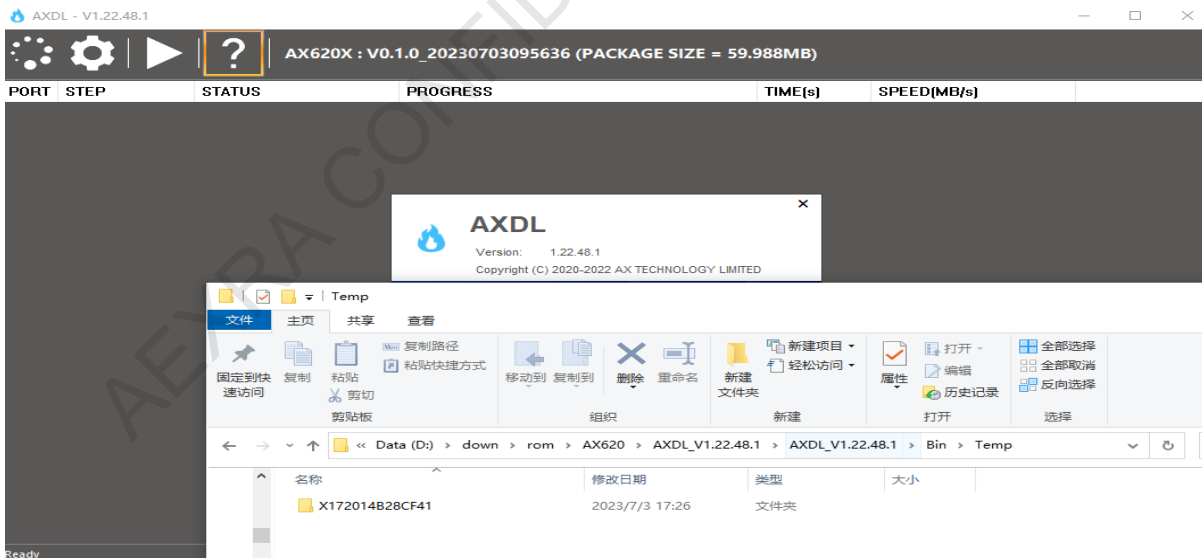


AXDL 烧录工具截图

**步骤1** 左键单击“问号”选项，在中间弹出窗口界面单击右键，弹出 image 镜像临时路径。



查询 XML 分区表路径



打开 XML 分区表路径

**步骤2** 在···\AXDL\_Vx.xx.xx.x\Bin\Temp\xxxxxxx 路径下打开查看 xxx.XML 文件。

```
<!-- Unit: 0, 1M Byte; 1, 512K Byte; 2, 1K Byte; 3, 1 Byte; 4, 1Sector -->
<Partitions strategy="1" unit="2">
  <!-- The env partition size cannot be manually changed. -->
  <Partition gap="0" id="spl" size="768"/>
  <Partition gap="0" id="ddrinit" size="512"/>
  <Partition gap="0" id="atf" size="256"/>
  <Partition gap="0" id="atf_b" size="256"/>
  <Partition gap="0" id="uboot" size="1536"/>
  <Partition gap="0" id="uboot_b" size="1536"/>
  <Partition gap="0" id="env" size="1024"/>
  <Partition gap="0" id="optee" size="1024"/>
  <Partition gap="0" id="param" size="5120"/>
  <Partition gap="0" id="logo" size="6144"/>
  <Partition gap="0" id="dtb" size="1024"/>
  <Partition gap="0" id="kernel" size="65536"/>
  <Partition gap="0" id="rootfs" size="2097152"/>
  <Partition gap="0" id="soc" size="1048576"/>
  <Partition gap="0" id="opt" size="0xffffffff"/>
</Partitions>
```

#### XML 分区表内容

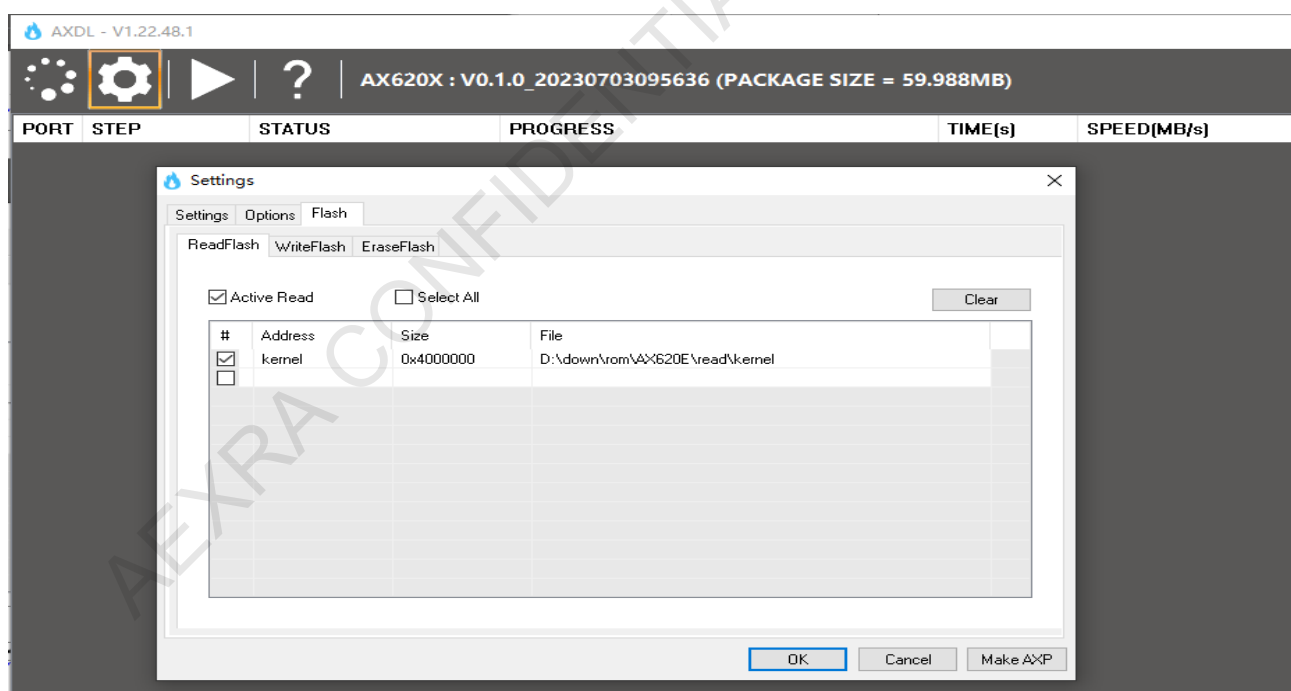
文件内容表示当前 AXP 版本对应的分区表信息，id="XXX"表示分区名，size="XXX"表示分区容量，单位默认为 Mbytes。

**步骤3** AXDL 烧录工具单击“Setting”，选择“Flash > Read Flash”选项，勾选“Active Read”，在窗口中根据 XML 内容信息填写，说明如下：

- (1) Address 列填写 XML 中 id 项对应的内容即分区名，如 id="kernel"对应的 Address 列填写 kernel。
- (2) Size 列填写 XML 中 size 项对应的内容单位为 Bytes，如 kernel 分区 size="65536"，分区大小为  $64\text{M} = 65536 \times 1024 = 67108864 = 0x4000000$ 。
- (3) File 列填写期望将回读的每个对应分区数据保存在 PC 的具体文件路径及文件名。



AXDL 烧录工具设置



配置回读分区

**步骤4** 单击“OK”，开始正常烧录版本，在下载完所有 image 后会继续回读分区镜像数据到步骤3中设定的 PC 路径保存分区数据文件。

## 9 网络安全注意事项

### 9.1 Uboot 使用注意事项

#### 9.1.1 串口

串口属于非常通用设备通信协议，用于设备的端到端调试，AX620E SDK 中的 uboot 缺省打开了串口功能，开发人员可以在 uboot 启动阶段通过按任意键方式中断 uboot 的执行过程，进入 uboot 的命令行，执行调试功能（需要将 CONFIG\_BOOTDELAY = 0 改 2，正式产品中需要将 CONFIG\_BOOTDELAY 设置为 0，否则影响开机时间）。产品正式发布，客户可采用以下措施对串口做出限制：

- 在正式生产的产品中，硬件上将串口删除。
- 可以删除 uboot 中 uart 初始化的代码，达到关闭串口目的。

具体方法如下：

1. 修改 boot/uboot/u-boot-2020.04/common/board\_f.c 文件，将 serial\_init 注释掉：

```
/boot/uboot/u-boot-2020.04/common$ vi board_f.c
```

```
serial_init,
```

2. 修改 boot/uboot/u-boot-2020.04/common/stdio.c 文件，将函数注空：

```
static void stdio_serial_putc(struct stdio_dev *dev, const char c)
{
    serial_putc(c);
}

static void stdio_serial_puts(struct stdio_dev *dev, const char *s)
{
    serial_puts(s);
}

static int stdio_serial_getc(struct stdio_dev *dev)
{
    return serial_getc();
}

static int stdio_serial_tstc(struct stdio_dev *dev)
{
    return serial_tstc();
}
```

### 9.1.2 命令行

Uboot 下的命令行有许多供开发人员和调试使用的命令，在正式产品中是不需要用到的。

用户只保留产品必需的命令：bootm、go、sf（nand/mmc/ufs）、setenv、saveenv。

```
bootm    - boot application image from memory
go       - start application at address 'addr'
sf       - SPI flash sub-system
setenv   - set environment variables
saveenv  - save environment variables to persistent storage
```

其他命令可采用修改 uboot 的代码，去掉 uboot 下的开发和使用的命令，修改方式如下：

**步骤1** 在 uboot 源代码中，根据工具名称找到对应的源代码文件，然后在 Makefile 中删除，对工具源码文件进行编译的语句，下面以 setenv 工具为例进行详细说明。

**步骤2** 在目录 boot/uboot/u-boot-2020.04/cmd 中执行：grep setenv，查找到对应的.c 文件，在 boot/uboot/u-boot-2020.04/cmd/Makefile 文件中把 nvedit.o 注释掉即可：

```
# core command
obj-y += nvedit.o
```

## 9.2 Linux 及 Busybox 使用安全

### 9.2.1 Root 账户

AX620E SDK 中的 Busybox 缺省只有 root 用户，没有设置密码，供开发人员开发调试用。

**！ 特别提醒：**

客户在自己的产品中请一定要自己设置 root 账户密码！

#### 设置 root 账户密码

**步骤1** 在/etc/inittab 文件中，内容如下：

添加 “::respawn:-/bin/login”，并将 “console::respawn:-/bin/sh” 行注释掉。

**步骤2** 在/etc/目录下创建一个 passwd 文件和 shadow 文件，passwd 文件中内容为：

```
root:x:0:0:root:/:/bin/sh
```

第一段表示登录用户名。

第二段如果是 x 表示密码在 shadow 中，为密文，不可见。

第三段表示当前用户的 UID 等级，一般 root 超级用户的 UID 为 0，表示等级最高，最好将其他用户的 UID 等级设置为 0。

第五段表示登录名的全名，不是很重要，可以表示该用户的地址等。第六段表示默认登录后

所在目录，第七段表示登录后执行的脚本。

**步骤3** shadow 文件中内容为：

```
root:Wdfdfqwe:17226:::
```

第一段表示登录的用户名

第二段表示密文密码。

**步骤4** 设置用户密码：

```
passwd root
```

之后输入该用户的密码，之后在 shadow 中就会生成密文密码，用户是无法知道具体含义的，生成密文密码后，可以将 shadow 文件中的密文密码放入到制作根文件系统的文件夹中，然后烧录以便不用每次都设置密码。

## 禁止 shell 登录

要在系统启动后，禁止 shell 登陆，需要执行以下操作：

**步骤1** 修改/etc/passwd 配置文件。

将/etc/passwd 文件中的内容：

root:x:0:0:root:/root:/bin/sh 修改为：root:x:0:0:root:/root:/bin/false

**步骤2** 修改/etc/shadow 配置文件。

将/etc/shadow 配置文件中的内容：

root:yf/wq7vpRPGxE:0:0:99999:7:::修改为 root!:10000:0:99999:7:::

**步骤3** 修改/etc/inittab 配置文件。

将/etc/inittab 文件中，以下内容删除掉：

::respawn:/sbin/getty -L ttyS000 115200 vt100 -I "Auto login as I..."

## 9.2.2 运行权限

建议用户的应用程序以非 root 方式运行。使用非 root 方式运行程序需要执行以下操作：

增加 filecap 工具，支持普通用户的特权操作。

**步骤1** 源码包

源码包使用最新的 [https://launchpad.net/ubuntu/+source/libcap-ng/libcap-ng\\_\\*\\*\\*.orig.tar.gz](https://launchpad.net/ubuntu/+source/libcap-ng/libcap-ng_***.orig.tar.gz)

```
tar -zxvf libcap-ng_***.orig.tar.gz
```

**步骤2** 交叉编译

```
cd libcap-ng-0.7.9
mkdir out
./configure --prefix=./out/ --host=aarch64-linux
make & make install
```

**步骤3** 完成后，将 out/bin/下的可执行文件拷贝到 rootfs/bin 目录下，将 out/lib/下的文件拷贝到 rootfs/lib 目录下。

**步骤4** 下面以非 root 用户访问/dev/mem 设备为例，说明如何为非 root 用户添加特权属

性:

1. 以 root 身份用登录系统。
2. 添加一个新用户 temp:

```
~ # adduser temp
Changing password for temp
New password: *****
Retype password: *****
Password for temp changed by root
```

3. 切 root 用户，用 filecap /bin/ax\_lookat 命令添加 sys\_rawio 和 sys\_admin 限制。

```
$ su root
# filecap /bin/ax_lookat sys_rawio sys_admin
```

4. 再次运行:

```
# filecap /bin/ax_lookat
```

### 9.2.3 SSH 服务

SSH 是 Secure Shell Protocol 的简写，由 IETF 网络工作小组（Network Working Group）制定。在进行数据传输之前，SSH 先对联机数据包通过加密技术进行加密处理，加密后在进行数据传输。确保了传递的数据安全。

以 AX620E SDK 为例:

板端已经移植了 sshd，并且默认开启，默认使用的端口是 22，用户名 root，密码是 123456。如果需要关掉 SSH 服务请做如下操作:

```
# rm /etc/init.d/S50sshd
```

### 9.2.4 SCP 服务

Linux SCP 命令用于 Linux 之间复制文件和目录，SCP 是 Secure Copy 的缩写，SCP 是 Linux 系统下基于 SSH 登陆进行安全的远程文件拷贝命令。SCP 是加密的，所以在使用 SCP 之前需要

在板端通过 `passwd` 命令设置板端的密码。

以 AX620E SDK 为例：

如果需要关掉 SCP 功能，需要禁用 `sshd` 服务。

**！ 特别提醒：**

AX620Q 由于采用 NOR 存储空间有限，所以 SSH 相关功能服务已禁用，只支持 Telnet 服务

## 9.2.5 SFTP 服务

`sftp-server` 是 `sshd` 的子系统，不能单独运行。默认已经使能 `sftp-server` 服务，如果需要关闭该服务，请做如下修改：

打开 `/etc/ssh/sshd_config`，删除下行：

```
Subsystem      sftp          /usr/bin/sftp-server
```

## 9.2.6 Telnet 服务

Telnet 协议是 TCP/IP 协议族中的一员，是 Internet 远程登录服务的标准协议和主要方式。它为用户提供了在本地计算机上完成远程主机工作的能力。在终端使用者的电脑上使用 `telnet` 程序，用它连接到服务器。终端使用者可以在 `telnet` 程序中输入命令，这些命令会在服务器上运行，就像直接在服务器的控制台上输入一样。可以在本地就能控制服务器。要开始一个 `telnet` 会话，必须输入用户名和密码来登录服务器。Telnet 是常用的远程控制 Web 服务器的方法。

以 AX620E SDK 为例：

板端已经移植了 `telnetd` 和 `telnet`，并且默认开启 `telnetd`，端口是 23，用户名是 `root`。如果需要关掉 `telnetd`，请做如下修改：

直接删掉 `/etc/init.d/S50telnet`。

## 9.2.7 文件权限

建议客户对 `rootfs` 中的文件和设备权限按照最小权限的原则进行控制。

## 9.3 SDK 驱动使用网络安全注意事项

### 9.3.1 串口

在用户的产品中需通过关闭串口来防止串口非法接入，导致信息泄漏的风险。

AX620E 可以在各个 build/projects 各个工程目录下的 partition.mak 里 KERNEL\_BOOTARGS 修改相应产品的 BOOTARGS 参数 console=null，来关闭内核串口输出。

### 9.3.2 USB

AX620E 支持 1 个 USB2.0 端口，可以做 Host 和 Device 功能，其中 USB2 Device 默认使能 USB debug 串口功能，登录名 root。如果关闭 USB debug 功能，请做如下修改：

打开 rootfs/etc/inittab，删除下行：

```
::respawn:/sbin/getty -L -t 600000 ttyGS0 115200 vt100 # GENERIC_SERIAL
```

打开 rootfs/etc/init.d/rcS，删除下行：

```
/opt/scripts/usb-acm.sh usb2
```

### 9.3.3 Linux 使用安全注意事项

只需要遵守 GPL 协议就可以。

## 9.4 其他注意事项

### 9.4.1 裸片烧写

AX620E SDK 支持 USB 裸片烧写功能。客户在实际产品中要关掉 USB 裸片升级，在产线可以用夹具升级和 SD 卡升级用户版本。

### 9.4.2 SD 卡 mount 权限

用户开发带可插拔存储介质（如 SD 卡，U 盘）产品形态时，请确保 mount 存储设备文件系统前，建议加上 `mount -o noexec /dev/mmcblk1 /mnt/sdcard`，以避免第三方程序运行。

### 9.4.3 JTAG

攻击者可以通过 JTAG 接口，篡改系统的任意配置，破坏系统。建议用户在正式产品中将 JTAG 从物理上删除。

# 10 Security 开发指南

## 10.1 Security 方案内容

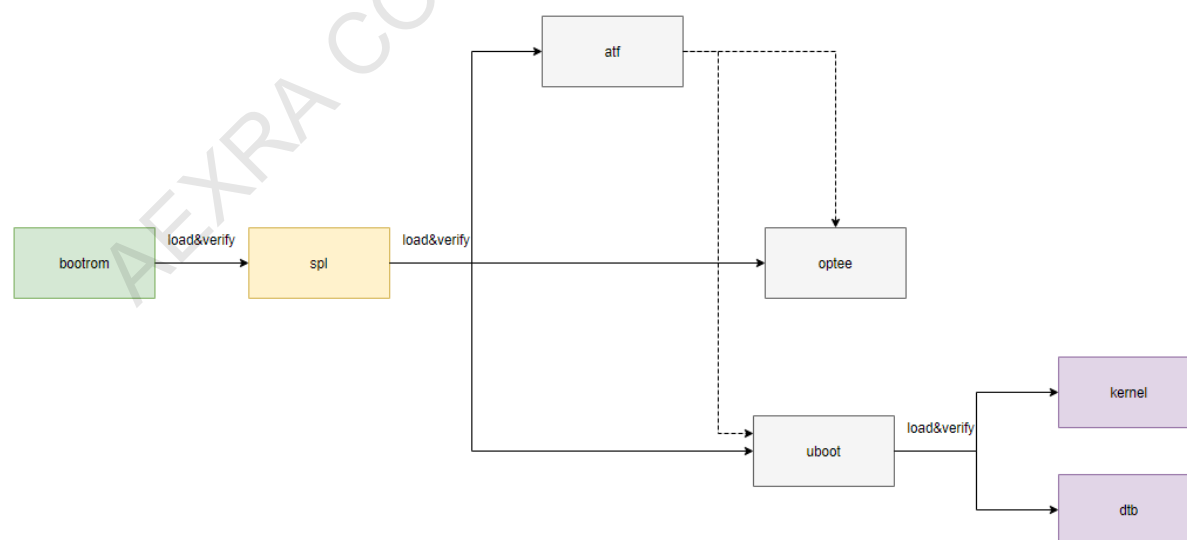
本文档主要介绍基于 AX620E 平台构建安全可信环境，Security 方案主要涉及如下 3 个主要内容：

- Secure boot;
- Optee 使用说明（运行时安全机制）；
- 平台安全配置。

## 10.2 Secure Boot

### 10.2.1 Secure boot 概述

AX620E 支持安全启动功能，防止非法篡改启动镜像，当使能 secure boot 功能时，会启动对启动镜像的校验，以 AX630C\_emmc\_arm64\_k419 为例，其 secure boot 启动流程如下：



启动流程

如上图，AX630C\_emmc\_arm64\_k419 在开启 secure boot 时，romcode 读取 spl 镜像到 OCM（内部的 SRAM）中校验并运行，而 spl 获得执行权后，在启动流程中需要初始化 ddr，然后读取 atf、optee（可选）、uboot 镜像到对应的加载地址并对镜像做校验，spl 接下来将执行权交给 atf，由 atf 完成 optee（可选）的启动引导后再将执行权交给 uboot，在 uboot 中完成 kernel 和 dtb 镜像的加载和校验，最后完成 linux 启动引导后，由 linux 接管系统。

以上述启动流程为例，secure boot 方案可分为 3 个阶段，第一阶段由 romcode 完成 spl 镜像的加载及校验，第二阶段是 spl 加载 atf、optee、uboot 并做校验，第三阶段由 uboot 完成 kernel、dtb 镜像的加载和校验。

**注意：**

SDK 中不同 board 启动要求稍有不同，例如有部分 board 启动过程中不使用 atf/optee/uboot，直接由 spl 引导 kernel 和 dtb 镜像，请以实际启动流程为准，对启动镜像的打包和校验方法和原理是一致的。

## 10.2.2 bootrom 加载校验 spl 镜像

### bootrom 阶段 secure boot 介绍

AX620E 芯片设计将 efuse bit map 的 blk15-blk22 这 8 个 blk 用于熔入 PK\_HASH（**注意：efuse 只能写入一次**），一旦熔入 PK\_HASH 之后，再将 blk13 的 bit26 写 1 就会使能 secure boot，即后续启动 bootrom 就会校验 spl 镜像的签名。

目前支持 RSA3072 和 RSA2048 两种长度的密钥进行对镜像进行签名，将密钥信息、签名信息，以及镜像的其他信息封装成一个 1K 大小的 header，header 信息如下：

```
struct img_header {
    u32 check_sum;//0x00
    u32 magic_data;//0x04, should be 0x55543322
    u32 capability;
    u32 img_size;
    u32 fw_size;
    u32 img_check_sum;
    u32 fw_check_sum;
    /*
```

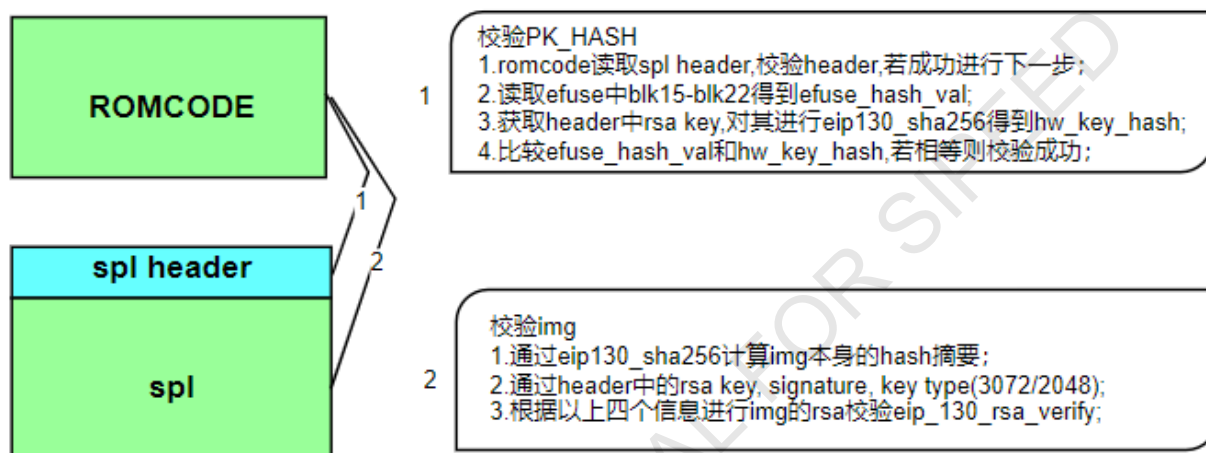
```

    * [1:0]: nand page_size, page_size = 2^(nand_nor_cfg[1:0] + 11), 2KB ~
16KB
    * [4:2]: nand block_size, block_size = 2^(nand_nor_cfg[4:2] + 16), 64KB ~
8MB
    * [6:5]: nand oob_size, oob_size = 2^(nand_nor_cfg[6:5] + 6), 64Bytes ~
512Bytes
    * [7]: nand bad block marker size, if nand_nor_cfg[7] is 0, the bad block
marker
    * is 1 byte, if nand_type[7] is 1, bad block marker is 2 bytes.
    * [15: 8]: nor QE read status register cmd
    * [23:16]: nor QE write status register cmd
    * [31:24]: nor QE bit offset
    */
u32 nand_nor_cfg;
u32 boot_bak_flash_addr;
u32 fw_flash_addr;
u32 fw_bak_flash_addr;
u32 key_n_header;
u32 rsa_key_n[96];
u32 key_e_header;
u32 rsa_key_e;
u32 sig_header;
u32 signature[96];
u32 nand_ext_cfg;
u32 nand_rx_sample_delay[3];
u32 nor_rx_sample_delay[3];
u32 ocm_start_addr;
u32 riscv_flash_addr;
u32 riscv_check_sum;
u32 riscv_img_size;
u32 nand_phy_setting[3];
u32 nor_phy_setting[3];
u32 emmc_phy_legacy_delayline;
u32 emmc_phy_hs_delayline;
u32 sd_phy_default_delayline;
u32 aes_key[12];
u32 reserved[17];
};

```

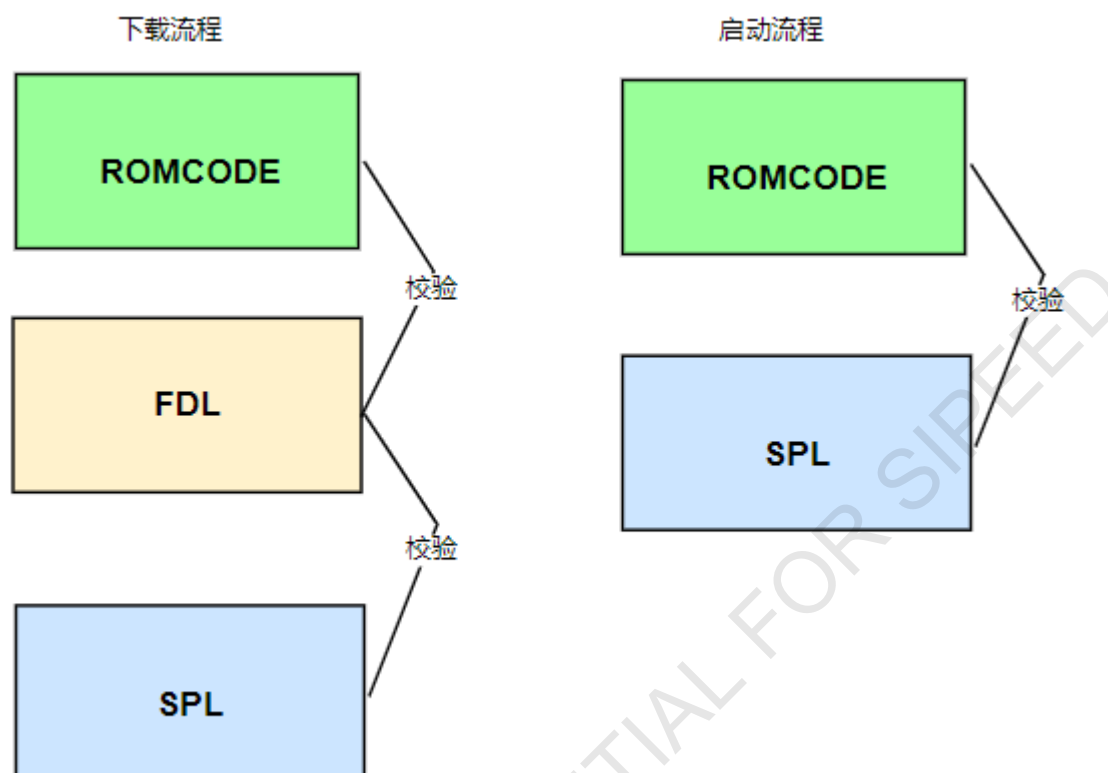
Secure boot verify 的信息主要分为两个部分：

1. 校验熔进 efuse 中的 PK\_HASH;
2. 校验 image，如下图：



Secure boot 图解

在整个系统的校验签名的流程中，AX620E 分别对下载和启动都做了校验，如下图校验流程：



安全校验流程图

## Secure boot 方案实施

### 1. 准备一对 RSA 密钥对

-- 生成 RSA 私钥（传统格式的）

```
openssl genrsa -out private_key.pem 2048(3072)
```

-- 将传统格式的私钥转换成 PKCS#8 格式的（JAVA 需要使用的私钥需要经过 PKCS#8 编码，PHP 程序不需要，可以直接略过）

```
openssl pkcs8 -topk8 -inform PEM -in private_key.pem -outform PEM -nocrypt
```

-- 生成 RSA 公钥 (php 和 java 都用转换前私钥生成公钥)

```
openssl rsa -in private_key.pem -RSAPublicKey_out -out public.pem
```

替换 {project}\tools\imgsign 目录下的公私密钥。

## 2. 对生成的镜像签名

以 demo 板为例，需要签名的镜像由 fdl2\_{project}.bin 生成 fdl2\_{project}\_signed.bin，spl\_{project}.bin 生成 spl\_{project}\_signed.bin。

签名脚本存放在 {project}\build\tools\imgsign 下，有四个相关的脚本，fdl\_AX620E\_sign.py，fdl\_AX620E\_sign\_3072.py，spl\_AX620E\_sign.py，spl\_AX620E\_sign\_3072.py。以 RSA2048 的密钥为例：

签名 fdl.bin:

```
@python3 @python3 $(IMGSIGN_SCRIPT)/fdl_AX620E_sign.py -i
$(BUILD_PATH)/out/$(PROJECT)/images/$(FDL_BIN) -o
$(BUILD_PATH)/out/$(PROJECT)/images/$(FDL_SING_BIN) -pub
$(HOME_PATH)/tools/imgsign/public.pem -prv
$(HOME_PATH)/tools/imgsign/private.pem -fw
$(HOME_PATH)/tools/imgsign/AX620E/eip_ax620e.bin -packsize 0x16800 -cap
0x47afe
```

签名 spl.bin:

```
@python3 $(IMGSIGN_SCRIPT)/spl_AX620E_sign.py -i
$(BUILD_PATH)/out/$(PROJECT)/images/$(SPL_BIN) -o
$(BUILD_PATH)/out/$(PROJECT)/images/$(SPL_SING_BIN) -pub
$(HOME_PATH)/tools/imgsign/public.pem -prv
$(HOME_PATH)/tools/imgsign/private.pem -fw $(IMGSIGN_SCRIPT)/eip_ax620e.bin -
cap 0x54FAFE
```

具体实现可见对应工程代码中的 Makefile。

## 3. 生成 PK\_HASH，烧录 PK\_HASH 到 efuse

获得 RSA key 指令如下：

```
openssl rsa -modulus -in private.pem -text
```

```
-engine val          Use engine, possibly a hardware device
wuqianlong@axcnsbussrv03p:~/mc20_1026/tools/imgsign$ openssl rsa -modulus -in private.pem -text
RSA Private-Key: (2048 bit, 2 primes)
modulus:
 00:90:e1:31:81:d2:22:72:c3:13:63:b4:04:54:a4:
 f5:45:95:e6:36:86:d7:27:77:c8:18:68:b9:09:59:
 a9:fa:4a:9a:eb:3b:8b:dc:2c:7c:cd:1d:6d:be:0e:
 5e:ae:ff:4f:9f:f0:40:90:e1:31:81:d2:22:72:c3:
 13:63:b4:04:54:a4:f5:45:95:e6:36:86:d7:27:77:
 c8:18:68:b9:09:59:a9:fa:4a:9a:eb:3b:8b:dc:2c:
 7c:cd:1d:6d:be:0e:5e:ae:ff:4f:9f:f0:40:90:e1:
 31:81:d2:22:72:c3:13:63:b4:04:54:a4:f5:45:95:
 e6:36:86:d7:27:77:c8:1b:d9:38:e8:98:47:f7:a7:
 57:06:b6:66:15:c5:75:24:d4:84:33:e3:93:42:f2:
 a2:52:01:b1:61:10:c0:70:1f:cf:7f:2e:de:8e:3d:
 ed:9d:4c:fc:ac:5c:0b:bb:6b:1a:ca:7a:29:d9:89:
 38:e8:98:47:f7:a7:57:06:b6:66:15:c5:75:24:d4:
 84:33:e3:93:42:f2:a2:52:01:b1:61:10:c0:70:1f:
 cf:7f:2e:de:8e:3d:ed:9d:4c:fc:ac:5c:0b:bb:6b:
 1a:ca:7a:29:d9:89:38:e8:98:47:f7:a7:57:06:b6:
 66:15:c5:75:24:d4:84:33:e3:93:42:f2:a2:52:06:
 be:2b
publicExponent: 65537 (0x10001)
privateExponent:
 06:8c:47:02:89:0e:c9:85:0b:91:4c:07:8e:13:ce:
 8a:10:96:51:0c:93:18:d3:8f:15:9b:56:11:98:1d:
 d8:94:1a:a0:5b:16:9d:22:dd:99:1f:a5:60:1b:a2:
```

### 生成 rsa key 示例

将以上数据转换成数组，代码中会将 key 大小端转换下，参考 {project}/msp/sample/efuse\_hash/efuse\_hash\_tool.c 中实现。

最后生成签名的 bin 类似如下：

```

00000030 2b be 06 52 a2 f2 42 93 e3 33 84 d4 24 75 c5 15 +?RIIB擗3員
00000040 66 b6 06 57 a7 f7 47 98 e8 38 89 d9 29 7a ca 1a f?W G.?±
00000050 6b bb 0b 5c ac fc 4c 9d ed 3d 8e de 2e 7f cf 1f k?\ L.?月
00000060 70 c0 10 61 b1 01 52 a2 f2 42 93 e3 33 84 d4 24 p?a?RIIB擗
00000070 75 c5 15 66 b6 06 57 a7 f7 47 98 e8 38 89 d9 29 u?f?W G.
00000080 7a ca 1a 6b bb 0b 5c ac fc 4c 9d ed 3d 8e de 2e z?k?\ L.
00000090 7f cf 1f 70 c0 10 61 b1 01 52 a2 f2 42 93 e3 33 .?p?a?RIIB
000000a0 84 d4 24 75 c5 15 66 b6 06 57 a7 f7 47 98 e8 38 昴$u?f?W
000000b0 d9 1b c8 77 27 d7 86 36 e6 95 45 f5 a4 54 04 b4 ?萎'譚6鐮
000000c0 63 13 c3 72 22 d2 81 31 e1 90 40 f0 9f 4f ff ae c.胷"?1?@
000000d0 5e 0e be 6d 1d cd 7c 2c dc 8b 3b eb 9a 4a fa a9 ^.緯.皇,車
000000e0 59 09 b9 68 18 c8 77 27 d7 86 36 e6 95 45 f5 a4 Y.筋.萎'諳
000000f0 54 04 b4 63 13 c3 72 22 d2 81 31 e1 90 40 f0 9f T.礪.胷"?
00000100 4f ff ae 5e 0e be 6d 1d cd 7c 2c dc 8b 3b eb 9a O 嘗.緯.
00000110 4a fa a9 59 09 b9 68 18 c8 77 27 d7 86 36 e6 95 J Y.筋.諳
00000120 45 f5 a4 54 04 b4 63 13 c3 72 22 d2 81 31 e1 90 E酪T.礪.諳
00000130 20 00 01 02 01 00 01 00 00 00 00 00 00 00 00 .....
00000140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

签名后生成镜像的 rsa key

#### 说明：

- {project}/msp/sample/efuse\_hash/efuse\_hash\_tool.c 中有 RSA2048 和 RSA3072 计算 public key 的 hash 值的 sample code, efuse 只能写入一次, 因此 sample 中真正写 efuse 的代码注释掉并未真正打开;
- 对于 efuse 的写入需要考虑系统是否有使能 secure 模式 (efuse blk13 bit15), 在使能 secure 模式时只能 CPU 在 secure mode 才能写 efuse, 而在未使能 secure 模式时可以从 linux 下写 efuse;
- 在未使能 secure 模式时, 写入 efuse 的函数调用 writeHashToEfuse(digest, 32)注释掉未编译, 如果需要写 efuse 值, 需要根据实际的 key 值填写好 key\_n\_2048 或者 key\_n\_3072 数组, 然后代码中修改 writeHashToEfuse(digestXXX, 32), 重新编译后, 再运行;
- 在使能 secure 模式时, 只能通过 CPU 切到 secure 模式下进行操作, 在使用 optee 的 board 上可以参考 {project}/msp/sample/optee/optee\_efuse\_hash, 通过 CA/TA 来写入 efuse; 如果没有使用 optee, 只能在 spl 或者 fdl 中修改 code, 调用相关 efuse 写接口进行写入, 可参考 {project}/boot/bl1/spl/spl\_main.c 中函数 sample\_efuse\_hash, 修改数组 pub\_key\_hash 值。

### 10.2.3 spl 加载校验 uboot/atf/optee 镜像

#### spl 阶段 secure boot 介绍

spl 启动阶段，读取 atf、optee（可选）、uboot 镜像到对应的加载地址并对镜像做校验，之后 spl 将执行权限交给 atf，由 atf 完成 optee 的启动引导并最终跳转到 uboot 执行。

目前支持 RSA3072 和 RSA2048 两种长度的密钥进行对 atf、optee、uboot 镜像进行签名，将密钥信息、签名信息，以及镜像的其他信息封装成一个 1K 大小的 header，header 信息如下：

```
struct img_header {
    u32 check_sum;//0x00
    u32 magic_data;//0x04, should be 0x55543322
    u32 capability;
    u32 img_size;
    u32 fw_size;
    u32 img_check_sum;
    u32 fw_check_sum;
    /*
     * [1:0]: nand page_size, page_size = 2^(nand_nor_cfg[1:0] + 11), 2KB ~
16KB
     * [4:2]: nand block_size, block_size = 2^(nand_nor_cfg[4:2] + 16), 64KB ~
8MB
     * [6:5]: nand oob_size, oob_size = 2^(nand_nor_cfg[6:5] + 6), 64Bytes ~
512Bytes
     * [7]: nand bad block marker size, if nand_nor_cfg[7] is 0, the bad block
marker
     * is 1 byte, if nand_type[7] is 1, bad block marker is 2 bytes.
     * [15: 8]: nor QE read status register cmd
     * [23:16]: nor QE write status register cmd
     * [31:24]: nor QE bit offset
     */
    u32 nand_nor_cfg;
    u32 boot_bak_flash_addr;
    u32 fw_flash_addr;
    u32 fw_bak_flash_addr;
    u32 key_n_header;
    u32 rsa_key_n[96];
}
```

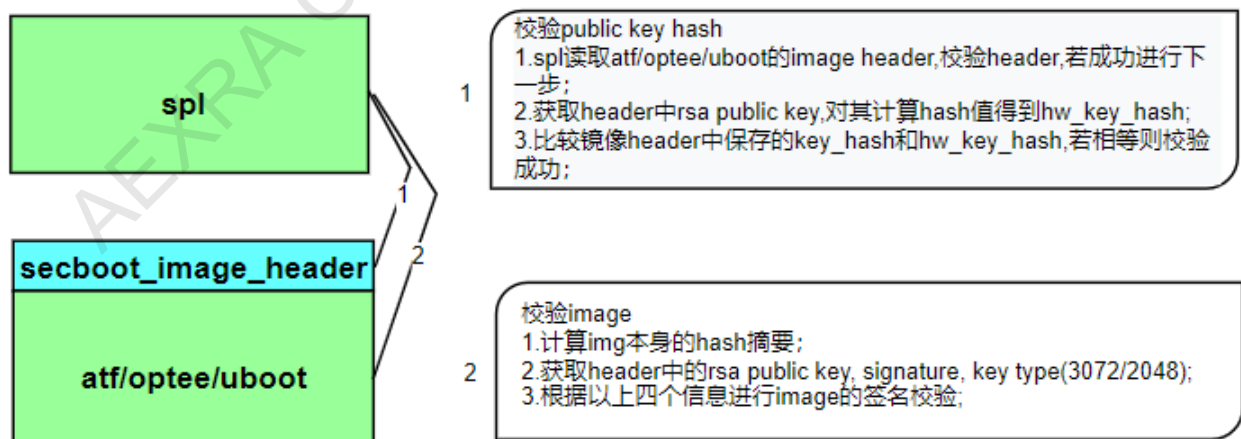
```

u32 key_e_header;
u32 rsa_key_e;
u32 sig_header;
u32 signature[96];
u32 nand_ext_cfg;
u32 nand_rx_sample_delay[3];
u32 nor_rx_sample_delay[3];
u32 ocm_start_addr;
u32 riscv_flash_addr;
u32 riscv_check_sum;
u32 riscv_img_size;
u32 nand_phy_setting[3];
u32 nor_phy_setting[3];
u32 emmc_phy_legacy_delayline;
u32 emmc_phy_hs_delayline;
u32 sd_phy_default_delayline;
u32 aes_key[12];
u32 reserved[17];
};

```

Secure boot verify 的信息主要分为两个部分：

1. 校验镜像中的 PK\_HASH;
2. 校验 image，如下图：



Secure boot 图解

第二阶段的校验原理与第一阶段的校验是一致的。

## Secure boot 方案实施

1. 准备一对 RSA 密钥对。

同 10.2.2 Secure boot 方案实施章节，使用上述共用公钥和私钥文件。

2. 对生成的镜像签名。

以 demo 板为例，SDK 中签名的镜像有 uboot\_signed.bin、atf\_bl31\_signed.bin、optee\_signed.bin。

签名脚本存放在 {project}\build\tools\imgsign\sec\_boot\_AX620E\_sign.py，以 RSA2048 的密钥为例：

签名 atf 镜像：

```
@python3 $(IMGSIGN_SCRIPT)/sec_boot_AX620E_sign.py -i  
$(BUILD_BASE)/$(ATF_PLAT)/$(ATF_RELEASE_PATH)/bl31_axgzip.bin -pub  
$(TOOLS_PATH)/imgsign/public.pem -prv $(TOOLS_PATH)/imgsign/private.pem -o  
$(IMAGE_OUTDIR)/$(ATF_BL31_SIGNED_BIN) -cap 0x54FAFE -key_bit 2048
```

具体实现可见对应工程代码中的 Makefile。

## 10.2.4 uboot 加载校验 kernel/dtb 镜像

### uboot 阶段 secure boot 介绍

uboot 启动阶段，读取 kernel 和 dtb 镜像到对应的加载地址并对镜像做校验，uboot 之后将执行权限交给 linux kernel，完成 linux 的启动引导。

目前支持 RSA3072 和 RSA2048 两种长度的密钥进行对 kernel、dtb 镜像进行签名，将密钥信息、签名信息，以及镜像的其他信息封装成一个 1K 大小的 header，header 信息如下：

```
struct img_header {  
    u32 check_sum;           //0x00  
    u32 magic_data;          //0x04, should be 0x55543322  
    u32 capability;  
    u32 img_size;  
    u32 reserved0;
```

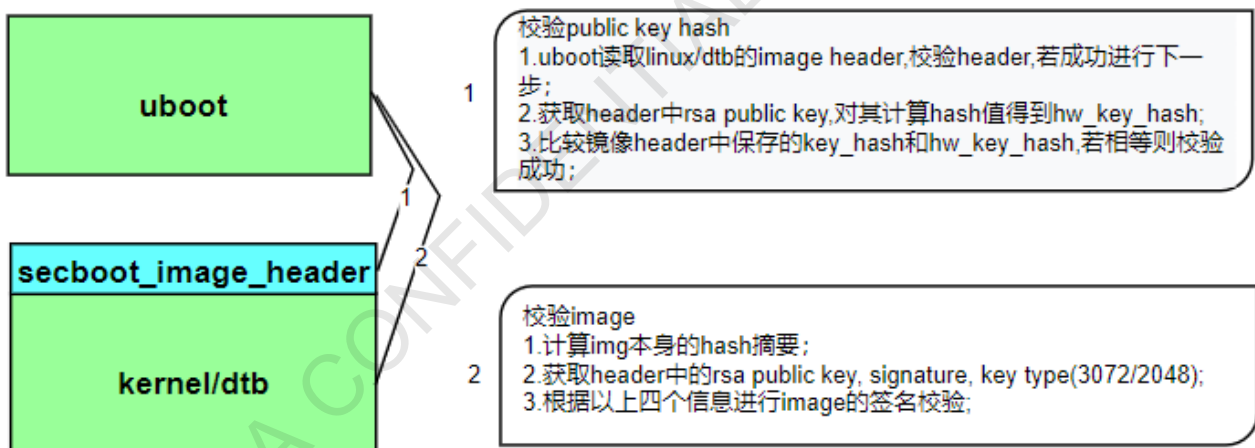
```

u32 img_check_sum;
u32 reserved1[2];
u32 boot_bak_flash_addr;
u32 reserved2[2];
struct rsa_key pub_key;
struct sig_struct signature;
u32 reserved3[20];
u32 aes_key[12];
u32 reserved4[17];
};

```

Secure boot verify 的信息主要分为两个部分：

1. 校验镜像中的 PK\_HASH;
2. 校验 image，如下图：



Secure boot 图解

第三阶段的校验原理与第一、二阶段的校验是一致的。

## Secure boot 方案实施

1. 准备一对 RSA 密钥对。

同 10.2.2 Secure boot 方案实施章节，使用上述共用公钥和私钥文件。

2. 对生成的镜像签名。

以 demo 板为例, SDK 中签名的镜像有 boot\_signed.bin、AX630C\_emmc\_arm64\_k419\_signed.dtb。

签名脚本存放在 {project}\build\tools\imgsign\sec\_boot\_AX620E\_sign.py, 以 RSA2048 的密钥为例:

签名 kernel 镜像:

```
@python3 $(IMGSIGN_SCRIPT)/sec_boot_AX620E_sign.py -i  
$(IMAGE_OUTDIR)/Image.axgzip -pub $(TOOLS_PATH)/imgsign/public.pem -prv  
$(TOOLS_PATH)/imgsign/private.pem -o  
$(BUILD_PATH)/out/$(PROJECT)/images/boot_signed.bin -cap 0x54FAFE -key_bit  
2048
```

具体实现可见对应工程代码中的 Makefile。

## 10.3 Optee 使用

### 10.3.1 Optee 编译

Optee 是针对有 trustzone 需求使用, 是可选择的, 暂时只支持编译 AX630C\_emmc\_arm64\_k419 工程, 默认是编译的, 编译后生成可用于下载的文件为 optee\_signed.bin, 可以单独编译, 步骤如下:

**步骤1** 进入安装目录/AX620E\_SDK\_Vx.x.x/boot/optee 目录。

**步骤2** 执行 make p=AX630C\_emmc\_arm64\_k419 clean all install 命令。

**步骤3** 编译成功后, 将在 build 目录下生成 out/AX630C\_emmc\_arm64\_k419 目录。

**步骤4** 进入 build/out/AX630C\_emmc\_arm64\_k419 目录后, 您将看到 optee\_signed.bin

**! 注意:**

如果不需要 optee, 可以在 build/projects/AX630C\_emmc\_arm64\_k419/project.mak 中配置 SUPPORT\_OPTEE := false, 在 build 目录编译 axp 时所有 optee 相关文件都不会被编译;

● SUPPORT\_OPTEE 会影响多个仓库的编译, 包括 spl、kernel、atf、third\_party 等仓库。

### 10.3.2 Optee 地址相关配置

可以通过配置 build/projects/AX630C\_emmc\_arm64\_k419/project.mak 中如下变量的数值

```
OPTEE_IMAGE_ADDR := 0x44000000
OPTEE_RESERVED_SIZE := 0x2000000
OPTEE_SHMEM_SIZE := 0x200000
```

OPTEE\_IMAGE\_ADDR: OPTEE 镜像加载地址, 也决定 linux 中给 optee reserved-memory 的起始地址, 以及 firewall 保护的起始地址;

OPTEE\_RESERVED\_SIZE: OPTEE 运行时需要占用的 ddr size, 也决定 linux 中给 optee reserved-memory 的起始 size;

OPTEE\_SHMEM\_SIZE: OPTEE 和 linux 共享的内存大小, 也决定 firewall 保护的 size, firewall 不能覆盖 OPTEE\_SHMEM 部分 ddr 范围;

**! 注意:**

Optee 运行的空间需要注意下 linux 中 reserved-memory 相关信息, 不要和其他预留的地址有重叠, 具体参考 kernel/linux/linux-4.19.125/arch/arm64/boot/dts/axera/AX630C\_emmc\_arm64\_k419.dts 中 reserved-memory 中相关预留项描述。

### 10.3.3 CA/TA 编译

参考 msp/sample/sample\_optee\_hello\_world 下 host (CA) 和 ta (TA) 目录的编译, 对于 TA 编译, 是会有签名的动作的, 默认使用 optee\_os-3.21.0/keys/default\_ta.pem, 如果需要修改, 则可以直接替换掉 default\_ta.pem 中内容或者通过 optee 目录 makefile 中 TA\_SIGN\_KEY 指定自己的私钥。

### 10.3.4 Optee debug

支持通过串口吐 log 方式进行 debug:

1. optee core log: 修改 optee/Makefile 中的 DEBUG\_LOG\_LEVEL 值为 1~4, 重新编译下载;

2. TA log: 修改 optee/optee\_os-3.21.0/mk/config.mk 中的 CFG\_TEE\_TA\_LOG\_LEVEL 值为 1~4, 重新编译下载;

### 10.3.5 Optee 相关路径说明

/opt/bin/OPTEE: optee sample 程序存放路径

/lib/optee\_armtz: optee TA 程序存放路径

/data/tee: tee 文件存放路径

### 10.3.6 HUK 配置

对于 optee 文件存储等业务, 需要有 HUK 支持, HUK 配置说明:

1. HUK 需要写入 efuse 中, 但是只能 CPU 在 secure 模式下有读写 efuse 权限;
2. SDK 中 HUK 分配 efuse blk32-35, 需要先使能 efuse blk13 bit24 (EFUSE\_SPACE);
3. Optee 默认代码中 HUK 是没有写入的。如果要需要开机的时候写入 HUK, 需要将 optee/optee\_os-3.21.0/core/arch/arm/plat-axera/main.c 函数 ax\_huk\_init 中注释掉的 code 使能后重新编译, 该函数会检查 efuse 中会检查 efuse blk13 bit24 (EFUSE\_SPACE) 是否使能, 如果已经使能并且 blk32-35 全为 0, 则会将 HUK 写入 efuse blk32-35, 否则直接返回。
4. optee/optee\_os-3.21.0/core/arch/arm/plat-axera/main.c 中重新定义了 tee\_opt\_get\_hw\_unique\_key 函数, 该函数会通过 ax\_cipher\_huk\_read 可以读取 efuse 中写入的 optee 的 huk。

### 10.3.7 DIE ID

Optee 提供了 tee\_opt\_get\_die\_id 函数, 通过该函数可以获得存在 efuse 中的芯片 UID, 不同的芯片该值是不同的。

## 10.4 平台安全配置

### 10.4.1 Firewall 简介

AX620E 支持通过 firewall 对 ddr 部分地址段进行保护，例如 optee 运行空间中会有使用 key 做加解密的操作，如果在 linux 端能直接获取这段 ddr 地址中的数据，可能会存在安全问题，需要考虑针对 optee 运行空间进行保护。

### 10.4.2 Firewall 配置

SDK 中针对 optee 运行空间，使用 firewall 做了保护，只允许 CPU\_SYS 在 secure 状态下访问受保护的 mem 区域，具体可参考

boot\atf\arm-trusted-firmware-2.7\plat\axera\ax620e\drivers\firewall.c 中 firewall\_config 函数的实现，另外需要 efuse bit map blk13 的 bit15 配置为 1 使能系统的 secsys 功能。

！注意：

AX620E 默认会根据 build/projects/AX620E\_xxx/project.mak 中配置的 OPTEE\_IMAGE\_ADDR、OPTEE\_RESERVED\_SIZE 以及 OPTEE\_SHMEM\_SIZE 对 optee 的运行空间  $\text{OPTEE\_IMAGE\_ADDR} + \text{OPTEE\_RESERVED\_SIZE} - \text{OPTEE\_SHMEM\_SIZE} - 1$  进行保护；

- firewall 初始化以及设置保护的 region 需要越早越好，需要在安全模式下。

## 10.5 Security 涉及的 Efuse 说明

### 10.5.1 PUB\_KEY\_HASH

如 10.2.2 bootrom 加载校验 spl 镜像章节中介绍，efuse bit map 的 blk15-blk22 这 8 个 blk 用于写入 bootrom 阶段 secure boot 使用的 public key 的 hash 值。

## 10.5.2 SEC\_BOOT

efuse blk13 的 bit26 配置为 1 使能 bootrom 阶段的 secure boot。

## 10.5.3 SEC\_SYS

efuse blk13 的 bit15 配置为 1 使能系统的 secsys 功能，使能后只有安全的 master 才能访问受保护的 mem 和安全外设（本 SDK 中暂不支持），并且在非安全下无法再读写 SEC\_GLB 寄存器进行配置。

！注意：

efuse 作为系统存储 PUB\_KEY\_HASH 以及 HUK 等 secure 资源，需要做安全保护；

- SEC\_GLB 中有 efuse 控制寄存器，可以控制 non-secure 下无法访问 efuse，已经在 atf 中 plat\_ax620e\_secure\_config 函数中对此做了使能，但是生效需要配置此 efuse bit。

## 10.5.4 EFUSE\_SPACE

efuse blk13 的 bit24 配置为 1，系统 efuse blk32-63 给 CPU 使用，optee 中 HUK SDK 分配使用 efuse blk32-35，需要使能此 bit。

## 10.5.5 JTAG\_DISABLE

efuse blk13 的 bit31 配置为 1 关闭 jtag 通路，对于安全产品需要配置为 1 进行关闭。

# 11 应用与内核 Debug 使用说明

## 11.1 软硬件配置要求

### 11.1.1 软件要求

软件环境要求

| 软件类型         | 配置描述            |
|--------------|-----------------|
| windows 操作系统 | X86             |
|              | Windows 7 以上    |
| 工具           | Xshell winscp 等 |

### 11.1.2 硬件要求

硬件环境要求

| 项目 | 配置描述                             |
|----|----------------------------------|
| PC | 普通的 pc 机硬件                       |
|    | 安装 windows 7 以上都可以。              |
|    | 建议:安装 ssh 客户端软件或者安装串口工具（xshell）。 |

|      |                                                                                                                                                                                                                                        |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 端口   | usb 串口设备用来下载使用<br><br>serial port 用来调试<br><br>(note : usb 串口 登录 用户名 root 密码: 123456)                                                                                                                                                   |
| SD 卡 | 用于 sysdump 文件存储满足如下要求:<br><br>大小至少 2GB 以上。<br><br>FAT32 格式类型。<br><br>建议: 一般我们用的 sd 卡的大小都超过了 32G,这样 windows 无法格式成 FAT32 格式, 比如 64G 卡, 显示格式 extFAT,这样我们可以到计算机管理中, 找到存储中的磁盘管理, 然后选中显示找到的移动磁盘, 点击右键格式几个卷标, 每个卷大小小于 32G 就满足了, 然后再去格式 FAT32。 |
| 网络要求 | 以太网线一根<br><br>主要用来板子和 pc 机通信, 通过板子上软件 ssh server 与 pc ssh client 通信 (推荐 winscp)                                                                                                                                                        |

## 11.2 Linux 内核 ramoops

### 11.2.1 简介

ramoops 是采用 ram 保存 oops 信息的一种技术, 该技术使用 pstore 机制实现, 在内核开关中用 3 个开关控制: PSTORE\_CONSOLE 控制是否保存控制台输出, PSTORE\_FTRACE 控制是否保存函数调用序列, PSTORE\_RAM 控制是否保存 panic/oops 信息。

## 11.2.2 使用方法

AX620E 的内核已经集成了 ramoops 功能当内核崩溃后重启系统会看到在/sys/fs/pstore 目录下有节点 console-ramoops-0 与 dmesg-ramoops-0 使用 cat 命令就可以读出 log 信息，注意因为 log 保存在 ram 中，所以当系统崩溃后只能选择让系统热重启即复位。

```
/sys/fs/pstore
# ls
console-ramoops-0  dmesg-ramoops-0
```

Ramoops 保存路径

## 11.3 Linux memory 转储

ax620E uboot 的 memory dump 配置决定 dump 文件的存储介质是 sd card、 emmc 或者 U 盘，将 dump 的文件默认分为两个 vmcore.dump 和 vmcore.dump.info。使用 crash 工具解析前需要将两个文件合并在一起 cat vmcore.dump.info.20230410160410 vmcore.dump.20230410160410 > vmcore.dump.bin，memory dump 的解析需要 vmlinux，解析命令如下 ./crash\_arm64 vmlinux vmcore.dump.bin

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...

    KERNEL: vmlinux
    DUMPFILE: vmcore.dump.bin
    CPUS: 8
    DATE: Mon Apr 10 16:04:11 CST 2023
    UPTIME: 00:00:19
LOAD AVERAGE: 0.44, 0.10, 0.03
    TASKS: 170
    NODENAME: AXERA
    RELEASE: 5.15.73
    VERSION: #2 SMP PREEMPT Mon Apr 10 14:49:31 CST 2023
    MACHINE: aarch64 (unknown Mhz)
    MEMORY: 256 MB
    PANIC: "Kernel panic - not syncing: sysrq triggered crash"
    PID: 485
    COMMAND: "sh"
    TASK: ffff000004c5b400 [THREAD_INFO: ffff000004c5b400]
    CPU: 4
    STATE: TASK_RUNNING (PANIC)

crash_arm64> █
```

crash 工具加载成功

可以执行 help 查看当前 crash 支持的命令，简单介绍几个常见命令。

help 命令列出 crash 支持的命令

```
crash_arm64> help

*          files          mod          sbitmapq      union
alias      foreach       mount       search         vm
ascii      fuser         net         set            vtop
bpf        gdb           p           sig            waitq
bt         help          ps          struct         whatis
btop       ipcs           ptb         swap           wr
dev        irq           ptov        sym            q
dis        kmem         rd          sys
eval       list          repeat      task
exit       log           runq        timer
extend     mach

crash_arm64 version: 8.0.1   gdb version: 10.2
For help on any command above, enter "help <command>".
For help on input options, enter "help input".
For help on output options, enter "help output".
```

help 命令

Log 命令查看内核 dmesg log

```
crash_arm64> log
[ 0.000000] Booting Linux on physical CPU 0x000000000 [0x412fd050]
[ 0.000000] Linux version 5.15.73 (xuxiyang@axcnshbussrv06p) (aarch64-none-linux-gnu-gcc (GNU Toolchain for the A-profile Architecture 9.2-2019.12 (arm-9.10)) 2.33.1.20191209) #2 SMP PREEMPT
[ 0.000000] Machine model: axera,ax650x
[ 0.000000] Memory limited to 256MB
[ 0.000000] earlycon: uart8250 at MMI032 0x0000000002016000 (options '')
[ 0.000000] printk: bootconsole [uart8250] enabled
[ 0.000000] Reserved memory: created CMA memory pool at 0x0000000108000000, size 128 MiB
[ 0.000000] OF: reserved mem: initialized node linux,cma, compatible id shared-dma-pool
[ 0.000000] Zone ranges:
[ 0.000000] Normal [mem 0x0000000100000000-0x000000010ffffffff]
[ 0.000000] Movable zone start for each node
[ 0.000000] Early memory node ranges
[ 0.000000] node 0: [mem 0x0000000100000000-0x000000010ffffffff]
[ 0.000000] Initmem setup node 0 [mem 0x0000000100000000-0x000000010ffffffff]
[ 0.000000] psci: probing for conduit method from DT.
[ 0.000000] psci: PSCIv1.1 detected in firmware.
[ 0.000000] psci: Using standard PSCI v0.2 function IDs
[ 0.000000] psci: Trusted OS migration not required
[ 0.000000] psci: SMC Calling Convention v1.2
[ 0.000000] percpu: Embedded 16 pages/cpu s28504 r8192 d28840 u65536
[ 0.000000] pcpu-alloc: s28504 r8192 d28840 u65536 alloc=16*4096
[ 0.000000] pcpu-alloc: [0] 0 [0] 1 [0] 2 [0] 3 [0] 4 [0] 5 [0] 6 [0] 7
[ 0.000000] Detected VIPT I-cache on CPU0
[ 0.000000] CPU features: kernel page table isolation disabled by kernel configuration
[ 0.000000] CPU features: detected: ARM errata 1165522, 1319367, or 1530923
```

log 命令

bt 命令查看内核 crash 栈

```

crash_arm64> bt
PID: 485    TASK: ffff000004c5b400 CPU: 4    COMMAND: "sh"
#0 [ffff80000ae23a50] axera_save_memory_dump at ffff800008572c20
#1 [ffff80000ae23c20] panic_at ffff800008a33698
#2 [ffff80000ae23d00] sysrq_handle_crash at ffff80000858db2c
#3 [ffff80000ae23d10] __handle_sysrq at ffff80000858e1bc
#4 [ffff80000ae23d60] write_sysrq_trigger at ffff80000858e784
#5 [ffff80000ae23d80] proc_reg_write at ffff800008229c30
#6 [ffff80000ae23da0] vfs_write at ffff80000819fa18
#7 [ffff80000ae23de0] ksys_write at ffff80000819fe64
#8 [ffff80000ae23e30] __arm64_sys_write at ffff80000819ff04
#9 [ffff80000ae23e40] invoke_syscall.constprop.0 at ffff800008020e04
#10 [ffff80000ae23e60] do_el0_svc at ffff800008020ed8
#11 [ffff80000ae23e80] el0_svc at ffff800008a4a750
#12 [ffff80000ae23ea0] el0t_64_sync_handler at ffff800008a4b324
#13 [ffff80000ae23fe0] el0t_64_sync at ffff8000080115a0
PC: 00000000004556a8 LR: 00000000004a22a8 SP: 0000fffffeecd35d0
X29: 0000fffffeecd35d0 X28: 000000000404c0d40 X27: 0000000000000000
X26: 0000000000000000 X25: 000000000404c0d20 X24: 0000000000000020
X23: 0000000000000001 X22: 000000000404bf700 X21: 0000000000000002
X20: 000000000404c6640 X19: 0000000000000001 X18: 0000000000000000
X17: 0000000000000000 X16: 00000000005e7030 X15: 0000000000000008
X14: 0000000000000013 X13: 0072656767697274 X12: 0101010101010101
X11: 0000000000000020 X10: 0101010101010101 X9: ffffffffefefefefef
X8: 0000000000000040 X7: 7f7f7f7f7f7f7f7f X6: 0080000080808080
X5: 0000000000000000 X4: 000000000404c0063 X3: 000000000404bef00
X2: 0000000000000002 X1: 000000000404c6640 X0: 0000000000000001
ORIG_X0: 0000000000000001 SYSCALLNO: 40 PSTATE: 80001000

```

bt 命令

ps 命令查看系统的系统进程和内核线程

```
crash_arm64> ps
  PID   PPID  CPU   TASK                ST  %MEM  VSZ   RSS  COMM
> 0      0    0   ffff8000008eaf480  RU  0.0   0     0   [swapper/0]
> 0      0    1   ffff00000018f4100  RU  0.0   0     0   [swapper/1]
> 0      0    2   ffff00000018f4e00  RU  0.0   0     0   [swapper/2]
> 0      0    3   ffff00000018f5b00  RU  0.0   0     0   [swapper/3]
> 0      0    4   ffff00000018f6800  RU  0.0   0     0   [swapper/4]
> 0      0    5   ffff0000001950000  RU  0.0   0     0   [swapper/5]
> 0      0    6   ffff0000001950d00  RU  0.0   0     0   [swapper/6]
> 0      0    7   ffff0000001951a00  RU  0.0   0     0   [swapper/7]
  1      0    1   ffff00000018d0000  IN  0.0  2188  1136  init
  2      0    1   ffff00000018d0d00  IN  0.0   0     0   [kthreadd]
  3      2    0   ffff00000018d1a00  ID  0.0   0     0   [rcu_gp]
  4      2    0   ffff00000018d2700  ID  0.0   0     0   [rcu_par_gp]
  5      2    0   ffff00000018d3400  ID  0.0   0     0   [slub_flushwq]
  6      2    0   ffff00000018d4100  ID  0.0   0     0   [netns]
  7      2    0   ffff00000018d4e00  ID  0.0   0     0   [kworker/0:0]
  8      2    0   ffff00000018d5b00  ID  0.0   0     0   [kworker/0:0H]
  9      2    5   ffff00000018d6800  ID  0.0   0     0   [kworker/u16:0]
 10      2    0   ffff00000018f0000  ID  0.0   0     0   [mm_percpu_wq]
 11      2    0   ffff00000018f0d00  IN  0.0   0     0   [rcu_tasks_kthre]
 12      2    0   ffff00000018f1a00  IN  0.0   0     0   [ksoftirqd/0]
 13      2    6   ffff00000018f2700  ID  0.0   0     0   [rcu_preempt]
 14      2    0   ffff00000018f3400  IN  0.0   0     0   [migration/0]
 15      2    0   ffff0000001952700  IN  0.0   0     0   [cpuhp/0]
 16      2    1   ffff0000001953400  IN  0.0   0     0   [cpuhp/1]
 17      2    1   ffff0000001954100  IN  0.0   0     0   [migration/1]
 18      2    1   ffff0000001954e00  IN  0.0   0     0   [ksoftirqd/1]
 19      2    1   ffff0000001955b00  ID  0.0   0     0   [kworker/1:0]
 20      2    1   ffff0000001956800  ID  0.0   0     0   [kworker/1:0H]
 21      2    2   ffff0000001958000  IN  0.0   0     0   [cpuhp/2]
 22      2    2   ffff0000001958d00  IN  0.0   0     0   [migration/2]
 23      2    2   ffff0000001959a00  IN  0.0   0     0   [ksoftirqd/2]
```

## ps 命令

mod 查看内核加载了哪些模块

```
crash> mod
  MODULE      NAME                SIZE  OBJECT  FILE
ffff00000008b2200 ax_isp             16384 (not loaded) [CONFIG_KALLSYMS]
ffff00000008bb2c0 ax_uirq            16384 (not loaded) [CONFIG_KALLSYMS]
ffff00000008c6280 ax_sys             28672 (not loaded) [CONFIG_KALLSYMS]
ffff00000008d2240 ax_venc            20480 (not loaded) [CONFIG_KALLSYMS]
ffff00000008db0c0 uio_pdrv_genirq    16384 (not loaded) [CONFIG_KALLSYMS]
ffff00000008fa700 ax_vdec            53248 (not loaded) [CONFIG_KALLSYMS]
ffff00000008fb2c0 ax_jpegenc         40960 (not loaded) [CONFIG_KALLSYMS]
ffff000000090a280 ax_jpegdec         40960 (not loaded) [CONFIG_KALLSYMS]
```

## Mod 命令

sym -l <模块的名字> 查看模块的符号表

```

crash> sym -l ax_isp
ffff000008080000 (t) .head.text
ffff000008080000 (t) _head
ffff000008080000 (T) _text
ffff000008080800 (t) .text
ffff000008080800 (T) __exception_text_start
ffff000008080800 (T) _stext
ffff000008080800 (T) do_undefinstr
ffff000008080a98 (T) do_sysinstr
ffff000008080b08 (T) do_mem_abort
ffff000008080c08 (T) do_el0_irq_bp_hardening
ffff000008080c58 (T) do_el0_ia_bp_hardening
ffff000008080cf0 (T) do_sp_pc_abort
ffff000008080e10 (T) do_debug_exception
ffff000008080fa0 (t) gic_handle_irq
ffff000008081048 (t) gic_handle_irq
ffff0000080811c8 (T) __do_softirq
ffff0000080811c8 (T) __exception_text_end
ffff0000080811c8 (T) __irqentry_text_end
ffff0000080811c8 (T) __irqentry_text_start
ffff0000080811c8 (T) __softirqentry_text_start
ffff0000080813d8 (T) __entry_text_start
ffff0000080813d8 (T) __softirqentry_text_end
ffff000008081800 (T) vectors

```

### sym 使用方法

crash 工具功能很强大这里只介绍了几个命令的功能，读者可以根据需要查看 crash 的相关手册。

## 11.4 gdb 调试应用程序 coredump

### 配置 kernel 支持 coredump

打开 kernel 部分的编译选项，使用 kernel\_menuconfig 选择，如下所示：

General setup=>Configure standard kernel features(expert users)=>Enable ELF core dumps

### 设置 core 文件命名格式和路径

因为默认情况下 core 文件的名称就是 core，生成的位置与应用程序相同的目录下面，但是我们系统只有/tmp 下面才是可写的，而且名字为 core 的话如果出现多个应用程序 crash，那么新的 core 文件会覆盖之前的，导致会遗漏问题点，因此考虑需要设置文件名字和 core 文件的路径，具体就是在 rcs 中加如下两行

```

echo 1 > /proc/sys/kernel/core_uses_pid

```

```
echo "/tmp/core-%e-%p-%t" > proc/sys/kernel/core_pattern
```

备注:

(1) `/proc/sys/kernel/core_uses_pid` 实现控制 `core` 文件的文件名中是否添加 `pid` 作为扩展。

文件内容为 1, 表示添加 `pid` 作为扩展名, 生成的 `core` 文件格式为 `core.xxxx`; 为 0 则表示生成的 `core` 文件统一命名为 `core`。

(2) `proc/sys/kernel/core_pattern` 可以控制 `core` 文件保存位置和文件名格式。

以下是参数列表:

`%p` - 添加 `pid`

`%u` - 添加当前 `uid`

`%g` - 添加当前 `gid`

`%s` - 添加导致产生 `core` 的信号

`%t` - 添加 `core` 文件生成时的 `unix` 时间

`%h` - 添加主机名

`%e` - 添加命令名

开启 `core dump` 转存功能

`ulimit -c unlimited` 这个命令开启 `core dump` 转存功能, `coredump` 文件的大小没有限制

`gdb` 调试 `core dump` 文件 `aarch64- none-linux-gnu-gdb test core-xxxxx` 出现了 `#0 xxxxxxxx in ??`

`()`字样, 请用 `info sharedlibrary` 命令查看是否链接的库都加载了, 如果没有, 可以使用 `set solib-search-path pathname` 命令来设置库搜索路径 `set solib-search-path`

`rootfs/rootfs/lib:/msp/out/lib:/rootfs/rootfs/lib64:/rootfs/rootfs/usr/lib:/rootfs/rootfs/usr/local/lib/`

```
warning: .dynamic section for "/lib/libgcc_s.so.1" is not at the expected address (wrong lib
Error while mapping shared library sections:
/lib/libc.so.0: No such file or directory.
Error while mapping shared library sections:
/lib/ld-uClibc.so.0: No such file or directory.
Reading symbols from /lib/libgcc_s.so.1...done.
Loaded symbols for /lib/libgcc_s.so.1
Symbol file not found for /lib/libc.so.0
Symbol file not found for /lib/ld-uClibc.so.0

warning: Unable to find dynamic linker breakpoint function.
GDB will be unable to debug shared library initializers
and track explicitly loaded dynamic code.
Core was generated by `crash'.
Program terminated with signal 11, Segmentation fault.
#0  0x0040083c in main () at crash.c:5
5      *ptr = 0x00;
(gdb) _
```

gdb 加载 coredump 文件

## 11.5 strace 调试应用程序

strace 是一个强大的应用层调试工具，他的特点是简单好用，局限是只能跟踪到系统调用层，以下是一些经常使用的例子更多的使用请大家网上搜索。

1. 跟踪应用程序中一些线程的状态。首先使用 `ps -T` 显示进程的线程号，使用 `strace -p <线程号>` 就可以跟踪线程当前的状态

```
# strace -p 1201
strace: Process 1201 attached
ppoll([{fd=3, events=POLLIN}, {fd=-1}], 2, {tv_sec=43038, tv_nsec=434339673}, NULL, 0
```

strace 使用线程

2. 查看某个程序或者命令执行过程中做了什么调用了哪些库文件

strace `date -s "2020-10-20"` 可以看到使用系统调用 `settimeofday` 设置了系统时间

```
getuid() = 0
brk(NULL) = 0x3dfdc000
brk(0x3dffd000) = 0x3dffd000
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
settimeofday({tv_sec=1603152000, tv_nsec=0}, NULL) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0
ioctl(1, TCGETS, {B115200 opost isig icanon echo ...}) = 0
write(1, "Tue Oct 20 00:00:00 UTC 2020\n", 29Tue Oct 20 00:00:00 UTC 2020
) = 29
```

strace 使用 date

3. 跟踪系统调用执行所需要的时间 `strace -tt -T date -s "2020-10-20"`

```
00:00:41.335625 getuid() = 0 <0.000015>
00:00:41.335778 brk(NULL) = 0x366e9000 <0.000015>
00:00:41.335844 brk(0x3670a000) = 0x3670a000 <0.000020>
00:00:41.335930 openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory) <0.000020>
00:00:41.336075 openat(AT_FDCWD, "/etc/localtime", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory) <0.000019>
00:00:41.336159 settimeofday({tv_sec=1603152000, tv_usec=0}, NULL) = 0 <0.000030>
00:00:00.000058 fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x5, 0x1), ...}) = 0 <0.000017>
00:00:00.000134 ioctl(1, TCGETS, {B115200 opost isig icanon echo ...}) = 0 <0.000023>
00:00:00.000225 write(1, "Tue Oct 20 00:00:00 UTC 2020\n", 29Tue Oct 20 00:00:00 UTC 2020
```

strace 跟踪系统时间

# 12 Clock 使用说明

本章节主要描述各模块时钟寄存器的信息。

## 12.1 CPU

CPU 时钟频率可通过读取寄存器 0x1900000 [4:2]来获得。

|             |       |    |      |                                                                                                                                                                     |
|-------------|-------|----|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| clk_cpu_sel | [4:2] | RW | 3'h0 | clk_cpu mux select:<br>3'b000 : cpll_24m<br>3'b001 : cpll_208m<br>3'b010 : epll_500m<br>3'b011 : nppll_800m<br>3'b100 : cpupll_1200m<br>default : cpll_24m (3'b000) |
|-------------|-------|----|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 12.2 DDR

DDR 时钟频率可通过读取 0x1800000 [18:17]与 0x265010 这两个寄存器的值来获得。

只有选择了 dpll\_3200m 才需要读取 0x265010 寄存器确认分频值。

|             |         |    |      |                                                                                                                                          |
|-------------|---------|----|------|------------------------------------------------------------------------------------------------------------------------------------------|
| clk_ddr_sel | [18:17] | RW | 2'h0 | clk_ddr mux select:<br>2'b00 : cpll_24m<br>2'b01 : nppll_1600m<br>2'b10 : cpll_2496m<br>2'b11 : dpll_3200m<br>default : cpll_24m (2'b00) |
|-------------|---------|----|------|------------------------------------------------------------------------------------------------------------------------------------------|

clk\_dds\_divn 值对应的分频系数如下：

0---1 分频

1---2 分频

2---4 分频

3---6 分频

.....

依此类推。

DDR 的时钟频率最终是根据 clk\_dds\_sel/clk\_dds\_divn 分频系数计算得到，注意系数修改后需要将 update bit 写 1，delay 50us 后再写 0。

## 12.3 EMMC

EMMC 时钟频率可通过读取寄存器 0x1900000 [6:5]来获得：

|                 |       |    |      |                                                                                                                                            |
|-----------------|-------|----|------|--------------------------------------------------------------------------------------------------------------------------------------------|
| clk_tx_emmc_sel | [6:5] | RW | 2'h0 | clk_emmc_card mux select:<br>2'b00 : cpll_24m<br>2'b01 : cpll_312m<br>2'b10 : epll_375m<br>2'b11 : npll_400m<br>default : cpll_24m (2'b00) |
|-----------------|-------|----|------|--------------------------------------------------------------------------------------------------------------------------------------------|

## 12.4 FLASH

FLASH 时钟频率可通过读取寄存器 0x1900000 [1:0]来获得：

|                   |       |    |      |                                               |
|-------------------|-------|----|------|-----------------------------------------------|
| clk_bus_flash_sel | [1:0] | RW | 2'h0 | clk_bus_flash mux select:<br>2'b00 : cpll_24m |
|-------------------|-------|----|------|-----------------------------------------------|

|  |  |  |  |                                                                                           |
|--|--|--|--|-------------------------------------------------------------------------------------------|
|  |  |  |  | 2'b01 : ep1l_125m<br>2'b10 : cp1l_208m<br>2'b11 : cp1l_312m<br>default : cp1l_24m (2'b00) |
|--|--|--|--|-------------------------------------------------------------------------------------------|

## 12.5 AUDIO

i2s 时钟频率可通过 clk\_i2s\_ref0\_sel 和 sclk\_i2s\_tdm\_sel 、 sclk\_i2s\_m\_sel 三组寄存器来获取：

地址：0x4870000

|                  |         |    |      |                                                                                                                                                |
|------------------|---------|----|------|------------------------------------------------------------------------------------------------------------------------------------------------|
| clk_i2s_ref0_sel | [6:5]   | RW | 2'h0 | clk_i2s_ref0 mux select:<br>2'b00 : cp1l_12m<br>2'b01 : hp1l_16p384m<br>2'b10 : hp1l_24p576m<br>2'b11 : ep1l_25m<br>default : cp1l_12m (2'b00) |
| sclk_i2s_tdm_sel | [24:23] | RW | 2'h0 | sclk_i2s_tdm mux select:<br>2'b00 : hp1l_16p384m<br>2'b01 : hp1l_20p48m<br>2'b10 : hp1l_24p576m<br>default : hp1l_16p384m (2'b00)              |
| sclk_i2s_m_sel   | [22:21] | RW | 2'h0 | sclk_i2s_m mux select:<br>2'b00 : hp1l_16p384m<br>2'b01 : hp1l_20p48m                                                                          |

|  |  |  |  |                                                        |
|--|--|--|--|--------------------------------------------------------|
|  |  |  |  | 2'b10 : hpll_24p576m<br>default : hpll_16p384m (2'b00) |
|--|--|--|--|--------------------------------------------------------|

地址：0x4870014

|                   |         |    |      |                                   |
|-------------------|---------|----|------|-----------------------------------|
| sclk_i2s_tdm_divn | [19:14] | RW | 6'h0 | clock sclk_i2s_tdm divider number |
| sclk_i2s_m_divn   | [12:7]  | RW | 6'h0 | clock sclk_i2s_m divider number   |
| clk_i2s_ref0_divn | [5:0]   | RW | 6'h0 | clock clk_i2s_ref0 divider number |

sclk\_i2s 时钟频率计算：

$\text{sclk\_i2s\_sel} / \text{sclk\_i2s\_divn}$  分频系数

sclk\_i2s\_divn 分频系计算方式如下：

0---1 分频

1---2 分频

2---4 分频

3---6 分频

.....

依此类推。后续所有的分频系数计算方法均如此，不再复述。

## 12.6 EMAC

EMAC Eth0 时钟频率可通过读取寄存器 0x10030000 [5:4]来获得：

|                     |       |    |      |                                                  |
|---------------------|-------|----|------|--------------------------------------------------|
| clk_mm_rgmii_tx_sel | [5:4] | RW | 2'h0 | clk_emac_rgmii_tx mux select:<br>2'b00 : ep1l_5m |
|---------------------|-------|----|------|--------------------------------------------------|

|  |  |  |  |                                                                    |
|--|--|--|--|--------------------------------------------------------------------|
|  |  |  |  | 2'b01 : ep1l_50m<br>2'b10 : ep1l_250m<br>default : ep1l_5m (2'b00) |
|--|--|--|--|--------------------------------------------------------------------|

## 12.7 MCLK

| MCLK  | 寄存器名称            | 寄存器地址      | 寄存器配置方法                                                          | 寄存器描述                                                                                                                                                                                                                                                                                                                                                               |
|-------|------------------|------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mclk0 | clk freq 频率选择寄存器 | 0x02340000 | 先 read 寄存器的值，然后修改 bit0~bit3 的值<br>示例：ax_lookat 0x02340000 -s 0x3 | MCLK0 mux select:<br>4'b0000 : cp1l_12m<br>4'b0001 : cp1l_19p2m<br>4'b0010 : hp1l_20p48m<br>4'b0011 : cp1l_24m<br>4'b0100 : hp1l_24p576m<br>4'b0101 : ep1l_25m<br>4'b0110 : cp1l_26m<br>4'b0111 : vp1l0_27m<br>4'b1000 : vp1l1_27m<br>4'b1001 : ep1l_50m<br>4'b1010 : vp1l0_74p25m<br>4'b1011 : vp1l1_74p25m<br>4'b1100 : ep1l_125m<br>default : cp1l_12m (4'b0000) |
|       | clk en 使能寄存器     | 0x02340024 | 先 read 寄存器的值，然后修改 bit0 的值<br>示例：ax_lookat 0x02340024 -s          | clock MCLK0 enable control:<br>1'b0 : disable<br>1'b1 : enable                                                                                                                                                                                                                                                                                                      |

| MCLK  | 寄存器名称            | 寄存器地址      | 寄存器配置方法                                                               | 寄存器描述                                                                                                                                                                                                                                                                                                                                                               |
|-------|------------------|------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                  |            | 0x1                                                                   |                                                                                                                                                                                                                                                                                                                                                                     |
| mclk1 | clk freq 频率选择寄存器 | 0x02340000 | 先 read 寄存器的值，然后修改 bit4~bit7 的值<br><br>示例：ax_lookat 0x02340000 -s 0x30 | MCLK1 mux select:<br>4'b0000 : cpll_12m<br>4'b0001 : cpll_19p2m<br>4'b0010 : hpll_20p48m<br>4'b0011 : cpll_24m<br>4'b0100 : hpll_24p576m<br>4'b0101 : epll_25m<br>4'b0110 : cpll_26m<br>4'b0111 : vpll0_27m<br>4'b1000 : vpll1_27m<br>4'b1001 : epll_50m<br>4'b1010 : vpll0_74p25m<br>4'b1011 : vpll1_74p25m<br>4'b1100 : epll_125m<br>default : cpll_12m (4'b0000) |
|       | clk en 使能寄存器     | 0x02340024 | 先 read 寄存器的值，然后修改 bit1 的值<br><br>示例：ax_lookat 0x02340024 -s 0x2       | clock MCLK1 enable control:<br>1'b0 : disable<br>1'b1 : enable                                                                                                                                                                                                                                                                                                      |
| mclk2 | clk freq 频率选择寄存器 | 0x02340000 | 先 read 寄存器的值，然后修改 bit8~bit11 的值                                       | MCLK2 mux select:<br>4'b0000 : cpll_12m<br>4'b0001 : cpll_19p2m                                                                                                                                                                                                                                                                                                     |

| MCLK  | 寄存器名称            | 寄存器地址      | 寄存器配置方法                                                                          | 寄存器描述                                                                                                                                                                                                                                                                                            |
|-------|------------------|------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                  |            | 示例: ax_lookat<br>0x02340000 -s<br>0x300                                          | 4'b0010 : hpll_20p48m<br>4'b0011 : cpll_24m<br>4'b0100 : hpll_24p576m<br>4'b0101 : epll_25m<br>4'b0110 : cpll_26m<br>4'b0111 : vpll0_27m<br>4'b1000 : vpll1_27m<br>4'b1001 : epll_50m<br>4'b1010 : vpll0_74p25m<br>4'b1011 : vpll1_74p25m<br>4'b1100 : epll_125m<br>default : cpll_12m (4'b0000) |
|       | clk en 使能寄存器     | 0x02340024 | 先 read 寄存器的值，然后修改 bit2 的值<br><br>示例: ax_lookat<br>0x02340024 -s<br>0x4           | clock MCLK2 enable control:<br>1'b0 : disable<br>1'b1 : enable                                                                                                                                                                                                                                   |
| mclk3 | clk freq 频率选择寄存器 | 0x02340000 | 先 read 寄存器的值，然后修改 bit12~bit15 的值<br><br>示例: ax_lookat<br>0x02340000 -s<br>0x3000 | MCLK3 mux select:<br>4'b0000 : cpll_12m<br>4'b0001 : cpll_19p2m<br>4'b0010 : hpll_20p48m<br>4'b0011 : cpll_24m<br>4'b0100 : hpll_24p576m<br>4'b0101 : epll_25m<br>4'b0110 : cpll_26m                                                                                                             |

| MCLK  | 寄存器名称            | 寄存器地址      | 寄存器配置方法                                                                    | 寄存器描述                                                                                                                                                                                                                                                                                                        |
|-------|------------------|------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                  |            |                                                                            | 4'b0111 : vpll0_27m<br>4'b1000 : vpll1_27m<br>4'b1001 : epll_50m<br>4'b1010 : vpll0_74p25m<br>4'b1011 : vpll1_74p25m<br>4'b1100 : epll_125m<br>default : cpll_12m (4'b0000)                                                                                                                                  |
|       | clk en 使能寄存器     | 0x02340024 | 先 read 寄存器的值，然后修改 bit3 的值<br><br>示例：ax_lookat 0x02340024 -s 0x8            | clock MCLK3 enable control:<br>1'b0 : disable<br>1'b1 : enable                                                                                                                                                                                                                                               |
| mclk4 | clk freq 频率选择寄存器 | 0x02340000 | 先 read 寄存器的值，然后修改 bit19~bit15 的值<br><br>示例：ax_lookat 0x02340000 -s 0x30000 | MCLK4 mux select:<br>4'b0000 : cpll_12m<br>4'b0001 : cpll_19p2m<br>4'b0010 : hpll_20p48m<br>4'b0011 : cpll_24m<br>4'b0100 : hpll_24p576m<br>4'b0101 : epll_25m<br>4'b0110 : cpll_26m<br>4'b0111 : vpll0_27m<br>4'b1000 : vpll1_27m<br>4'b1001 : epll_50m<br>4'b1010 : vpll0_74p25m<br>4'b1011 : vpll1_74p25m |

| MCLK  | 寄存器名称            | 寄存器地址      | 寄存器配置方法                                                                     | 寄存器描述                                                                                                                                                                                                                                                                                                                                                               |
|-------|------------------|------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                  |            |                                                                             | 4'b1100 : ep1l_125m<br>default : cpll_12m (4'b0000)                                                                                                                                                                                                                                                                                                                 |
|       | clk en 使能寄存器     | 0x02340024 | 先 read 寄存器的值，然后修改 bit4 的值<br><br>示例：ax_lookat 0x02340024 -s 0x10            | clock MCLK4 enable control:<br>1'b0 : disable<br>1'b1 : enable                                                                                                                                                                                                                                                                                                      |
| mclk5 | clk freq 频率选择寄存器 | 0x02340000 | 先 read 寄存器的值，然后修改 bit20~bit23 的值<br><br>示例：ax_lookat 0x02340000 -s 0x300000 | MCLK5 mux select:<br>4'b0000 : cpll_12m<br>4'b0001 : cpll_19p2m<br>4'b0010 : hp1l_20p48m<br>4'b0011 : cpll_24m<br>4'b0100 : hp1l_24p576m<br>4'b0101 : ep1l_25m<br>4'b0110 : cpll_26m<br>4'b0111 : vp1l0_27m<br>4'b1000 : vp1l1_27m<br>4'b1001 : ep1l_50m<br>4'b1010 : vp1l0_74p25m<br>4'b1011 : vp1l1_74p25m<br>4'b1100 : ep1l_125m<br>default : cpll_12m (4'b0000) |
|       | clk en 使能寄存器     | 0x02340024 | 先 read 寄存器的值，然后修改                                                           | clock MCLK5 enable control:<br>1'b0 : disable                                                                                                                                                                                                                                                                                                                       |

| MCLK | 寄存器名称 | 寄存器地址 | 寄存器配置方法                                           | 寄存器描述         |
|------|-------|-------|---------------------------------------------------|---------------|
|      |       |       | bit5 的值<br>示例: ax_lookat<br>0x02340024 -s<br>0x20 | 1'b1 : enable |

# 13 SDK OS Memory 统计

## 13.1 概要说明

### 13.1.1 需求背景

在进行稳定性问题分析时候，内存泄漏问题不太容易分析，在发生内存泄漏的时候不太好分析那个模块或者那个进程发生了内存泄漏；

在进行内存统计的时候需要统计某个模块分配的系统内存的空间大小，方便进行优化，以及内存需求的计算；

SDK 提供了 OS Memory 的统计功能，可以分模块统计每个模块申请的系统内存大小以及申请内存的次数，简称 OS\_MEM。

### 13.1.2 支持的功能

- 同时支持应用态和内核态的内存统计；
- 能同时统计每个模块对应申请的内存大小和申请次数；
- 针对内核的分配，大小计算支持 aligned 操作，aligned 策略和 linux 版本保持一致；
- 支持名字为 common 的 ID，方便给 osal 等一些不属于具体模块的内存申请；
- 支持不同统计策略，在小内存的软件中，只是统计申请的次数，不记录每次申请的 ID 和大小，对于大内存统计更加详细，该开关通过/proc 节点来配置，默认是小内存，减少统计的内存消耗和时间开销；
- 提供显示某个模块内核态内存的大小和 mod\_id/sub\_id 的详细信息。

### 13.1.3 不支持的功能

- 不支持单个模块内核或者用户态申请的内存大于 4GB，但是支持内核态和用户态申请的内存总和大于 4GB；
- 不支持没有调用接口的内存统计，例如 C++ 的 new/delete、系统的文件 cache。

### 13.1.4 统计策略

policy: OS\_MEM 进行内存统计的时候的策略，引入是为了 os\_mem 可以使用不同的场景，目前支持 0-2 3 个 policy:

0: OS\_MEM 分配内存可以支持多进程，统计只能统计多进程的次数，不支持统计多进程的大小；

1: OS\_MEM 可以统计每个模块分配内存的次数，每个模块每次分配的内存大小，在计算最大分配次数和分配内存大小的时候，只统计当前正在运行的所有进程历史的最大分配次数和分配大小，不包括已经关闭的进程的内存分配；

2: OS\_MEM 可以统计每个模块分配内存的次数，每个模块每次分配的内存大小，在计算最大分配次数和分配内存大小的时候，统计从开机到当前所有进程的最大分配次数和分配大小，包括已经关闭的进程的内存分配；

policy 可以通过文件节点来配置，默认使用 0 policy（CPU loading 影响最小，内存占用最小），如果要统计详细信息，可以在所有模块的 ko load 之前通过 echo 节点进行配置；

方法如下：

```
echo 0 > /proc/ax_proc/os_mem/policy # 设置 policy 为 0
```

```
echo 1 > /proc/ax_proc/os_mem/policy # 设置 policy 为 1
```

```
echo 2 > /proc/ax_proc/os_mem/policy # 设置 policy 为 2
```

13.1.5 PROC 文件节点的说明

| 文件节点                             | 作用说明                                    |
|----------------------------------|-----------------------------------------|
| /proc/ax_proc/os_mem/max_display | 该节点用来配置可以查看每个 mod_id 的详细信息的最大个数         |
| /proc/ax_proc/os_mem/policy      | 用来控制统计的 policy:                         |
| /proc/ax_proc/os_mem/stat        | 用来显示统计信息                                |
| /proc/ax_proc/os_mem/ctrl        | 用来清除 user 层的历史内存统计，主要给 policy 为 2 的时候使用 |

13.2 使用说明

13.2.1 内存次数统计使用

OS\_MEM 默认的 policy 是 0，只统计内存申请的次数，可以通过/proc/ax\_proc/os\_mem/stat 文件节点来得到每个模块申请次数：

```
# cat /proc/ax_proc/os_mem/stat
```

| module | mod_id | usr_size | ker_size | usr_times | ker_times | max_ker_size | max_ker_times | max_usr_size | max_usr_times |
|--------|--------|----------|----------|-----------|-----------|--------------|---------------|--------------|---------------|
| COMMON | 0      | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| ISP    | 1      | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| CE     | 2      | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| VO     | 3      | 0        | 0        | 0         | 4         | 0            | 0             | 0            | 0             |
| VDSP   | 4      | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| EFUSE  | 5      | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| NPU    | 6      | 0        | 0        | 0         | 2         | 0            | 2             | 0            | 0             |
| VENC   | 7      | 0        | 0        | 0         | 13        | 0            | 13            | 0            | 0             |
| VDEC   | 8      | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| JENC   | 9      | 0        | 0        | 0         | 5         | 0            | 5             | 0            | 0             |
| JDEC   | 10     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| SYS    | 11     | 0        | 0        | 0         | 13        | 0            | 13            | 0            | 0             |
| AENC   | 12     | 0        | 0        | 0         | 1         | 0            | 1             | 0            | 0             |
| IVPS   | 13     | 0        | 0        | 0         | 6         | 0            | 6             | 0            | 0             |
| MIPI   | 14     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| ADEC   | 15     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| DMA    | 16     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| VIN    | 17     | 0        | 0        | 0         | 8         | 0            | 8             | 0            | 0             |
| USER   | 18     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| IVES   | 19     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| SKEL   | 20     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| IVE    | 21     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 22     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 23     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 24     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 25     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| AUDIO  | 26     | 0        | 0        | 0         | 12        | 0            | 12            | 0            | 0             |
| ALGO   | 27     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| ENGINE | 28     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 29     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 30     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 31     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| AI     | 32     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| AO     | 33     | 0        | 0        | 0         | 1         | 0            | 1             | 0            | 0             |
| SENSOR | 34     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| NT     | 35     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 36     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 37     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 38     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 39     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| BASE   | 40     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| THM    | 41     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| 3A-AE  | 42     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| 3A-AWB | 43     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| 3A-AF  | 44     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |
| RESERV | 45     | 0        | 0        | 0         | 0         | 0            | 0             | 0            | 0             |

All: 0k(usr) 0k(ker) 0(usr\_times) 65(ker\_times)

## 13.2.2 统计每个模块申请的内存大小和次数

只有 policy 设置为 1 和 2 的时候才支持统计内存的大小

使用方法在 ko 加载前执行下面命令，这样才能统计到所有的 KO 申请的内存。

对于 SDK 来说，已将下面的命令加到/soc/scripts/auto\_load\_all\_drv.sh 文件开始的地方。

```
# echo 1 > /proc/ax_proc/os_mem/policy
```

或者

```
# echo 2 > /proc/ax_proc/os_mem/policy
```

```

/proc/ax_proc/os_mem # cat stat
module mod_id usr_size ker_size usr_times ker_times max_ker_size max_ker_times max_usr_size max_usr_times
COMMON 0 0 0 0 0 0 0 0 0 0
ISP 1 0 0 0 0 0 0 0 0 0
CE 2 0 0 0 0 0 0 0 0 0
VO 3 0 2304 0 2 2304 2 0 0 0
VDSP 4 0 0 0 0 0 0 0 0 0
EFUSE 5 0 0 0 0 0 0 0 0 0
NPU 6 469 182912 10 105 404544 106 3550 61
VENC 7 298112 1282048 96 19 1290240 21 298112 96
VDEC 8 0 0 0 0 0 0 0 0 0
JENC 9 80976 28672 8 7 36864 9 80976 8
JDEC 10 0 0 0 0 0 0 0 0 0
SYS 11 0 27776 0 149 27776 149 1712 1
AENC 12 0 4096 0 1 4096 1 0 0
IVPS 13 968 108800 1 25 108800 25 968 1
MIPI 14 0 0 0 0 0 0 0 0 0
ADEC 15 0 0 0 0 0 0 0 0 0
DMA 16 0 0 0 0 0 0 0 0 0
VIN 17 4273588 7493504 106 2614 7493504 2614 6896782 106
USER 18 0 0 0 0 0 0 0 0 0
IVES 19 0 0 0 0 0 0 0 0 0
SKEL 20 0 0 0 0 0 0 0 0 0
IVE 21 0 0 0 0 0 0 0 0 0
RESERV 22 0 0 0 0 0 0 0 0 0
RESERV 23 0 0 0 0 0 0 0 0 0
RESERV 24 0 0 0 0 0 0 0 0 0
RESERV 25 0 0 0 0 0 0 0 0 0
AUDIO 26 0 66048 0 12 66048 12 0 0
ALGO 27 3041660 0 19 0 0 0 3041660 19
ENGINE 28 8682 0 137 0 0 0 46128 680
RESERV 29 0 0 0 0 0 0 0 0 0
RESERV 30 0 0 0 0 0 0 0 0 0
RESERV 31 0 0 0 0 0 0 0 0 0
AI 32 0 0 0 0 0 0 0 0 0
AO 33 0 61440 0 1 61440 1 0 0
SENSOR 34 0 0 0 0 0 0 0 0 0
NT 35 0 0 0 0 0 0 0 0 0
RESERV 36 0 0 0 0 0 0 0 0 0
RESERV 37 0 0 0 0 0 0 0 0 0
RESERV 38 0 0 0 0 0 0 0 0 0
RESERV 39 0 0 0 0 0 0 0 0 0
BASE 40 0 0 0 0 0 0 0 0 0
THM 41 0 0 0 0 0 0 0 0 0
3A-AE 42 686896 0 3 0 0 0 686896 3
3A-AWB 43 426204 0 5 0 0 0 426204 5
3A-AF 44 0 0 0 0 0 0 0 0 0
RESERV 45 0 0 0 0 0 0 0 0 0
All: 8610k(usr) 9040k(ker) 385(usr_times) 2935(ker_times)

```

### 13.2.3 模块的内核内存详细信息查看

如果需要查看某个模块的单个信息需要执行下面的命令：

```
# echo mod_id > /proc/ax_proc/os_mem/stat
```

其中 mod\_id 可以查看 /proc/ax\_proc/os\_mem/stat 文件中第二列某块对应的数字。

例如要查看 JENC 每次分配的内存大小和 ID，可以执行下面命令：

```
# echo 9 > /proc/ax_proc/os_mem/stat
```

```

/proc/ax_proc/os_mem # echo 9 > stat
/proc/ax_proc/os_mem # cat stat
(JENC) mod_id: 9, memory(kernel) stat

```

```

-----
      id      size
131081      4096
131081      4096
131081      4096
131081      4096
131081      4096
131081      4096
131081      4096
131081      4096

```

### 13.2.4 设置模块内核分配的次数显示限制

通过 13.2.3 的方法默认只显示 512 个，如果只想显示 10 次，可以通过下面的命令来限制显示的次数。

```
# echo 10 > /proc/ax_proc/os_mem/max_display
```

## 13.3 注意事项

### 13.3.1 单模块模式和 ALL 模式切换

在通过下面命令切换到单模块模式后，想再次查看所有模块的内存统计

```
#echo mod_id > /proc/ax_proc/os_mem/stat
```

需要执行下面命令切换到 ALL 模式。

```
#echo 255 > /proc/ax_proc/os_mem/stat
```

### 13.3.2 Policy 设置时机

为了使统计准确 policy 的设置时机做了限制：

- 1.在开机设置一次后不能再次设置；
- 2.在开机后执行 AX\_SYS\_Init 后不能再次设置；

### 13.3.3 Policy 选择

在 Debug 时可以使用 policy 1/2，方便分析每个模块的内存大小和次数，在最终产品建议使用 policy 0 模式，消耗的内存最小。

AEXRA CONFIDENTIAL FOR SIPEED

# 14 Recovery 升级使用说明

本章节主要介绍 recovery 升级功能的使用方法，以便用户在开发过程中了解升级的过程及注意事项。

## 14.1 Recovery 编译

目前版本 sdk AX620Q\_fastnor\_arm32\_k419 项目支持了 recovery 升级功能，可以实现本地升级和通过网络下载进行升级。recovery 是分区名称，实际烧录的是一个完整 OS 系统，编译生成的 bin 包含 kernel+rootfs+dtb 文件。

1. 使能 recovery 全局宏定义。

```
--- a/projects/AX620Q_fastnor_arm32_k419/project.mak
+++ b/projects/AX620Q_fastnor_arm32_k419/project.mak
--- a/projects/AX620Q_fastnor_arm32_k419/project.mak
+++ b/projects/AX620Q_fastnor_arm32_k419/project.mak
@@ -52,7 +52,7 @@ COPY_DWMAC_KO                := TRUE
AX_SPL_RISCV_SUPPORT                := TRUE
FDL2_UBOOT_COMPILE_INDEPENDENT := TRUE
COPY_SD_KO                          := TRUE
-SUPPORT_RECOVERY                   := FALSE
+Support Recovery                   := TRUE
```

2. 编译 recovery image 文件

```
cd kernel/linux/
```

```
make p=AX620Q_fastnor_arm32_k419 all install axp
```

！注意：

由于 fastnor 工程 riscv 增加 raw 检测操作，当 raw 检测被支持时，riscv 分区 size 需要从 256KB 增加到 2MB 用来存放检测模型，由于 nor flash 空间不足，导致 raw 检测被支持时，riscv 分区增大，recovery 功能将不被支持，如果确定需要支持 recovery 功能，需要调整 riscv partition size，关闭 raw 检测功能支持。这些配置都将在 project.mak 中配置。

## 14.2 Recovery 脚本说明

添加升级的分区信息：

```
projects/AX620Q_fastnor_arm32_k419/recovery/mkswu/sw-description-source
```

生成 xxxx.swu 的升级包，gz 的压缩格式：

```
projects/AX620Q_fastnor_arm32_k419/recovery/mkswu/swugenerator.sh
```

升级启动脚本，支持本地升级和 url 网络升级：

```
rootfs/arm/rootfs_recovery/usr/bin/swupdate.sh
```

切换 recovery 和 normal 系统：

```
projects/AX620Q_fastnor_arm32_k419/rootfs/usr/bin/bootable-set.sh
```

升级之后，重启 check recovery 升级的是否完成：

```
projects/AX620Q_fastnor_arm32_k419/rootfs/etc/init.d/S90checkrecovery
```

## 14.3 Recovery 升级方法说明

recovery 应用是基于开源项目 swupdate 进行开发的，并通过 swupdate.sh 脚本进行升级。

添加需要升级的 imge 文件到 sw-description-source 文件中：

```
{  
    filename = "AX620Q_fastnor_arm32_k419_signed.dtb";  
    device = "/dev/mtdblock6";  
    type = "raw";  
},
```

以上为一个升级分区，其中 filename 为升级固件名称，device 为升级固件对应的分区编号。

通过 build all 默认会将生成的打包文件添加到 out 目录，其中.swu 文件为升级文件；

单独编译 swu 文件，首先 cd build 进入 build 目录然后执行下面命令：

```
./projects/AX620Q_fastnor_arm32_k419/recovery/make_recovery_pkg.sh AX620Q_fastnor_arm32_k419
```

切换到 recovery 模式：

- 1) 执行命令 `bootable-set.sh recovery`;
- 2) 重启系统 `reboot`;

进入 recovery 模式后执行 `swupdate.sh` 脚本进行升级；

执行升级脚本方法：`swupdate.sh <parms>`

注意：

本地升级 `parms` 为本地 `swu` 文件的路径；

网络升级 `parms` 为 `url`，`url` 是新升级软件包更新的地址，是一个指向有效 `.swu` 文件的链接。

例如：

本地升级：`swupdate.sh -i /mnt/test.swu`

网络升级：`swupdate.sh -u 192.168.1.12:8000/test.swu`

升级完成后重启系统进行 `normal` 模式。

# 15 OTA 升级使用说明

本章节主要介绍 OTA 升级功能的使用方法，以便用户在开发过程中了解升级的过程及注意事项。

## 15.1 OTA 编译

### 15.1.1 依赖文件：

根据备注进行修改，其他文件无需修改。

```
build/projects/AX630C_emmc_arm32_k419/swupdate
├── hwrevision
├── sw-description-source #ota img 配置文件，需要修改
└── swupdate.sh #升级脚本，默认不修改，可以自定义
```

```
build/projects/${project}/rootfs/etc/fw_env.config
# 填充 env 挂载点，需要修改
```

```
third-party/swupdate/arm_uclibc/
├── bin
│   ├── fw_printenv
│   ├── fw_setenv -> fw_printenv
│   └── swupdate
├── etc
│   └── swupdate.cfg
└── lib
    ├── libconfig.so.11
    ├── libcurl.so.4
    └── libswupdate.so.0.1
```

```

├── libubootenv.so.0
└── libz.so.1

build/tools/swupdate
├── make_ota_pkg.sh
└── mkswu
    ├── ota.sh
    ├── priv.pem
    ├── public.pem
    ├── swugenerator.sh
    └── swupdate.cfg

```

### 15.1.2 打包 img:

修改 build/projects/\${project}/Makefile, 把 swupdate 依赖文件打包进预期分区

```

ifeq ($(ENABLE_SWUPDATE), TRUE)
    @cp -rf swupdate/mkswu/hwrevision ${DST_ROOTFS_DIR}/etc/hwrevision
    @cp -rf swupdate/mkswu/swupdate.cfg ${DST_ROOTFS_DIR}/etc/swupdate.cfg
endif

```

```

ifeq ($(ENABLE_SWUPDATE), TRUE)
    @mkdir -p ${DST_OPT_DIR}/swupdate/
    @cp -rf ${MSP_OUT_PATH}/third-party/swupdate/* ${DST_OPT_DIR}/swupdate/
    @cp -rf swupdate/swupdate.sh ${DST_OPT_DIR}/swupdate/bin
    # @cp -rf ${COMM_SWUPDATE_PATH}/mkswu/public.pem ${DST_OPT_DIR}/swupdate/
endif

```

生成 ota 包.swu 文件

```

ifeq ($(ENABLE_SWUPDATE), TRUE)
    @bash ${COMM_SWUPDATE_PATH}/make_ota_pkg.sh ${PROJECT}
endif

```

### 15.1.3 ota 升级:

#### 1. 本地升级:

- a) 把升级包文件挂载到板子上, 如/mnt/test.swu

- b) 根据 15.1.2 章节，找到 `swupdate.sh` 文件，执行如下命令

```
/opt/swupdate/bin/swupdate.sh -i /mnt/test.swu
```

## 2. URL 升级:

- a) 把 `swu` 文件放到远程服务器上（如 ip 为 192.168.1.12），在 `test.swu` 文件所在目录开启服务 `python -m http.server 8000`

- b) 根据 15.1.2 章节，找到 `swupdate.sh` 文件，执行如下命令

```
/opt/swupdate/bin/swupdate.sh -u 192.168.1.12:8000/test.swu
```

# 16 BOOTROM 相关介绍

## 16.1 chip mode 介绍

| chip_mode[3:0] = {strap_bond1, strap_bond0, strap_gpio3_a3, strap_gpio3_a2} |      |      |         |                             |
|-----------------------------------------------------------------------------|------|------|---------|-----------------------------|
| bit3                                                                        | bit2 | bit1 | bit0    | Boot Option                 |
| 0                                                                           | 1    | 1    | 1       | NAND 2K                     |
| 1                                                                           | 0    | 1    | 1       | NAND 4K                     |
| 1                                                                           | 1    | 1    | 1       | NOR                         |
| 0                                                                           | 0    | 0    | 1       | EMMC UDA                    |
| 0                                                                           | 1    | 0    | 1       | SPI SLAVE                   |
| X                                                                           | X    | X    | 0       | USB DL or SD Card or UART下载 |
| X                                                                           | X    | X    | X       | UART下载                      |
| 区分启动模式                                                                      |      |      | 区分usb下载 |                             |

由上图可以看出 AX620E 系列芯片支持的启动和下载模式，一般硬件设计前需要和硬件工程师确认 strap pin 的设置，进而让芯片进入预期的启动或者下载方式。

## 16.2 BOOTROM 涉及的字符含义介绍

```
static const char * log_id_to_string[LOG_PRINT_MAX] = {
    [LOG_UART_DOWNLOAD] = "UA",
    [LOG_USB_DOWNLOAD] = "US",
    [LOG_SPI_SLV_DOWNLOAD] = "SL",
    [LOG_SPI_MST_CS0_BOOT] = "C0",
    [LOG_SPI_MST_CS1_BOOT] = "C1",
    [LOG_EMMC_UDA_BOOT] = "EU",
    [LOG_NAND_2K_BOOT] = "N2",
    [LOG_NAND_4K_BOOT] = "N4",
    [LOG_NOR_BOOT] = "NO",
    [LOG_SD_BOOT] = "SD",
    [LOG_FLASH_INIT_FAIL] = "IF",
    [LOG_FLASH_READ_SUCC] = "RS",
    [LOG_FLASH_READ_FAIL] = "RF",
    [LOG_FLASH_CHKSUM_FAIL] = "CF",
    [LOG_RISCV_CHKSUM_FAIL] = "RCF",
    [LOG_SECURE_INIT_FAIL] = "SIF",
    [LOG_SECURE_PUBKEY_FAIL] = "PKF",
    [LOG_SECURE_SHA_FAIL] = "SHF",
    [LOG_SECURE_RSA_FAIL] = "RSF",
    [LOG_AES_DEC_FAIL] = "DF",
};
```

"UA":表示进入 UART 下载模式。

"US":表示进入 USB 下载模式。

"SL":表示进入 SPI SLV 下载模式。

"C0":表示使用 SPI CS0 进行启动。

"C1":表示使用 SPI CS1 进行启动。

"EU":表示使用 eMMC UDA 启动。

"N2":表示使用 Nand 2K page 启动。

"N4":表示使用 Nand 4K page 启动。

"NO":表示使用 Nor 启动。

"SD":表示使用 SD 卡启动。

"IF":表示 FLASH 初始化失败。

"RS":表示 FLASH 读取镜像成功。

"RF":表示 FLASH 读取镜像失败。

"CF":表示读取的数据 checksum 失败。

"RCF":表示读取的 RISCv 镜像 checksum 失败。

"SIF":表示 CE 模块初始化失败。

"PKF":表示校验 PUBLIC HASH 失败。

"SHF":表示 CE 模块计算 HASH 失败。

"RSF":表示 CE 模块使用 RSA 算法计算失败。

"DF":表示 CE 模块解密失败。